u-boot.lists.denx.de archive mirror
search help / color / mirror / Atom feed

*** [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage**
@ 2026-05-17 19:24 Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 01/18] clk: rockchip: rk3568: Fix building with OF_PLATDATA enabled Pavel Golikov
` (17 more replies)
0 siblings, 18 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
To: u-boot
Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
Pavel Golikov

The only missing piece to make rk356x platform completely free now is the source of DDR initialization BLOB provided by Rockchip. This series introduces DRAM controller driver that is ought to be run at TPL stage and initialize DRAM without any BLOBs. Having a DRAM driver in TPL can also let us do some VBE related experiments on rk356x platform, as complete TPL can be built from source.

DRAM initialization code was tested on rk3568 with 4GB of LPDDR4 dram, other configurations was not tested as all boards I have are rk3568 with the same memory configurations.

DRAM initialization should also work on rk3566, as the only difference between rk3566 and rk3588 is a maximum DDR operating frequency. So, the maximum frequency was limited by last patch to provide compatibility with rk3566.

Looks like rk3562 has the same memory controller and slightly upgraded PHY, so the driver code might be partially reused for rk3562 bringup (if it will ever be done).

The driver should also be able to initialize DDR4, but this part is untested because of lack of hardware.

LPDDR4x support is missing, but could be added with little effort (should work with LPDDR4 timings slightly patched at runtime, bit 16 of pctl's INIT4 register should be cleared).

The series enables LPDDR4 DRAM initialization for rk356x-generic board, other supported boards require CONFIG_RAM_ROCKCHIP_LPDDR4 or CONFIG_RAM_ROCKCHIP_DDR4 to be enabled to build properly.

A few questions:

The driver uses precalculated timings as most other Rockchip DRAM drivers do, however, timings calculator can be embedded directly into the driver at cost of TPL image size (we have 59.5Kb of SRAM, ~48Kb is used for now). This would allow dynamic DDR type detection (the same way as BLOB does). Should we stay with precalculated timings, or embed the calculator? Calculator code (with few missing bits) can be found in ARM Trusted Firmware sources [1].

What is the correct way to choose different maximum frequency for rk3568 and rk3566? One of possible way to implement this is to move frequency configuration to device tree. Should we choose different operating frequencies for different DDR types then?

I am not quite sure about sysreset-psci driver related patch. Are there any

boards that have PSCI services up and running at TPL or SPL?

I am also not quite sure about rockchip_uart driver in TPL. Should we have one? If we should, we also should have a (heavily cutoff) pinctrl driver that will configure pinmux accordingly?

Do we want to have DDR3 and LPDDR3 support? Is there any hardware with such configurations?

rk3568's DDR controller supports 4 memory ranks, but I've never seen hardware with such configuration either. Should we support this?

There is also commented-out SGRF manipulation code in rk3568_dmc_init function. Left that commented-out, as there is no public SGRF-related documentation, and DRAM is initialized well without setting those.

Testing on rk3568 and rk3566 with different DDR configurations is greatly appreciated.

Thanks.

[1] https://github.com/ayufan-rock64/arm-trusted-firmware/blob/f947c7e05a34db0c5b908a5347184fcaa9a32d95/plat/rockchip/rk322xh/drivers/dram/dram_spec_timing.c

---
Pavel Golikov (18):
      clk: rockchip: rk3568: Fix building with OF_PLATDATA enabled
      clk: rockchip: rk3568: Fix probe with OF_PLATDATA enabled
      rockchip: rk3568: syscon: Add PMUGRF support for OF_PLATDATA
      rockchip: rk3568: syscon: Add MSCH, SGRF and DDRGRF support
      ram: rockchip: Update ddr pctl regs for px30
      ram: rockchip: Add rk3568 lpddr4 support
      ram: rockchip: Add rk3568 ddr4 support
      ram: rockchip: Add rk3568 ddr loader params
      ram: rockchip: Add 4 ranks support to controller code for PX30
      ram: rockchip: Add missing bits to common code for sdram driver
      ram: rockchip: Add SDRAM initialization support for rk3568
      clk: rockchip: rk3568: Reduce code size for TPL build
      rockchip: rk356x: Add register spaces to DMC in u-boot.dtsi
      rockchip: rk356x: Add DMC related syscons to u-boot.dtsi
      sysreset: psci: Stop building for XPL
      rockchip: serial: Add driver alias for rk3568
      rockchip: rk356x-generic: Initialize SDRAM at TPL stage
      ram: rockchip: rk3568: Limit maximum DDR frequency to 1056 MHz

```
 arch/arm/dts/rk356x-u-boot.dtsi                     |   30 +-
 arch/arm/include/asm/arch-rockchip/clock.h          |    1 +
 arch/arm/include/asm/arch-rockchip/sdram_common.h   |   13 +-
 .../include/asm/arch-rockchip/sdram_pctl_px30.h     |   26 +-
 .../include/asm/arch-rockchip/sdram_phy_rk3568.h    |  549 +++
 .../asm/arch-rockchip/sdram_phy_ron_rtt_rk3568.h    |  209 +
 arch/arm/include/asm/arch-rockchip/sdram_rk3568.h   |  197 +
 arch/arm/mach-rockchip/Kconfig                      |   28 +-
 arch/arm/mach-rockchip/rk3568/Kconfig               |   16 +
 arch/arm/mach-rockchip/rk3568/syscon_rk3568.c       |   43 +-
 common/spl/Kconfig.tpl                              |    1 +
 configs/generic-rk3568_defconfig                    |    1 +
 drivers/clk/rockchip/clk_rk3568.c                   |   26 +-
 drivers/ram/rockchip/Makefile                       |    2 +-
 .../ram/rockchip/sdram-rk3568-ddr4-detect-1056.inc |   75 +
 .../ram/rockchip/sdram-rk3568-ddr4-detect-1184.inc |   75 +
 .../ram/rockchip/sdram-rk3568-ddr4-detect-1332.inc |   75 +
 .../ram/rockchip/sdram-rk3568-ddr4-detect-1560.inc |   75 +
 .../ram/rockchip/sdram-rk3568-ddr4-detect-324.inc  |   75 +
 .../ram/rockchip/sdram-rk3568-ddr4-detect-396.inc  |   75 +
```

```
 .../ram/rockchip/sdram-rk3568-ddr4-detect-528.inc  |   75 +
 .../ram/rockchip/sdram-rk3568-ddr4-detect-630.inc  |   75 +
 .../ram/rockchip/sdram-rk3568-ddr4-detect-780.inc  |   75 +
 .../ram/rockchip/sdram-rk3568-ddr4-detect-920.inc  |   75 +
 .../ram/rockchip/sdram-rk3568-loader_params.inc    |  109 +
 .../rockchip/sdram-rk3568-lpddr4-detect-1056.inc   |   79 +
 .../rockchip/sdram-rk3568-lpddr4-detect-1184.inc   |   79 +
 .../rockchip/sdram-rk3568-lpddr4-detect-1332.inc   |   79 +
 .../rockchip/sdram-rk3568-lpddr4-detect-1560.inc   |   79 +
 .../rockchip/sdram-rk3568-lpddr4-detect-324.inc    |   79 +
 .../rockchip/sdram-rk3568-lpddr4-detect-396.inc    |   79 +
 .../rockchip/sdram-rk3568-lpddr4-detect-528.inc    |   79 +
 .../rockchip/sdram-rk3568-lpddr4-detect-630.inc    |   79 +
 .../rockchip/sdram-rk3568-lpddr4-detect-780.inc    |   79 +
 .../rockchip/sdram-rk3568-lpddr4-detect-920.inc    |   79 +
 drivers/ram/rockchip/sdram_pctl_px30.c             |    2 +-
 drivers/ram/rockchip/sdram_rk3568.c                | 4384 +++++++++++++++++++-
 drivers/serial/serial_rockchip.c                   |    1 +
 drivers/sysreset/Makefile                          |    2 +-
 39 files changed, 7164 insertions(+), 16 deletions(-)
---
base-commit: 215496fec59b3fa09256b4fb62f92af46e2ec7f9
change-id: 20260517-rk3568-raminit-c6f93f937e54

Best regards,
--
Pavel Golikov <paullo612@ya.ru>
```

^ permalink raw reply [flat|nested] 22+ messages in thread

*** [PATCH RFC 01/18] clk: rockchip: rk3568: Fix building with OF_PLATDATA enabled**
2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 02/18] clk: rockchip: rk3568: Fix probe " Pavel Golikov
` (16 subsequent siblings)
17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
To: u-boot
Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
Pavel Golikov

```
Fix missing include statement.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 drivers/clk/rockchip/clk_rk3568.c | 1 +
 1 file changed, 1 insertion(+)

diff --git a/drivers/clk/rockchip/clk_rk3568.c b/drivers/clk/rockchip/clk_rk3568.c
index bb49af358e6..67bc04b81c0 100644
--- a/drivers/clk/rockchip/clk_rk3568.c
+++ b/drivers/clk/rockchip/clk_rk3568.c
@@ -7,6 +7,7 @@
 #include <bitfield.h>
 #include <clk-uclass.h>
 #include <dm.h>
+#include <dt-structs.h>
 #include <errno.h>
```

```
 #include <syscon.h>
 #include <asm/arch-rockchip/cru_rk3568.h>

--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** [PATCH RFC 02/18] clk: rockchip: rk3568: Fix probe with OF_PLATDATA enabled**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 01/18] clk: rockchip: rk3568: Fix building with OF_PLATDATA enabled Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 03/18] rockchip: rk3568: syscon: Add PMUGRF support for OF_PLATDATA Pavel Golikov
                  ` (15 subsequent siblings)
  17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

Properly initialize cru address space reference when CONFIG_OF_PLATDATA
is enabled.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 drivers/clk/rockchip/clk_rk3568.c | 13 +++++++++++--
 1 file changed, 11 insertions(+), 2 deletions(-)

diff --git a/drivers/clk/rockchip/clk_rk3568.c b/drivers/clk/rockchip/clk_rk3568.c
index 67bc04b81c0..199bb51fa6f 100644
--- a/drivers/clk/rockchip/clk_rk3568.c
+++ b/drivers/clk/rockchip/clk_rk3568.c
@@ -9,6 +9,7 @@
 #include <dm.h>
 #include <dt-structs.h>
 #include <errno.h>
+#include <mapmem.h>
 #include <syscon.h>
 #include <asm/arch-rockchip/cru_rk3568.h>
 #include <asm/arch-rockchip/clock.h>
@@ -2906,8 +2907,14 @@ static void rk3568_clk_init(struct rk3568_clk_priv *priv)
 static int rk3568_clk_probe(struct udevice *dev)
 {
        struct rk3568_clk_priv *priv = dev_get_priv(dev);
+#if CONFIG_IS_ENABLED(OF_PLATDATA)
+   struct rk3568_clk_plat *plat = dev_get_plat(dev);
+#endif
        int ret;

+#if CONFIG_IS_ENABLED(OF_PLATDATA)
+   priv->cru = map_sysmem(plat->dtd.reg[0], plat->dtd.reg[1]);
+#endif
        priv->grf = syscon_get_first_range(ROCKCHIP_SYSCON_GRF);
        if (IS_ERR(priv->grf))
                return PTR_ERR(priv->grf);
@@ -2926,9 +2933,11 @@ static int rk3568_clk_probe(struct udevice *dev)
```
```

```
 static int rk3568_clk_ofdata_to_platdata(struct udevice *dev)
 {
-       struct rk3568_clk_priv *priv = dev_get_priv(dev);
+       if (CONFIG_IS_ENABLED(OF_REAL)) {
+               struct rk3568_clk_priv *priv = dev_get_priv(dev);

-       priv->cru = dev_read_addr_ptr(dev);
+               priv->cru = dev_read_addr_ptr(dev);
+       }

        return 0;
 }

--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** [PATCH RFC 03/18] rockchip: rk3568: syscon: Add PMUGRF support for OF_PLATDATA**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 01/18] clk: rockchip: rk3568: Fix building with OF_PLATDATA enabled Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 02/18] clk: rockchip: rk3568: Fix probe " Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 04/18] rockchip: rk3568: syscon: Add MSCH, SGRF and DDRGRF support Pavel Golikov
                  ` (14 subsequent siblings)
  17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

The RK3568 has a limited TPL size, so, we expect RK3568 to always use
OF_PLATDATA for its TPL stage.

Add support for the PMUGRF register region in syscon, which is necessary
for RK3568's DRAM controller initialization.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 arch/arm/mach-rockchip/rk3568/syscon_rk3568.c | 17 +++++++++++++++++
 1 file changed, 17 insertions(+)

diff --git a/arch/arm/mach-rockchip/rk3568/syscon_rk3568.c b/arch/arm/mach-rockchip/rk3568/
syscon_rk3568.c
index 255259eabfd..b9d59a4721e 100644
--- a/arch/arm/mach-rockchip/rk3568/syscon_rk3568.c
+++ b/arch/arm/mach-rockchip/rk3568/syscon_rk3568.c
@@ -21,3 +21,20 @@ U_BOOT_DRIVER(syscon_rk3568) = {
        .bind = dm_scan_fdt_dev,
 #endif
 };
+
+#if CONFIG_IS_ENABLED(OF_PLATDATA)
+static int rk3568_syscon_bind_of_plat(struct udevice *dev)
+{
```
```

```
+       dev->driver_data = dev->driver->of_match->data;
+       debug("syscon: %s %d\n", dev->name, (uint)dev->driver_data);
+
+       return 0;
+}
+
+U_BOOT_DRIVER(rockchip_rk3568_pmugrf) = {
+       .name = "rockchip_rk3568_pmugrf",
+       .id = UCLASS_SYSCON,
+       .of_match = rk3568_syscon_ids + 1,
+       .bind = rk3568_syscon_bind_of_plat,
+};
+#endif

--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** [PATCH RFC 04/18] rockchip: rk3568: syscon: Add MSCH, SGRF and DDRGRF support**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
  ` (2 preceding siblings ...)
  2026-05-17 19:24 ` [PATCH RFC 03/18] rockchip: rk3568: syscon: Add PMUGRF support for OF_PLATDATA Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 20:42 ` Jonas Karlman
  2026-05-17 19:24 ` [PATCH RFC 05/18] ram: rockchip: Update ddr pctl regs for px30 Pavel Golikov
  ` (13 subsequent siblings)
  17 siblings, 1 reply; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

```
Add support for the MSCH, SGRF and DDRGRF register regions in syscon,
which are necessary for RK3568's DRAM controller initialization.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 arch/arm/include/asm/arch-rockchip/clock.h    |  1 +
 arch/arm/mach-rockchip/rk3568/syscon_rk3568.c | 26 ++++++++++++++++++++++++-
 2 files changed, 26 insertions(+), 1 deletion(-)

diff --git a/arch/arm/include/asm/arch-rockchip/clock.h b/arch/arm/include/asm/arch-rockchip/
clock.h
index 95b08bfd046..a6b399da4ff 100644
--- a/arch/arm/include/asm/arch-rockchip/clock.h
+++ b/arch/arm/include/asm/arch-rockchip/clock.h
@@ -39,6 +39,7 @@ enum {
        ROCKCHIP_SYSCON_PIPE_PHY2_GRF,
        ROCKCHIP_SYSCON_VOP_GRF,
        ROCKCHIP_SYSCON_VO_GRF,
+       ROCKCHIP_SYSCON_DDRGRF,
 };

 /* Standard Rockchip clock numbers */
diff --git a/arch/arm/mach-rockchip/rk3568/syscon_rk3568.c b/arch/arm/mach-rockchip/rk3568/
syscon_rk3568.c
```

```
index b9d59a4721e..7f6caf9ca67 100644
--- a/arch/arm/mach-rockchip/rk3568/syscon_rk3568.c
+++ b/arch/arm/mach-rockchip/rk3568/syscon_rk3568.c
@@ -8,8 +8,11 @@
 #include <asm/arch-rockchip/clock.h>

 static const struct udevice_id rk3568_syscon_ids[] = {
-       { .compatible = "rockchip,rk3568-grf", .data = ROCKCHIP_SYSCON_GRF },
+       { .compatible = "rockchip,rk3568-grf",    .data = ROCKCHIP_SYSCON_GRF    },
        { .compatible = "rockchip,rk3568-pmugrf", .data = ROCKCHIP_SYSCON_PMUGRF },
+       { .compatible = "rockchip,rk3568-msch",   .data = ROCKCHIP_SYSCON_MSCH   },
+       { .compatible = "rockchip,rk3568-sgrf",   .data = ROCKCHIP_SYSCON_SGRF   },
+       { .compatible = "rockchip,rk3568-ddrgrf", .data = ROCKCHIP_SYSCON_DDRGRF },
        { }
 };

@@ -37,4 +40,25 @@ U_BOOT_DRIVER(rockchip_rk3568_pmugrf) = {
        .of_match = rk3568_syscon_ids + 1,
        .bind = rk3568_syscon_bind_of_plat,
 };
+
+U_BOOT_DRIVER(rockchip_rk3568_msch) = {
+       .name = "rockchip_rk3568_msch",
+       .id = UCLASS_SYSCON,
+       .of_match = rk3568_syscon_ids + 2,
+       .bind = rk3568_syscon_bind_of_plat,
+};
+
+U_BOOT_DRIVER(rockchip_rk3568_sgrf) = {
+       .name = "rockchip_rk3568_sgrf",
+       .id = UCLASS_SYSCON,
+       .of_match = rk3568_syscon_ids + 3,
+       .bind = rk3568_syscon_bind_of_plat,
+};
+
+U_BOOT_DRIVER(rockchip_rk3568_ddrgrf) = {
+       .name = "rockchip_rk3568_ddrgrf",
+       .id = UCLASS_SYSCON,
+       .of_match = rk3568_syscon_ids + 4,
+       .bind = rk3568_syscon_bind_of_plat,
+};
 #endif

--
2.25.1
```

^ permalink raw reply related [**flat**|nested] 22+ messages in thread

---

*** [PATCH RFC 05/18] ram: rockchip: Update ddr pctl regs for px30**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
                  ` (3 preceding siblings ...)
  2026-05-17 19:24 ` [PATCH RFC 04/18] rockchip: rk3568: syscon: Add MSCH, SGRF and DDRGRF support Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 06/18] ram: rockchip: Add rk3568 lpddr4 support Pavel Golikov
                  ` (12 subsequent siblings)
  17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,

```
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

Add RK3568 specific bits.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 .../include/asm/arch-rockchip/sdram_pctl_px30.h    | 26 +++++++++++++++++++++--
 1 file changed, 24 insertions(+), 2 deletions(-)

diff --git a/arch/arm/include/asm/arch-rockchip/sdram_pctl_px30.h b/arch/arm/include/asm/arch-
rockchip/sdram_pctl_px30.h
index 3780dc6ea5b..62ea55df744 100644
--- a/arch/arm/include/asm/arch-rockchip/sdram_pctl_px30.h
+++ b/arch/arm/include/asm/arch-rockchip/sdram_pctl_px30.h
@@ -37,6 +37,7 @@ struct ddr_pctl_regs {
 #define DDR_PCTL2_RFSHTMG              0x64
 #define DDR_PCTL2_RFSHTMG1             0x68
 #define DDR_PCTL2_RFSHCTL5             0x6c
+#define DDR_PCTL2_ECCCFG0              0x70
 #define DDR_PCTL2_INIT0                        0xd0
 #define DDR_PCTL2_INIT1                        0xd4
 #define DDR_PCTL2_INIT2                        0xd8
@@ -127,10 +128,15 @@ struct ddr_pctl_regs {
        ((0x1000 * (n) + (((n) > 0) ? 0x1000 : 0)))

 /* PCTL2_MSTR */
-#define PCTL2_FREQUENCY_MODE_MASK      (1)
-#define PCTL2_FREQUENCY_MODE_SHIFT     (29)
+#define PCTL2_FREQUENCY_MODE_FREQ1     BIT(29)
 #define PCTL2_DLL_OFF_MODE             BIT(15)
 /* PCTL2_STAT */
+#define PCTL2_SELFREF_STATE_MASK       (3 << 8)
+#define PCTL2_SELFREF_STATE_SHIFT      (8)
+#define PCTL2_SELFREF_STATE_IDLE       (0)
+#define PCTL2_SELFREF_STATE_SR1                (1)
+#define PCTL2_SELFREF_STATE_SR_PD      (2)
+#define PCTL2_SELFREF_STATE_SR2                (3)
 #define PCTL2_SELFREF_TYPE_MASK                (3 << 4)
 #define PCTL2_SELFREF_TYPE_SR_NOT_AUTO (2 << 4)
 #define PCTL2_OPERATING_MODE_MASK      (7)
@@ -150,8 +156,11 @@ struct ddr_pctl_regs {
 /* PCTL2_MRSTAT */
 #define PCTL2_MR_WR_BUSY               BIT(0)
 /* PCTL2_DERATEEN */
+#define PCTL2_DERATE_MR4_PAUSE_FC      BIT(13)
+#define PCTL2_DERATE_MR4_TUF_DIS       BIT(12)
 #define PCTL2_DERATE_ENABLE            (1)
 /* PCTL2_PWRCTL */
+#define PCTL2_STAY_IN_SELFREF          BIT(6)
 #define PCTL2_SELFREF_SW               BIT(5)
 #define PCTL2_POWERDOWN_EN             BIT(1)
 #define PCTL2_SELFREF_EN               (1)
@@ -159,6 +168,12 @@ struct ddr_pctl_regs {
 #define PCTL2_SELFREF_TO_X32_MASK      (0xFF)
 #define PCTL2_SELFREF_TO_X32_SHIFT     (16)
 #define PCTL2_POWERDOWN_TO_X32_MASK    (0x1F)
+/* PCTL2_ECCCFG0*/
+#define PCTL2_ECC_MODE_MASK            (7)
+#define PCTL2_ECC_MODE_SHIFT           (0)
+#define PCTL2_ECC_MODE_DIS             (0)
+#define PCTL2_ECC_MODE_EN              (4)
+#define PCTL2_ECC_MODE_EN_ADVANCED     (5)
 /* PCTL2_INIT3 */
```

```
 #define PCTL2_DDR34_MR0_SHIFT          (16)
 #define PCTL2_LPDDR234_MR1_SHIFT       (16)
@@ -183,11 +198,16 @@ struct ddr_pctl_regs {

 #define PCTL2_MR_MASK                  (0xffff)

+/* PCTL2_RANKCTL */
+#define PCTL2_DIFF_RANK_WR_GAP_SHIFT   (8)
+#define PCTL2_DIFF_RANK_WR_GAP_MASK    (0xf << 8)
+
 /* PCTL2_RFSHCTL3 */
 #define PCTL2_DIS_AUTO_REFRESH         (1)
 /* PCTL2_ZQCTL0 */
 #define PCTL2_DIS_AUTO_ZQ              BIT(31)
 #define PCTL2_DIS_SRX_ZQCL             BIT(30)
+#define PCTL2_ZQ_RESISTOR_SHARED       BIT(29)
 /* PCTL2_DFILPCFG0 */
 #define PCTL2_DFI_LP_EN_SR             BIT(8)
 #define PCTL2_DFI_LP_EN_SR_MASK                BIT(8)
@@ -197,6 +217,8 @@ struct ddr_pctl_regs {
 /* PCTL2_DFISTAT */
 #define PCTL2_DFI_LP_ACK               BIT(1)
 #define PCTL2_DFI_INIT_COMPLETE                (1)
+/* DDR_PCTL2_SCHED */
+#define PCTL2_PAGECLOSE_EN             BIT(2)
 /* PCTL2_DBG1 */
 #define PCTL2_DIS_HIF                  BIT(1)
 /* PCTL2_DBGCAM */

--
2.25.1
```

^ permalink raw reply related [**flat**|nested] 22+ messages in thread

---

*** [PATCH RFC 06/18] ram: rockchip: Add rk3568 lpddr4 support**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
                  ` (4 preceding siblings ...)
  2026-05-17 19:24 ` [PATCH RFC 05/18] ram: rockchip: Update ddr pctl regs for px30 Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 07/18] ram: rockchip: Add rk3568 ddr4 support Pavel Golikov
                  ` (11 subsequent siblings)
  17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

Add LPDDR4 detection timings for Rockchip RK3568 SoC.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>

 .../rockchip/sdram-rk3568-lpddr4-detect-1056.inc | 79 +++++++++++++++++++++++
 .../rockchip/sdram-rk3568-lpddr4-detect-1184.inc | 79 +++++++++++++++++++++++
 .../rockchip/sdram-rk3568-lpddr4-detect-1332.inc | 79 +++++++++++++++++++++++
 .../rockchip/sdram-rk3568-lpddr4-detect-1560.inc | 79 +++++++++++++++++++++++
 .../rockchip/sdram-rk3568-lpddr4-detect-324.inc | 79 +++++++++++++++++++++++
 .../rockchip/sdram-rk3568-lpddr4-detect-396.inc | 79 +++++++++++++++++++++++
 .../rockchip/sdram-rk3568-lpddr4-detect-528.inc | 79 +++++++++++++++++++++++
```
```

```
 .../rockchip/sdram-rk3568-lpddr4-detect-630.inc    | 79 +++++++++++++++++++++++
 .../rockchip/sdram-rk3568-lpddr4-detect-780.inc    | 79 +++++++++++++++++++++++
 .../rockchip/sdram-rk3568-lpddr4-detect-920.inc    | 79 +++++++++++++++++++++++
 10 files changed, 790 insertions(+)

diff --git a/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-1056.inc b/drivers/ram/rockchip/
sdram-rk3568-lpddr4-detect-1056.inc
new file mode 100644
index 00000000000..fe9b5d92d08
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-1056.inc
@@ -0,0 +1,79 @@
+{
+       {
+               {
+                       .rank = 2,
+                       .col = 10,
+                       .bk = 3,
+                       .bw = 2,
+                       .dbw = 1,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 17,
+                       .cs0_high16bit_row = 0,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x40231420},
+                       {0x15050b07},
+                       {0x00000c04},
+                       {0x00000000},
+                       {0x00001111},
+                       {0x00000054},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 1056,       /* clock rate(MHz) */
+               .dramtype = LPDDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x8f080020},       /* MSTR */
+                       {0x00000064, 0x004000c9},       /* RFSHTMG */
+                       {0x00000068, 0x00300000},       /* RFSHTMG1 */
+                       {0x000000d0, 0x00030409},       /* INIT0 */
+                       {0x000000d4, 0x00690000},       /* INIT1 */
+                       {0x000000d8, 0x00000206},       /* INIT2 */
+                       {0x000000dc, 0x0034001b},       /* INIT3 */
+                       {0x000000e0, 0x00310000},       /* INIT4 */
+                       {0x000000e8, 0x00230000},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x14161117},       /* DRAMTMG0 */
+                       {0x00000104, 0x00040421},       /* DRAMTMG1 */
+                       {0x00000108, 0x050a0e0f},       /* DRAMTMG2 */
+                       {0x0000010c, 0x00808000},       /* DRAMTMG3 */
+                       {0x00000110, 0x0a04060a},       /* DRAMTMG4 */
+                       {0x00000114, 0x02040808},       /* DRAMTMG5 */
+                       {0x00000118, 0x01010005},       /* DRAMTMG6 */
+                       {0x0000011c, 0x00000401},       /* DRAMTMG7 */
```

```
+                       {0x00000120, 0x00000808},       /* DRAMTMG8 */
+                       {0x00000130, 0x00020000},       /* DRAMTMG12 */
+                       {0x00000134, 0x0a100002},       /* DRAMTMG13 */
+                       {0x00000138, 0x000000cd},       /* DRAMTMG14 */
+                       {0x00000180, 0x02100010},       /* ZQCTL0 */
+                       {0x00000184, 0x01b00000},       /* ZQCTL1 */
+                       {0x00000190, 0x07070001},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x0b050d3c},       /* ODTCFG */
+                       {0x00000244, 0x00000101},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fd7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x14000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x0a000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-1184.inc b/drivers/ram/rockchip/
sdram-rk3568-lpddr4-detect-1184.inc
new file mode 100644
index 00000000000..506871f27ce
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-1184.inc
@@ -0,0 +1,79 @@
+{
+       {
+               {
+                       .rank = 2,
+                       .col = 10,
+                       .bk = 3,
+                       .bw = 2,
+                       .dbw = 1,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 17,
+                       .cs0_high16bit_row = 0,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x44271724},
+                       {0x18060c09},
+                       {0x00000c04},
+                       {0x00000000},
+                       {0x00001111},
+                       {0x00000054},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 1184,       /* clock rate(MHz) */
+               .dramtype = LPDDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
```

```
+       {
+               {
+                       {0x00000000, 0x8f080020},       /* MSTR */
+                       {0x00000064, 0x004800e1},       /* RFSHTMG */
+                       {0x00000068, 0x00360000},       /* RFSHTMG1 */
+                       {0x000000d0, 0x00030486},       /* INIT0 */
+                       {0x000000d4, 0x00750000},       /* INIT1 */
+                       {0x000000d8, 0x00000206},       /* INIT2 */
+                       {0x000000dc, 0x00440024},       /* INIT3 */
+                       {0x000000e0, 0x00310000},       /* INIT4 */
+                       {0x000000e8, 0x00230000},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x16181319},       /* DRAMTMG0 */
+                       {0x00000104, 0x00050524},       /* DRAMTMG1 */
+                       {0x00000108, 0x060c1011},       /* DRAMTMG2 */
+                       {0x0000010c, 0x00909000},       /* DRAMTMG3 */
+                       {0x00000110, 0x0b04060b},       /* DRAMTMG4 */
+                       {0x00000114, 0x02050909},       /* DRAMTMG5 */
+                       {0x00000118, 0x01010006},       /* DRAMTMG6 */
+                       {0x0000011c, 0x00000501},       /* DRAMTMG7 */
+                       {0x00000120, 0x00000909},       /* DRAMTMG8 */
+                       {0x00000130, 0x00020000},       /* DRAMTMG12 */
+                       {0x00000134, 0x0b100002},       /* DRAMTMG13 */
+                       {0x00000138, 0x000000e6},       /* DRAMTMG14 */
+                       {0x00000180, 0x02500012},       /* ZQCTL0 */
+                       {0x00000184, 0x01e00000},       /* ZQCTL1 */
+                       {0x00000190, 0x07090002},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x0c060f48},       /* ODTCFG */
+                       {0x00000244, 0x00000101},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fd7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x18000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x0c000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-1332.inc b/drivers/ram/rockchip/
sdram-rk3568-lpddr4-detect-1332.inc
new file mode 100644
index 00000000000..1d2bc4c22f1
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-1332.inc
@@ -0,0 +1,79 @@
+{
+       {
+               {
+                       .rank = 2,
+                       .col = 10,
+                       .bk = 3,
+                       .bw = 2,
+                       .dbw = 1,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 17,
```

```
+                       .cs0_high16bit_row = 0,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x462a1928},
+                       {0x1b070d09},
+                       {0x00000c04},
+                       {0x00000000},
+                       {0x00001111},
+                       {0x00000054},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 1332,       /* clock rate(MHz) */
+               .dramtype = LPDDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x8f080020},       /* MSTR */
+                       {0x00000064, 0x005100fe},       /* RFSHTMG */
+                       {0x00000068, 0x003c0000},       /* RFSHTMG1 */
+                       {0x000000d0, 0x00030516},       /* INIT0 */
+                       {0x000000d4, 0x00840000},       /* INIT1 */
+                       {0x000000d8, 0x00000207},       /* INIT2 */
+                       {0x000000dc, 0x00440024},       /* INIT3 */
+                       {0x000000e0, 0x00310000},       /* INIT4 */
+                       {0x000000e8, 0x00230000},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x171b161c},       /* DRAMTMG0 */
+                       {0x00000104, 0x00050528},       /* DRAMTMG1 */
+                       {0x00000108, 0x060c1012},       /* DRAMTMG2 */
+                       {0x0000010c, 0x00a0a000},       /* DRAMTMG3 */
+                       {0x00000110, 0x0c04070c},       /* DRAMTMG4 */
+                       {0x00000114, 0x02050a0a},       /* DRAMTMG5 */
+                       {0x00000118, 0x01010006},       /* DRAMTMG6 */
+                       {0x0000011c, 0x00000501},       /* DRAMTMG7 */
+                       {0x00000120, 0x00000a0a},       /* DRAMTMG8 */
+                       {0x00000130, 0x00020000},       /* DRAMTMG12 */
+                       {0x00000134, 0x0b100002},       /* DRAMTMG13 */
+                       {0x00000138, 0x00000103},       /* DRAMTMG14 */
+                       {0x00000180, 0x029a0014},       /* ZQCTL0 */
+                       {0x00000184, 0x02200000},       /* ZQCTL1 */
+                       {0x00000190, 0x07090002},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x0c060f48},       /* ODTCFG */
+                       {0x00000244, 0x00000101},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fd7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x18000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x0c000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
```

```
+                }
+        }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-1560.inc b/drivers/ram/rockchip/
sdram-rk3568-lpddr4-detect-1560.inc
new file mode 100644
index 00000000000..1afaec6fe11
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-1560.inc
@@ -0,0 +1,79 @@
+{
+        {
+                {
+                        .rank = 2,
+                        .col = 10,
+                        .bk = 3,
+                        .bw = 2,
+                        .dbw = 1,
+                        .row_3_4 = 0,
+                        .cs0_row = 17,
+                        .cs1_row = 17,
+                        .cs0_high16bit_row = 0,
+                        .cs1_high16bit_row = 0,
+                        .ddrconfig = 0
+                },
+                {
+                        {0x4d321f2f},
+                        {0x1f080f0a},
+                        {0x00000c04},
+                        {0x00000000},
+                        {0x00001111},
+                        {0x00000054},
+                        0x000000ff
+                }
+        },
+        {
+                .ddr_freq = 1560,       /* clock rate(MHz) */
+                .dramtype = LPDDR4,
+                .num_channels = 1,
+                .stride = 0,
+                .odt = 0
+        },
+        {
+                {
+                        {0x00000000, 0x8f080020},       /* MSTR */
+                        {0x00000064, 0x005f0129},       /* RFSHTMG */
+                        {0x00000068, 0x00470000},       /* RFSHTMG1 */
+                        {0x000000d0, 0x000305f5},       /* INIT0 */
+                        {0x000000d4, 0x009a0000},       /* INIT1 */
+                        {0x000000d8, 0x00000208},       /* INIT2 */
+                        {0x000000dc, 0x0054002d},       /* INIT3 */
+                        {0x000000e0, 0x00310000},       /* INIT4 */
+                        {0x000000e8, 0x00230000},       /* INIT6 */
+                        {0x000000ec, 0x00000000},       /* INIT7 */
+                        {0x000000f4, 0x000f022f},       /* RANKCTL */
+                        {0x00000100, 0x1a201a21},       /* DRAMTMG0 */
+                        {0x00000104, 0x00060630},       /* DRAMTMG1 */
+                        {0x00000108, 0x070e1114},       /* DRAMTMG2 */
+                        {0x0000010c, 0x00b0b000},       /* DRAMTMG3 */
+                        {0x00000110, 0x0f04080f},       /* DRAMTMG4 */
+                        {0x00000114, 0x02060c0c},       /* DRAMTMG5 */
+                        {0x00000118, 0x01010007},       /* DRAMTMG6 */
+                        {0x0000011c, 0x00000601},       /* DRAMTMG7 */
+                        {0x00000120, 0x00000b0b},       /* DRAMTMG8 */
+                        {0x00000130, 0x00020000},       /* DRAMTMG12 */
```

```
+                       {0x00000134, 0x0c100002},       /* DRAMTMG13 */
+                       {0x00000138, 0x0000012f},       /* DRAMTMG14 */
+                       {0x00000180, 0x030c0018},       /* ZQCTL0 */
+                       {0x00000184, 0x02700000},       /* ZQCTL1 */
+                       {0x00000190, 0x070b0003},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x0d071154},       /* ODTCFG */
+                       {0x00000244, 0x00000101},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fd7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x1c000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x0e000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-324.inc b/drivers/ram/rockchip/
sdram-rk3568-lpddr4-detect-324.inc
new file mode 100644
index 00000000000..0b796f8a0ae
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-324.inc
@@ -0,0 +1,79 @@
+{
+       {
+               {
+                       .rank = 2,
+                       .col = 10,
+                       .bk = 3,
+                       .bw = 2,
+                       .dbw = 1,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 17,
+                       .cs0_high16bit_row = 0,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x2f0d060a},
+                       {0x06020804},
+                       {0x00000c04},
+                       {0x00000000},
+                       {0x00001111},
+                       {0x00000054},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 324,        /* clock rate(MHz) */
+               .dramtype = LPDDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
```

```
+                       {0x00000000, 0x8f080020},       /* MSTR */
+                       {0x00000064, 0x0013003e},       /* RFSHTMG */
+                       {0x00000068, 0x000f0000},       /* RFSHTMG1 */
+                       {0x000000d0, 0x0002013e},       /* INIT0 */
+                       {0x000000d4, 0x00210000},       /* INIT1 */
+                       {0x000000d8, 0x00000202},       /* INIT2 */
+                       {0x000000dc, 0x00240012},       /* INIT3 */
+                       {0x000000e0, 0x00310000},       /* INIT4 */
+                       {0x000000e8, 0x00000000},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x0c070507},       /* DRAMTMG0 */
+                       {0x00000104, 0x0003040a},       /* DRAMTMG1 */
+                       {0x00000108, 0x0407090d},       /* DRAMTMG2 */
+                       {0x0000010c, 0x00505000},       /* DRAMTMG3 */
+                       {0x00000110, 0x03040203},       /* DRAMTMG4 */
+                       {0x00000114, 0x02030303},       /* DRAMTMG5 */
+                       {0x00000118, 0x01010004},       /* DRAMTMG6 */
+                       {0x0000011c, 0x00000301},       /* DRAMTMG7 */
+                       {0x00000120, 0x00000303},       /* DRAMTMG8 */
+                       {0x00000130, 0x00020000},       /* DRAMTMG12 */
+                       {0x00000134, 0x00100002},       /* DRAMTMG13 */
+                       {0x00000138, 0x0000003f},       /* DRAMTMG14 */
+                       {0x00000180, 0x00a20005},       /* ZQCTL0 */
+                       {0x00000184, 0x00900000},       /* ZQCTL1 */
+                       {0x00000190, 0x07040000},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x0905092c},       /* ODTCFG */
+                       {0x00000244, 0x00000101},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fd7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x0e000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x08000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-396.inc b/drivers/ram/rockchip/
sdram-rk3568-lpddr4-detect-396.inc
new file mode 100644
index 00000000000..8f2a3191329
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-396.inc
@@ -0,0 +1,79 @@
+{
+       {
+               {
+                       .rank = 2,
+                       .col = 10,
+                       .bk = 3,
+                       .bw = 2,
+                       .dbw = 1,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 17,
+                       .cs0_high16bit_row = 0,
+                       .cs1_high16bit_row = 0,
```

```
+                       .ddrconfig = 0
+               },
+               {
+                       {0x3110080c},
+                       {0x80208004},
+                       {0x00000c04},
+                       {0x00000000},
+                       {0x00001111},
+                       {0x00000054},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 396,        /* clock rate(MHz) */
+               .dramtype = LPDDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x8f080020},       /* MSTR */
+                       {0x00000064, 0x0018004c},       /* RFSHTMG */
+                       {0x00000068, 0x00120000},       /* RFSHTMG1 */
+                       {0x000000d0, 0x00020184},       /* INIT0 */
+                       {0x000000d4, 0x00280000},       /* INIT1 */
+                       {0x000000d8, 0x00000202},       /* INIT2 */
+                       {0x000000dc, 0x00240012},       /* INIT3 */
+                       {0x000000e0, 0x00310000},       /* INIT4 */
+                       {0x000000e8, 0x00000000},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x0d080609},       /* DRAMTMG0 */
+                       {0x00000104, 0x0003040d},       /* DRAMTMG1 */
+                       {0x00000108, 0x0407090d},       /* DRAMTMG2 */
+                       {0x0000010c, 0x00505000},       /* DRAMTMG3 */
+                       {0x00000110, 0x04040204},       /* DRAMTMG4 */
+                       {0x00000114, 0x02030303},       /* DRAMTMG5 */
+                       {0x00000118, 0x01010004},       /* DRAMTMG6 */
+                       {0x0000011c, 0x00000301},       /* DRAMTMG7 */
+                       {0x00000120, 0x00000404},       /* DRAMTMG8 */
+                       {0x00000130, 0x00020000},       /* DRAMTMG12 */
+                       {0x00000134, 0x00100002},       /* DRAMTMG13 */
+                       {0x00000138, 0x0000004d},       /* DRAMTMG14 */
+                       {0x00000180, 0x00c60006},       /* ZQCTL0 */
+                       {0x00000184, 0x00a00000},       /* ZQCTL1 */
+                       {0x00000190, 0x07040000},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x0905092c},       /* ODTCFG */
+                       {0x00000244, 0x00000101},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fd7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x0e000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x08000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
```

```
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-528.inc b/drivers/ram/rockchip/
sdram-rk3568-lpddr4-detect-528.inc
new file mode 100644
index 00000000000..51c786e0ada
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-528.inc
@@ -0,0 +1,79 @@
+{
+       {
+               {
+                       .rank = 2,
+                       .col = 10,
+                       .bk = 3,
+                       .bw = 2,
+                       .dbw = 1,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 17,
+                       .cs0_high16bit_row = 0,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x33130a10},
+                       {0x0b030804},
+                       {0x00000c04},
+                       {0x00000000},
+                       {0x00001111},
+                       {0x00000054},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 528,        /* clock rate(MHz) */
+               .dramtype = LPDDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x8f080020},       /* MSTR */
+                       {0x00000064, 0x00200065},       /* RFSHTMG */
+                       {0x00000068, 0x00180000},       /* RFSHTMG1 */
+                       {0x000000d0, 0x00020205},       /* INIT0 */
+                       {0x000000d4, 0x00350000},       /* INIT1 */
+                       {0x000000d8, 0x00000203},       /* INIT2 */
+                       {0x000000dc, 0x00240012},       /* INIT3 */
+                       {0x000000e0, 0x00310000},       /* INIT4 */
+                       {0x000000e8, 0x00000000},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x0e0b080c},       /* DRAMTMG0 */
+                       {0x00000104, 0x00030411},       /* DRAMTMG1 */
+                       {0x00000108, 0x0407090d},       /* DRAMTMG2 */
+                       {0x0000010c, 0x00505000},       /* DRAMTMG3 */
+                       {0x00000110, 0x05040305},       /* DRAMTMG4 */
+                       {0x00000114, 0x02030404},       /* DRAMTMG5 */
+                       {0x00000118, 0x01010004},       /* DRAMTMG6 */
+                       {0x0000011c, 0x00000301},       /* DRAMTMG7 */
+                       {0x00000120, 0x00000505},       /* DRAMTMG8 */
+                       {0x00000130, 0x00020000},       /* DRAMTMG12 */
+                       {0x00000134, 0x00100002},       /* DRAMTMG13 */
+                       {0x00000138, 0x00000067},       /* DRAMTMG14 */
```

```
+                       {0x00000180, 0x01080008},       /* ZQCTL0 */
+                       {0x00000184, 0x00e00000},       /* ZQCTL1 */
+                       {0x00000190, 0x07040000},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x0905092c},       /* ODTCFG */
+                       {0x00000244, 0x00000101},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fd7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x0e000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x08000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-630.inc b/drivers/ram/rockchip/
sdram-rk3568-lpddr4-detect-630.inc
new file mode 100644
index 00000000000..70084613475
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-630.inc
@@ -0,0 +1,79 @@
+{
+       {
+               {
+                       .rank = 2,
+                       .col = 10,
+                       .bk = 3,
+                       .bw = 2,
+                       .dbw = 1,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 17,
+                       .cs0_high16bit_row = 0,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x35160c13},
+                       {0x0d030805},
+                       {0x00000c04},
+                       {0x00000000},
+                       {0x00001111},
+                       {0x00000054},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 630,        /* clock rate(MHz) */
+               .dramtype = LPDDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x8f080020},       /* MSTR */
+                       {0x00000064, 0x00260078},       /* RFSHTMG */
```

```
+                       {0x00000068, 0x001d0000},       /* RFSHTMG1 */
+                       {0x000000d0, 0x00020269},       /* INIT0 */
+                       {0x000000d4, 0x003f0000},       /* INIT1 */
+                       {0x000000d8, 0x00000204},       /* INIT2 */
+                       {0x000000dc, 0x00240012},       /* INIT3 */
+                       {0x000000e0, 0x00310000},       /* INIT4 */
+                       {0x000000e8, 0x00000000},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x0f0d0a0e},       /* DRAMTMG0 */
+                       {0x00000104, 0x00030414},       /* DRAMTMG1 */
+                       {0x00000108, 0x04070b0d},       /* DRAMTMG2 */
+                       {0x0000010c, 0x00505000},       /* DRAMTMG3 */
+                       {0x00000110, 0x06040406},       /* DRAMTMG4 */
+                       {0x00000114, 0x02030505},       /* DRAMTMG5 */
+                       {0x00000118, 0x01010004},       /* DRAMTMG6 */
+                       {0x0000011c, 0x00000301},       /* DRAMTMG7 */
+                       {0x00000120, 0x00000505},       /* DRAMTMG8 */
+                       {0x00000130, 0x00020000},       /* DRAMTMG12 */
+                       {0x00000134, 0x00100002},       /* DRAMTMG13 */
+                       {0x00000138, 0x0000007b},       /* DRAMTMG14 */
+                       {0x00000180, 0x013b000a},       /* ZQCTL0 */
+                       {0x00000184, 0x01000000},       /* ZQCTL1 */
+                       {0x00000190, 0x07040000},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x0a040b28},       /* ODTCFG */
+                       {0x00000244, 0x00000101},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fd7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x0e000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x08000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-780.inc b/drivers/ram/rockchip/
sdram-rk3568-lpddr4-detect-780.inc
new file mode 100644
index 00000000000..2df058f06fa
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-780.inc
@@ -0,0 +1,79 @@
+{
+       {
+               {
+                       .rank = 2,
+                       .col = 10,
+                       .bk = 3,
+                       .bw = 2,
+                       .dbw = 1,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 17,
+                       .cs0_high16bit_row = 0,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
```

```
+                {
+                        {0x381a0f18},
+                        {0x10040805},
+                        {0x00000c04},
+                        {0x00000000},
+                        {0x00001111},
+                        {0x00000054},
+                        0x000000ff
+                }
+        },
+        {
+                .ddr_freq = 780,        /* clock rate(MHz) */
+                .dramtype = LPDDR4,
+                .num_channels = 1,
+                .stride = 0,
+                .odt = 0
+        },
+        {
+                {
+                        {0x00000000, 0x8f080020},       /* MSTR */
+                        {0x00000064, 0x002f0095},       /* RFSHTMG */
+                        {0x00000068, 0x00240000},       /* RFSHTMG1 */
+                        {0x000000d0, 0x000202fb},       /* INIT0 */
+                        {0x000000d4, 0x004e0000},       /* INIT1 */
+                        {0x000000d8, 0x00000204},       /* INIT2 */
+                        {0x000000dc, 0x00240012},       /* INIT3 */
+                        {0x000000e0, 0x00310000},       /* INIT4 */
+                        {0x000000e8, 0x00000000},       /* INIT6 */
+                        {0x000000ec, 0x00000000},       /* INIT7 */
+                        {0x000000f4, 0x000f022f},       /* RANKCTL */
+                        {0x00000100, 0x10100c11},       /* DRAMTMG0 */
+                        {0x00000104, 0x00030418},       /* DRAMTMG1 */
+                        {0x00000108, 0x04070a0d},       /* DRAMTMG2 */
+                        {0x0000010c, 0x00606000},       /* DRAMTMG3 */
+                        {0x00000110, 0x08040408},       /* DRAMTMG4 */
+                        {0x00000114, 0x02030606},       /* DRAMTMG5 */
+                        {0x00000118, 0x01010004},       /* DRAMTMG6 */
+                        {0x0000011c, 0x00000301},       /* DRAMTMG7 */
+                        {0x00000120, 0x00000606},       /* DRAMTMG8 */
+                        {0x00000130, 0x00020000},       /* DRAMTMG12 */
+                        {0x00000134, 0x00100002},       /* DRAMTMG13 */
+                        {0x00000138, 0x00000098},       /* DRAMTMG14 */
+                        {0x00000180, 0x0186000c},       /* ZQCTL0 */
+                        {0x00000184, 0x01400000},       /* ZQCTL1 */
+                        {0x00000190, 0x07040000},       /* DFITMG0 */
+                        {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                        {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                        {0x00000240, 0x0a040b28},       /* ODTCFG */
+                        {0x00000244, 0x00000101},       /* ODTMAP */
+                        {0x00000250, 0x00001f00},       /* SCHED */
+                        {0x00000490, 0x00000001},       /* PCTRL_0 */
+                        {0xffffffff, 0xffffffff}
+                }
+        },
+        {
+                {
+                        {0x00000000, 0x00001fd7},       /* DDRPHY_REG0 */
+                        {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                        {0x0000000c, 0x0e000000},       /* DDRPHY_REG3 */
+                        {0x00000010, 0x08000000},       /* DDRPHY_REG4 */
+                        {0xffffffff, 0xffffffff}
+                }
+        }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-920.inc b/drivers/ram/rockchip/
```

```
sdram-rk3568-lpddr4-detect-920.inc
new file mode 100644
index 00000000000..f253e4ccf5b
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-lpddr4-detect-920.inc
@@ -0,0 +1,79 @@
+{
+       {
+               {
+                       .rank = 2,
+                       .col = 10,
+                       .bk = 3,
+                       .bw = 2,
+                       .dbw = 1,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 17,
+                       .cs0_high16bit_row = 0,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x3d1e111c},
+                       {0x12050a07},
+                       {0x00000c04},
+                       {0x00000000},
+                       {0x00001111},
+                       {0x00000054},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 920,        /* clock rate(MHz) */
+               .dramtype = LPDDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x8f080020},       /* MSTR */
+                       {0x00000064, 0x003800af},       /* RFSHTMG */
+                       {0x00000068, 0x002a0000},       /* RFSHTMG1 */
+                       {0x000000d0, 0x00020384},       /* INIT0 */
+                       {0x000000d4, 0x005b0000},       /* INIT1 */
+                       {0x000000d8, 0x00000205},       /* INIT2 */
+                       {0x000000dc, 0x0034001b},       /* INIT3 */
+                       {0x000000e0, 0x00310000},       /* INIT4 */
+                       {0x000000e8, 0x00230000},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x12130f14},       /* DRAMTMG0 */
+                       {0x00000104, 0x0004041c},       /* DRAMTMG1 */
+                       {0x00000108, 0x050a0e0f},       /* DRAMTMG2 */
+                       {0x0000010c, 0x00707000},       /* DRAMTMG3 */
+                       {0x00000110, 0x09040509},       /* DRAMTMG4 */
+                       {0x00000114, 0x02040707},       /* DRAMTMG5 */
+                       {0x00000118, 0x01010005},       /* DRAMTMG6 */
+                       {0x0000011c, 0x00000401},       /* DRAMTMG7 */
+                       {0x00000120, 0x00000707},       /* DRAMTMG8 */
+                       {0x00000130, 0x00020000},       /* DRAMTMG12 */
+                       {0x00000134, 0x0a100002},       /* DRAMTMG13 */
+                       {0x00000138, 0x000000b3},       /* DRAMTMG14 */
+                       {0x00000180, 0x01cc000e},       /* ZQCTL0 */
+                       {0x00000184, 0x01700000},       /* ZQCTL1 */
```

```
+                       {0x00000190, 0x07070001},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x0b050d3c},       /* ODTCFG */
+                       {0x00000244, 0x00000101},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fd7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x14000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x0a000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},

--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** [PATCH RFC 07/18] ram: rockchip: Add rk3568 ddr4 support**
2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
` (5 preceding siblings ...)
2026-05-17 19:24 ` [PATCH RFC 06/18] ram: rockchip: Add rk3568 lpddr4 support Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 08/18] ram: rockchip: Add rk3568 ddr loader params Pavel Golikov
` (10 subsequent siblings)
17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
To: u-boot
Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
Pavel Golikov

Add DDR4 detection timings for Rockchip RK3568 SoC.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>

```
 .../ram/rockchip/sdram-rk3568-ddr4-detect-1056.inc | 75 +++++++++++++++++++++++
 .../ram/rockchip/sdram-rk3568-ddr4-detect-1184.inc | 75 +++++++++++++++++++++++
 .../ram/rockchip/sdram-rk3568-ddr4-detect-1332.inc | 75 +++++++++++++++++++++++
 .../ram/rockchip/sdram-rk3568-ddr4-detect-1560.inc | 75 +++++++++++++++++++++++
 .../ram/rockchip/sdram-rk3568-ddr4-detect-324.inc  | 75 +++++++++++++++++++++++
 .../ram/rockchip/sdram-rk3568-ddr4-detect-396.inc  | 75 +++++++++++++++++++++++
 .../ram/rockchip/sdram-rk3568-ddr4-detect-528.inc  | 75 +++++++++++++++++++++++
 .../ram/rockchip/sdram-rk3568-ddr4-detect-630.inc  | 75 +++++++++++++++++++++++
 .../ram/rockchip/sdram-rk3568-ddr4-detect-780.inc  | 75 +++++++++++++++++++++++
 .../ram/rockchip/sdram-rk3568-ddr4-detect-920.inc  | 75 +++++++++++++++++++++++
 10 files changed, 750 insertions(+)
```

diff --git a/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-1056.inc b/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-1056.inc
new file mode 100644

```
index 00000000000..55fb5d8e376
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-1056.inc
@@ -0,0 +1,75 @@
+{
+       {
+               {
+                       .rank = 1,
+                       .col = 10,
+                       .bk = 2,
+                       .bw = 1,
+                       .dbw = 0,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 0,
+                       .cs0_high16bit_row = 17,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x561d1219},
+                       {0x0b020703},
+                       {0x00000002},
+                       {0x0000023a},
+                       {0x00001111},
+                       {0x0000000c},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 1056,       /* clock rate(MHz) */
+               .dramtype = DDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x4f041010},       /* MSTR */
+                       {0x00000064, 0x008000b9},       /* RFSHTMG */
+                       {0x000000d0, 0x00020103},       /* INIT0 */
+                       {0x000000d4, 0x00690000},       /* INIT1 */
+                       {0x000000d8, 0x00000100},       /* INIT2 */
+                       {0x000000dc, 0x07340201},       /* INIT3 */
+                       {0x000000e0, 0x00100000},       /* INIT4 */
+                       {0x000000e4, 0x00110000},       /* INIT5 */
+                       {0x000000e8, 0x00000420},       /* INIT6 */
+                       {0x000000ec, 0x00000800},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x0f0c2312},       /* DRAMTMG0 */
+                       {0x00000104, 0x0004041a},       /* DRAMTMG1 */
+                       {0x00000108, 0x0608060c},       /* DRAMTMG2 */
+                       {0x0000010c, 0x0040400c},       /* DRAMTMG3 */
+                       {0x00000110, 0x08030309},       /* DRAMTMG4 */
+                       {0x00000114, 0x06060403},       /* DRAMTMG5 */
+                       {0x00000120, 0x07070d07},       /* DRAMTMG8 */
+                       {0x00000124, 0x00020209},       /* DRAMTMG9 */
+                       {0x00000180, 0x01000040},       /* ZQCTL0 */
+                       {0x00000184, 0x00000000},       /* ZQCTL1 */
+                       {0x00000190, 0x07060004},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x06000614},       /* ODTCFG */
+                       {0x00000244, 0x00000201},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
```

```
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fc7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x10000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x0b000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-1184.inc b/drivers/ram/rockchip/sdram-
rk3568-ddr4-detect-1184.inc
new file mode 100644
index 00000000000..838475cb95f
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-1184.inc
@@ -0,0 +1,75 @@
+{
+       {
+               {
+                       .rank = 1,
+                       .col = 10,
+                       .bk = 2,
+                       .bw = 1,
+                       .dbw = 0,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 0,
+                       .cs0_high16bit_row = 17,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x5921141c},
+                       {0x0c020704},
+                       {0x00000002},
+                       {0x0000023a},
+                       {0x00001111},
+                       {0x0000000c},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 1184,       /* clock rate(MHz) */
+               .dramtype = DDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x4f041010},       /* MSTR */
+                       {0x00000064, 0x009000d0},       /* RFSHTMG */
+                       {0x000000d0, 0x00020123},       /* INIT0 */
+                       {0x000000d4, 0x00750000},       /* INIT1 */
+                       {0x000000d8, 0x00000100},       /* INIT2 */
+                       {0x000000dc, 0x09400201},       /* INIT3 */
+                       {0x000000e0, 0x00180000},       /* INIT4 */
+                       {0x000000e4, 0x00110000},       /* INIT5 */
+                       {0x000000e8, 0x00000420},       /* INIT6 */
+                       {0x000000ec, 0x00000800},       /* INIT7 */
```

```
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x110d2813},       /* DRAMTMG0 */
+                       {0x00000104, 0x0004041c},       /* DRAMTMG1 */
+                       {0x00000108, 0x0609060d},       /* DRAMTMG2 */
+                       {0x0000010c, 0x0040400c},       /* DRAMTMG3 */
+                       {0x00000110, 0x0903030a},       /* DRAMTMG4 */
+                       {0x00000114, 0x06060403},       /* DRAMTMG5 */
+                       {0x00000120, 0x08080d08},       /* DRAMTMG8 */
+                       {0x00000124, 0x0002020a},       /* DRAMTMG9 */
+                       {0x00000180, 0x01000040},       /* ZQCTL0 */
+                       {0x00000184, 0x00000000},       /* ZQCTL1 */
+                       {0x00000190, 0x07070004},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x06000618},       /* ODTCFG */
+                       {0x00000244, 0x00000201},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fc7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x12000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x0c000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-1332.inc b/drivers/ram/rockchip/sdram-
rk3568-ddr4-detect-1332.inc
new file mode 100644
index 00000000000..4fb1ba95e82
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-1332.inc
@@ -0,0 +1,75 @@
+{
+       {
+               {
+                       .rank = 1,
+                       .col = 10,
+                       .bk = 2,
+                       .bw = 1,
+                       .dbw = 0,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 0,
+                       .cs0_high16bit_row = 17,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x5c25171f},
+                       {0x0e020904},
+                       {0x00000002},
+                       {0x0000024a},
+                       {0x00001111},
+                       {0x0000000c},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 1332,       /* clock rate(MHz) */
```

```
+                .dramtype = DDR4,
+                .num_channels = 1,
+                .stride = 0,
+                .odt = 0
+        },
+        {
+                {
+                        {0x00000000, 0x4f041010},        /* MSTR */
+                        {0x00000064, 0x00a200ea},        /* RFSHTMG */
+                        {0x000000d0, 0x00020147},        /* INIT0 */
+                        {0x000000d4, 0x00840000},        /* INIT1 */
+                        {0x000000d8, 0x00000100},        /* INIT2 */
+                        {0x000000dc, 0x0b440201},        /* INIT3 */
+                        {0x000000e0, 0x00200000},        /* INIT4 */
+                        {0x000000e4, 0x00110000},        /* INIT5 */
+                        {0x000000e8, 0x00000420},        /* INIT6 */
+                        {0x000000ec, 0x00000c00},        /* INIT7 */
+                        {0x000000f4, 0x000f022f},        /* RANKCTL */
+                        {0x00000100, 0x130e2d16},        /* DRAMTMG0 */
+                        {0x00000104, 0x00040520},        /* DRAMTMG1 */
+                        {0x00000108, 0x070a060e},        /* DRAMTMG2 */
+                        {0x0000010c, 0x0040400c},        /* DRAMTMG3 */
+                        {0x00000110, 0x0a04040b},        /* DRAMTMG4 */
+                        {0x00000114, 0x07070404},        /* DRAMTMG5 */
+                        {0x00000120, 0x09091109},        /* DRAMTMG8 */
+                        {0x00000124, 0x0002020b},        /* DRAMTMG9 */
+                        {0x00000180, 0x01000040},        /* ZQCTL0 */
+                        {0x00000184, 0x00000000},        /* ZQCTL1 */
+                        {0x00000190, 0x07080005},        /* DFITMG0 */
+                        {0x00000198, 0x0a000101},        /* DFILPCFG0 */
+                        {0x000001a0, 0xc0400003},        /* DFIUPD0 */
+                        {0x00000240, 0x06000618},        /* ODTCFG */
+                        {0x00000244, 0x00000201},        /* ODTMAP */
+                        {0x00000250, 0x00001f00},        /* SCHED */
+                        {0x00000490, 0x00000001},        /* PCTRL_0 */
+                        {0xffffffff, 0xffffffff}
+                }
+        },
+        {
+                {
+                        {0x00000000, 0x00001fc7},        /* DDRPHY_REG0 */
+                        {0x00000008, 0x00000000},        /* DDRPHY_REG2 */
+                        {0x0000000c, 0x14000000},        /* DDRPHY_REG3 */
+                        {0x00000010, 0x0e000000},        /* DDRPHY_REG4 */
+                        {0xffffffff, 0xffffffff}
+                }
+        }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-1560.inc b/drivers/ram/rockchip/sdram-
rk3568-ddr4-detect-1560.inc
new file mode 100644
index 00000000000..8643e96f18f
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-1560.inc
@@ -0,0 +1,75 @@
+{
+        {
+                {
+                        .rank = 1,
+                        .col = 10,
+                        .bk = 2,
+                        .bw = 1,
+                        .dbw = 0,
+                        .row_3_4 = 0,
+                        .cs0_row = 17,
```

```
+                       .cs1_row = 0,
+                       .cs0_high16bit_row = 17,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x622c1c25},
+                       {0x10020a05},
+                       {0x00000002},
+                       {0x00000252},
+                       {0x00001111},
+                       {0x0000000c},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 1560,       /* clock rate(MHz) */
+               .dramtype = DDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x4f041010},       /* MSTR */
+                       {0x00000064, 0x00be0111},       /* RFSHTMG */
+                       {0x000000d0, 0x0002017e},       /* INIT0 */
+                       {0x000000d4, 0x009a0000},       /* INIT1 */
+                       {0x000000d8, 0x00000100},       /* INIT2 */
+                       {0x000000dc, 0x0d540201},       /* INIT3 */
+                       {0x000000e0, 0x00280000},       /* INIT4 */
+                       {0x000000e4, 0x00110000},       /* INIT5 */
+                       {0x000000e8, 0x00000420},       /* INIT6 */
+                       {0x000000ec, 0x00001000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x16113419},       /* DRAMTMG0 */
+                       {0x00000104, 0x00050625},       /* DRAMTMG1 */
+                       {0x00000108, 0x080c0710},       /* DRAMTMG2 */
+                       {0x0000010c, 0x0040400c},       /* DRAMTMG3 */
+                       {0x00000110, 0x0c04040d},       /* DRAMTMG4 */
+                       {0x00000114, 0x08080504},       /* DRAMTMG5 */
+                       {0x00000120, 0x0a0a110a},       /* DRAMTMG8 */
+                       {0x00000124, 0x0002020c},       /* DRAMTMG9 */
+                       {0x00000180, 0x01000040},       /* ZQCTL0 */
+                       {0x00000184, 0x00000000},       /* ZQCTL1 */
+                       {0x00000190, 0x070a0006},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x06000620},       /* ODTCFG */
+                       {0x00000244, 0x00000201},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fc7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x18000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x10000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
```

```
diff --git a/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-324.inc b/drivers/ram/rockchip/sdram-
rk3568-ddr4-detect-324.inc
new file mode 100644
index 00000000000..eed2e1948b3
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-324.inc
@@ -0,0 +1,75 @@
+{
+       {
+               {
+                       .rank = 1,
+                       .col = 10,
+                       .bk = 2,
+                       .bw = 1,
+                       .dbw = 0,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 0,
+                       .cs0_high16bit_row = 17,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x4d110a08},
+                       {0x0a020501},
+                       {0x00000002},
+                       {0x0000022a},
+                       {0x00001111},
+                       {0x0000000c},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 324,        /* clock rate(MHz) */
+               .dramtype = DDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x4f049010},       /* MSTR */
+                       {0x00000064, 0x00270039},       /* RFSHTMG */
+                       {0x000000d0, 0x00020051},       /* INIT0 */
+                       {0x000000d4, 0x00210000},       /* INIT1 */
+                       {0x000000d8, 0x00000100},       /* INIT2 */
+                       {0x000000dc, 0x00040000},       /* INIT3 */
+                       {0x000000e0, 0x00000000},       /* INIT4 */
+                       {0x000000e4, 0x00110000},       /* INIT5 */
+                       {0x000000e8, 0x00000420},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x090a0a06},       /* DRAMTMG0 */
+                       {0x00000104, 0x00020209},       /* DRAMTMG1 */
+                       {0x00000108, 0x05050409},       /* DRAMTMG2 */
+                       {0x0000010c, 0x0040400c},       /* DRAMTMG3 */
+                       {0x00000110, 0x05020206},       /* DRAMTMG4 */
+                       {0x00000114, 0x03030202},       /* DRAMTMG5 */
+                       {0x00000120, 0x03030b03},       /* DRAMTMG8 */
+                       {0x00000124, 0x00020208},       /* DRAMTMG9 */
+                       {0x00000180, 0x01000040},       /* ZQCTL0 */
+                       {0x00000184, 0x00000000},       /* ZQCTL1 */
+                       {0x00000190, 0x07030003},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
```

```
+                       {0x00000240, 0x06000604},       /* ODTCFG */
+                       {0x00000244, 0x00000201},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fc7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x0a000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x09000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-396.inc b/drivers/ram/rockchip/sdram-
rk3568-ddr4-detect-396.inc
new file mode 100644
index 00000000000..f3090af646f
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-396.inc
@@ -0,0 +1,75 @@
+{
+       {
+               {
+                       .rank = 1,
+                       .col = 10,
+                       .bk = 2,
+                       .bw = 1,
+                       .dbw = 0,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 0,
+                       .cs0_high16bit_row = 17,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x4d110a0a},
+                       {0x0a020501},
+                       {0x00000002},
+                       {0x0000022a},
+                       {0x00001111},
+                       {0x0000000c},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 396,        /* clock rate(MHz) */
+               .dramtype = DDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x4f049010},       /* MSTR */
+                       {0x00000064, 0x00300046},       /* RFSHTMG */
+                       {0x000000d0, 0x00020062},       /* INIT0 */
+                       {0x000000d4, 0x00280000},       /* INIT1 */
+                       {0x000000d8, 0x00000100},       /* INIT2 */
+                       {0x000000dc, 0x00040000},       /* INIT3 */
+                       {0x000000e0, 0x00000000},       /* INIT4 */
```

```
+                       {0x000000e4, 0x00110000},       /* INIT5 */
+                       {0x000000e8, 0x00000420},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x090a0d07},       /* DRAMTMG0 */
+                       {0x00000104, 0x0002020a},       /* DRAMTMG1 */
+                       {0x00000108, 0x05050409},       /* DRAMTMG2 */
+                       {0x0000010c, 0x0040400c},       /* DRAMTMG3 */
+                       {0x00000110, 0x05020206},       /* DRAMTMG4 */
+                       {0x00000114, 0x03030202},       /* DRAMTMG5 */
+                       {0x00000120, 0x04040b04},       /* DRAMTMG8 */
+                       {0x00000124, 0x00020208},       /* DRAMTMG9 */
+                       {0x00000180, 0x01000040},       /* ZQCTL0 */
+                       {0x00000184, 0x00000000},       /* ZQCTL1 */
+                       {0x00000190, 0x07030003},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x06000604},       /* ODTCFG */
+                       {0x00000244, 0x00000201},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fc7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x0a000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x09000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-528.inc b/drivers/ram/rockchip/sdram-
rk3568-ddr4-detect-528.inc
new file mode 100644
index 00000000000..2ddf374880f
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-528.inc
@@ -0,0 +1,75 @@
+{
+       {
+               {
+                       .rank = 1,
+                       .col = 10,
+                       .bk = 2,
+                       .bw = 1,
+                       .dbw = 0,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 0,
+                       .cs0_high16bit_row = 17,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x4d120a0d},
+                       {0x0a020501},
+                       {0x00000002},
+                       {0x0000022a},
+                       {0x00001111},
+                       {0x0000000c},
+                       0x000000ff
+               }
```

```
+       },
+       {
+               .ddr_freq = 528,        /* clock rate(MHz) */
+               .dramtype = DDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x4f049010},       /* MSTR */
+                       {0x00000064, 0x0040005d},       /* RFSHTMG */
+                       {0x000000d0, 0x00020082},       /* INIT0 */
+                       {0x000000d4, 0x00350000},       /* INIT1 */
+                       {0x000000d8, 0x00000100},       /* INIT2 */
+                       {0x000000dc, 0x00040000},       /* INIT3 */
+                       {0x000000e0, 0x00000000},       /* INIT4 */
+                       {0x000000e4, 0x00110000},       /* INIT5 */
+                       {0x000000e8, 0x00000420},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x0a0a110a},       /* DRAMTMG0 */
+                       {0x00000104, 0x0002020e},       /* DRAMTMG1 */
+                       {0x00000108, 0x05050409},       /* DRAMTMG2 */
+                       {0x0000010c, 0x0040400c},       /* DRAMTMG3 */
+                       {0x00000110, 0x05020206},       /* DRAMTMG4 */
+                       {0x00000114, 0x03030202},       /* DRAMTMG5 */
+                       {0x00000120, 0x04040b04},       /* DRAMTMG8 */
+                       {0x00000124, 0x00020208},       /* DRAMTMG9 */
+                       {0x00000180, 0x01000040},       /* ZQCTL0 */
+                       {0x00000184, 0x00000000},       /* ZQCTL1 */
+                       {0x00000190, 0x07030003},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x06000604},       /* ODTCFG */
+                       {0x00000244, 0x00000201},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fc7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x0a000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x09000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-630.inc b/drivers/ram/rockchip/sdram-
rk3568-ddr4-detect-630.inc
new file mode 100644
index 00000000000..659e38ab74e
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-630.inc
@@ -0,0 +1,75 @@
+{
+       {
+               {
+                       .rank = 1,
+                       .col = 10,
+                       .bk = 2,
+                       .bw = 1,
```

```
+                       .dbw = 0,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 0,
+                       .cs0_high16bit_row = 17,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x4d130a10},
+                       {0x0a020501},
+                       {0x00000002},
+                       {0x0000022a},
+                       {0x00001111},
+                       {0x0000000c},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 630,        /* clock rate(MHz) */
+               .dramtype = DDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x4f041010},       /* MSTR */
+                       {0x00000064, 0x004c006f},       /* RFSHTMG */
+                       {0x000000d0, 0x0002009b},       /* INIT0 */
+                       {0x000000d4, 0x003f0000},       /* INIT1 */
+                       {0x000000d8, 0x00000100},       /* INIT2 */
+                       {0x000000dc, 0x01040201},       /* INIT3 */
+                       {0x000000e0, 0x00000000},       /* INIT4 */
+                       {0x000000e4, 0x00110000},       /* INIT5 */
+                       {0x000000e8, 0x00000420},       /* INIT6 */
+                       {0x000000ec, 0x00000000},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x0b0a150c},       /* DRAMTMG0 */
+                       {0x00000104, 0x00020210},       /* DRAMTMG1 */
+                       {0x00000108, 0x05050409},       /* DRAMTMG2 */
+                       {0x0000010c, 0x0040400c},       /* DRAMTMG3 */
+                       {0x00000110, 0x05020206},       /* DRAMTMG4 */
+                       {0x00000114, 0x04040302},       /* DRAMTMG5 */
+                       {0x00000120, 0x05050b05},       /* DRAMTMG8 */
+                       {0x00000124, 0x00020208},       /* DRAMTMG9 */
+                       {0x00000180, 0x01000040},       /* ZQCTL0 */
+                       {0x00000184, 0x00000000},       /* ZQCTL1 */
+                       {0x00000190, 0x07030003},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x06000604},       /* ODTCFG */
+                       {0x00000244, 0x00000201},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fc7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x0a000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x09000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
```

```
+                }
+        }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-780.inc b/drivers/ram/rockchip/sdram-
rk3568-ddr4-detect-780.inc
new file mode 100644
index 00000000000..e7c6d75c528
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-780.inc
@@ -0,0 +1,75 @@
+{
+        {
+                {
+                        .rank = 1,
+                        .col = 10,
+                        .bk = 2,
+                        .bw = 1,
+                        .dbw = 0,
+                        .row_3_4 = 0,
+                        .cs0_row = 17,
+                        .cs1_row = 0,
+                        .cs0_high16bit_row = 17,
+                        .cs1_high16bit_row = 0,
+                        .ddrconfig = 0
+                },
+                {
+                        {0x50160d13},
+                        {0x0a020502},
+                        {0x00000002},
+                        {0x0000022a},
+                        {0x00001111},
+                        {0x0000000c},
+                        0x000000ff
+                }
+        },
+        {
+                .ddr_freq = 780,        /* clock rate(MHz) */
+                .dramtype = DDR4,
+                .num_channels = 1,
+                .stride = 0,
+                .odt = 0
+        },
+        {
+                {
+                        {0x00000000, 0x4f041010},       /* MSTR */
+                        {0x00000064, 0x005f0089},       /* RFSHTMG */
+                        {0x000000d0, 0x000200c0},       /* INIT0 */
+                        {0x000000d4, 0x004e0000},       /* INIT1 */
+                        {0x000000d8, 0x00000100},       /* INIT2 */
+                        {0x000000dc, 0x03140201},       /* INIT3 */
+                        {0x000000e0, 0x00000000},       /* INIT4 */
+                        {0x000000e4, 0x00110000},       /* INIT5 */
+                        {0x000000e8, 0x00000420},       /* INIT6 */
+                        {0x000000ec, 0x00000400},       /* INIT7 */
+                        {0x000000f4, 0x000f022f},       /* RANKCTL */
+                        {0x00000100, 0x0c0a1a0e},       /* DRAMTMG0 */
+                        {0x00000104, 0x00030314},       /* DRAMTMG1 */
+                        {0x00000108, 0x0506050a},       /* DRAMTMG2 */
+                        {0x0000010c, 0x0040400c},       /* DRAMTMG3 */
+                        {0x00000110, 0x06030307},       /* DRAMTMG4 */
+                        {0x00000114, 0x04040302},       /* DRAMTMG5 */
+                        {0x00000120, 0x06060b06},       /* DRAMTMG8 */
+                        {0x00000124, 0x00020208},       /* DRAMTMG9 */
+                        {0x00000180, 0x01000040},       /* ZQCTL0 */
+                        {0x00000184, 0x00000000},       /* ZQCTL1 */
```

```
+                       {0x00000190, 0x07040003},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x0600060c},       /* ODTCFG */
+                       {0x00000244, 0x00000201},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fc7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x0c000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x09000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},
diff --git a/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-920.inc b/drivers/ram/rockchip/sdram-
rk3568-ddr4-detect-920.inc
new file mode 100644
index 00000000000..0deab38f720
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-ddr4-detect-920.inc
@@ -0,0 +1,75 @@
+{
+       {
+               {
+                       .rank = 1,
+                       .col = 10,
+                       .bk = 2,
+                       .bw = 1,
+                       .dbw = 0,
+                       .row_3_4 = 0,
+                       .cs0_row = 17,
+                       .cs1_row = 0,
+                       .cs0_high16bit_row = 17,
+                       .cs1_high16bit_row = 0,
+                       .ddrconfig = 0
+               },
+               {
+                       {0x531a0f17},
+                       {0x0b020603},
+                       {0x00000002},
+                       {0x00000232},
+                       {0x00001111},
+                       {0x0000000c},
+                       0x000000ff
+               }
+       },
+       {
+               .ddr_freq = 920,        /* clock rate(MHz) */
+               .dramtype = DDR4,
+               .num_channels = 1,
+               .stride = 0,
+               .odt = 0
+       },
+       {
+               {
+                       {0x00000000, 0x4f041010},       /* MSTR */
+                       {0x00000064, 0x007000a1},       /* RFSHTMG */
+                       {0x000000d0, 0x000200e2},       /* INIT0 */
+                       {0x000000d4, 0x00580000},       /* INIT1 */
```

```
+                       {0x000000d8, 0x00000100},       /* INIT2 */
+                       {0x000000dc, 0x05240201},       /* INIT3 */
+                       {0x000000e0, 0x00080000},       /* INIT4 */
+                       {0x000000e4, 0x00110000},       /* INIT5 */
+                       {0x000000e8, 0x00000420},       /* INIT6 */
+                       {0x000000ec, 0x00000400},       /* INIT7 */
+                       {0x000000f4, 0x000f022f},       /* RANKCTL */
+                       {0x00000100, 0x0e0b1f10},       /* DRAMTMG0 */
+                       {0x00000104, 0x00030317},       /* DRAMTMG1 */
+                       {0x00000108, 0x0507050b},       /* DRAMTMG2 */
+                       {0x0000010c, 0x0040400c},       /* DRAMTMG3 */
+                       {0x00000110, 0x07030308},       /* DRAMTMG4 */
+                       {0x00000114, 0x05050303},       /* DRAMTMG5 */
+                       {0x00000120, 0x07070b07},       /* DRAMTMG8 */
+                       {0x00000124, 0x00020209},       /* DRAMTMG9 */
+                       {0x00000180, 0x01000040},       /* ZQCTL0 */
+                       {0x00000184, 0x00000000},       /* ZQCTL1 */
+                       {0x00000190, 0x07050003},       /* DFITMG0 */
+                       {0x00000198, 0x0a000101},       /* DFILPCFG0 */
+                       {0x000001a0, 0xc0400003},       /* DFIUPD0 */
+                       {0x00000240, 0x06000610},       /* ODTCFG */
+                       {0x00000244, 0x00000201},       /* ODTMAP */
+                       {0x00000250, 0x00001f00},       /* SCHED */
+                       {0x00000490, 0x00000001},       /* PCTRL_0 */
+                       {0xffffffff, 0xffffffff}
+               }
+       },
+       {
+               {
+                       {0x00000000, 0x00001fc7},       /* DDRPHY_REG0 */
+                       {0x00000008, 0x00000000},       /* DDRPHY_REG2 */
+                       {0x0000000c, 0x0e000000},       /* DDRPHY_REG3 */
+                       {0x00000010, 0x0a000000},       /* DDRPHY_REG4 */
+                       {0xffffffff, 0xffffffff}
+               }
+       }
+},

--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** [PATCH RFC 08/18] ram: rockchip: Add rk3568 ddr loader params**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
                  ` (6 preceding siblings ...)
  2026-05-17 19:24 ` [PATCH RFC 07/18] ram: rockchip: Add rk3568 ddr4 support Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 09/18] ram: rockchip: Add 4 ranks support to controller code for PX30 Pavel Golikov
                  ` (9 subsequent siblings)
  17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

Add DDR loader parameters for Rockchip RK3568 SoC.

```
Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 .../ram/rockchip/sdram-rk3568-loader_params.inc    | 109 +++++++++++++++++++++
 1 file changed, 109 insertions(+)

diff --git a/drivers/ram/rockchip/sdram-rk3568-loader_params.inc b/drivers/ram/rockchip/sdram-
rk3568-loader_params.inc
new file mode 100644
index 00000000000..d9beea3519a
--- /dev/null
+++ b/drivers/ram/rockchip/sdram-rk3568-loader_params.inc
@@ -0,0 +1,109 @@
+0x12345678,
+2, /* version */
+(0  << 0) | (1  << 8) | (9  << 16) | (8 << 24), /* cpu_gen,global index */
+(0  << 0) | (0  << 8) | (0  << 16) | (0 << 24), /* d2,d3 index */
+(17 << 0) | (9  << 8) | (0  << 16) | (0 << 24), /* d4,d5 index */
+(0  << 0) | (0  << 8) | (0  << 16) | (0 << 24), /* lp2,lp3 index */
+(26 << 0) | (13 << 8) | (0  << 16) | (0 << 24), /* lp4,lp5 index */
+(0  << 0) | (0  << 8) | (39 << 16) | (8 << 24), /* skew index, dq_map index */
+(0  << 0) | (0  << 8) | (0  << 16) | (0 << 24), /*lp4x index*/
+/* global info */
+0,
+(93 << 16) | 13,/* sr_idle << 16 | pd_idle */
+0,/* channel info */
+(1 << 26) | 1, /* derate, 2t info */
+0, /* reserved */
+(0 << 3), /* pageclose */
+0, 0, /* reserved */
+
+/* ddr4 */
+(1560 << DDR_FREQ_F0_SHIFT) | (324 << DDR_FREQ_F1_SHIFT),
+(528 << DDR_FREQ_F2_SHIFT) | (780 << DDR_FREQ_F3_SHIFT),
+(0 << DDR_FREQ_F4_SHIFT) | (0 << DDR_FREQ_F5_SHIFT),
+/* drv when odt on */
+(37 << PHY_DQ_DRV_SHIFT) | (37 << PHY_CA_DRV_SHIFT) |
+       (37 << PHY_CLK_DRV_SHIFT) | (34 << DRAM_DQ_DRV_SHIFT),
+/* drv when odt off */
+(37 << PHY_DQ_DRV_SHIFT) | (37 << PHY_CA_DRV_SHIFT) |
+       (37 << PHY_CLK_DRV_SHIFT) | (34 << DRAM_DQ_DRV_SHIFT),
+/* odt info */
+(120 << DRAM_ODT_SHIFT) | (139 << PHY_ODT_SHIFT) |
+       (1 << PHY_ODT_PUUP_EN_SHIFT) | (1 << PHY_ODT_PUDN_EN_SHIFT),
+/* odt enable freq */
+(625 << DRAM_ODT_EN_FREQ_SHIFT) | (625 << PHY_ODT_EN_FREQ_SHIFT),
+/* slew rate when odt enable */
+(0x0 << PHY_DQ_SR_SHIFT) | (0x1 << PHY_CA_SR_SHIFT) |
+       (0x1 << PHY_CLK_SR_SHIFT),
+/* slew  ratee when odt disable */
+(0x0 << PHY_DQ_SR_SHIFT) | (0x1 << PHY_CA_SR_SHIFT) |
+       (0x1 << PHY_CLK_SR_SHIFT),
+
+/* lpddr4 */
+(1560 << DDR_FREQ_F0_SHIFT) | (324 << DDR_FREQ_F1_SHIFT),
+(528 << DDR_FREQ_F2_SHIFT) | (780 << DDR_FREQ_F3_SHIFT),
+(0 << DDR_FREQ_F4_SHIFT) | (0 << DDR_FREQ_F5_SHIFT),
+
+/* drv when odt on */
+(30 << PHY_DQ_DRV_SHIFT) | (38 << PHY_CA_DRV_SHIFT) |
+       (38 << PHY_CLK_DRV_SHIFT) | (40 << DRAM_DQ_DRV_SHIFT),
+/* drv when odt off */
+(30 << PHY_DQ_DRV_SHIFT) | (38 << PHY_CA_DRV_SHIFT) |
+       (38 << PHY_CLK_DRV_SHIFT) | (40 << DRAM_DQ_DRV_SHIFT),
+/* odt info and PU-cal info */
```

```
+(80 << DRAM_ODT_SHIFT) | (60 << PHY_ODT_SHIFT) |
+       (120 << LP4_CA_ODT_SHIFT) |
+       (LPDDR4_VDDQ_3 << LP4_DRV_PU_CAL_ODTEN_SHIFT) |
+       (LPDDR4_VDDQ_3 << LP4_DRV_PU_CAL_ODTOFF_SHIFT) |
+       (0 << PHY_LP4_DRV_PULLDOWN_EN_ODTEN_SHIFT) |
+       (0 << PHY_LP4_DRV_PULLDOWN_EN_ODTOFF_SHIFT),
+/* odt enable freq */
+(800 << PHY_LP4_ODT_EN_FREQ_SHIFT) | (800 << LP4_DQ_ODT_EN_FREQ_SHIFT),
+/* slew rate when odt enable */
+(0 << PHY_DQ_SR_SHIFT) | (0 << PHY_CA_SR_SHIFT) |
+       (0 << PHY_CLK_SR_SHIFT),
+/* slew  rate when odt disable */
+(0 << PHY_DQ_SR_SHIFT) | (0 << PHY_CA_SR_SHIFT) |
+       (0 << PHY_CLK_SR_SHIFT),
+/* ca odt en freq */
+(800 << LP4_CA_ODT_EN_FREQ_SHIFT),
+/* cs drv info and ca odt info */
+(0 << PHY_LP4_CS_DRV_ODTEN_SHIFT) |
+       (0 << PHY_LP4_CS_DRV_ODTOFF_SHIFT) |
+       (1 << LP4_ODTE_CK_SHIFT) | (1 << LP4_ODTE_CS_EN_SHIFT) |
+       (0 << LP4_ODTD_CA_EN_SHIFT),
+/* vref info when odt enable */
+(166 << PHY_LP4_DQ_VREF_SHIFT) | (300 << LP4_DQ_VREF_SHIFT) |
+       (380 << LP4_CA_VREF_SHIFT),
+/* vref info when odt disable */
+(420 << PHY_LP4_DQ_VREF_SHIFT) | (420 << LP4_DQ_VREF_SHIFT) |
+       (420 << LP4_CA_VREF_SHIFT),
+
+/* ddr4 map << 0 | ddr3 map << 24 */
+((0x3 << 6) | (0x2 << 4) | (0x1 << 2) | (0x0 << 0)) | (0 << 8) | (0 << 16),
+/* lp3 map << 16 | lp4 map << 24 */
+((0x3 << 6) | (0x2 << 4) | (0x1 << 2) | (0x0 << 0)) << 24,
+/* lp3 dq0-7 map */
+0,
+/* lp2 dq0-7 map */
+0,
+/* ddr4 dq map */
+/* cs0 dq0-15 */
+       ((0 << 0 | 2 << 2 | 0 << 4 | 2 << 6) << 0) |
+       ((3 << 0 | 1 << 2 | 3 << 4 | 1 << 6) << 8) |
+       ((3 << 0 | 1 << 2 | 3 << 4 | 1 << 6) << 16) |
+       ((2 << 0 | 0 << 2 | 2 << 4 | 0 << 6) << 24),
+/* cs0 dq16-31 */
+       ((3 << 0 | 1 << 2 | 3 << 4 | 1 << 6) << 0) |
+       ((0 << 0 | 2 << 2 | 0 << 4 | 2 << 6) << 8) |
+       ((0 << 0 | 2 << 2 | 0 << 4 | 2 << 6) << 16) |
+       ((1 << 0 | 3 << 2 | 3 << 4 | 1 << 6) << 24),
+/* cs1 dq0-15 */
+       ((0 << 0 | 2 << 2 | 0 << 4 | 2 << 6) << 0) |
+       ((3 << 0 | 1 << 2 | 3 << 4 | 1 << 6) << 8) |
+       ((3 << 0 | 1 << 2 | 3 << 4 | 1 << 6) << 16) |
+       ((2 << 0 | 0 << 2 | 2 << 4 | 0 << 6) << 24),
+/* cs1 dq16-31 */
+       ((3 << 0 | 1 << 2 | 3 << 4 | 1 << 6) << 0) |
+       ((0 << 0 | 2 << 2 | 0 << 4 | 2 << 6) << 8) |
+       ((0 << 0 | 2 << 2 | 0 << 4 | 2 << 6) << 16) |
+       ((1 << 0 | 3 << 2 | 3 << 4 | 1 << 6) << 24),

--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

*** [PATCH RFC 09/18] ram: rockchip: Add 4 ranks support to controller code for PX30**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
                  ` (7 preceding siblings ...)
  2026-05-17 19:24 ` [PATCH RFC 08/18] ram: rockchip: Add rk3568 ddr loader params Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 10/18] ram: rockchip: Add missing bits to common code for sdram driver Pavel Golikov
                  ` (8 subsequent siblings)
  17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

RK3568 ram initialization code uses helper function from PX30
controller. RK3568 pctl supports 4 memory ranks. Handle this properly
when modifying pctl's PCTL2_MSTR register by clearing bits for all four
ranks before applying detected DRAM rank configuration.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 drivers/ram/rockchip/sdram_pctl_px30.c | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

```diff
diff --git a/drivers/ram/rockchip/sdram_pctl_px30.c b/drivers/ram/rockchip/sdram_pctl_px30.c
index 3ec98af536e..0f42c191dc3 100644
--- a/drivers/ram/rockchip/sdram_pctl_px30.c
+++ b/drivers/ram/rockchip/sdram_pctl_px30.c
@@ -149,7 +149,7 @@ u32 pctl_remodify_sdram_params(struct ddr_pctl_regs *pctl_regs,
                }
        }

-       tmp &= ~((3ul << 30) | (3ul << 24) | (3ul << 12));
+       tmp &= ~((3ul << 30) | (0xful << 24) | (3ul << 12));

        switch (cap_info->dbw) {
        case 2:
```

--
2.25.1

^ permalink raw reply related [**flat**|nested] 22+ messages in thread

---

*** [PATCH RFC 10/18] ram: rockchip: Add missing bits to common code for sdram driver**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
                  ` (8 preceding siblings ...)
  2026-05-17 19:24 ` [PATCH RFC 09/18] ram: rockchip: Add 4 ranks support to controller code for PX30 Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 11/18] ram: rockchip: Add SDRAM initialization support for rk3568 Pavel Golikov
                  ` (7 subsequent siblings)
  17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,

```
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

Add some missing macros used to retrieve SDRAM initialization code
configuration from global info. Extract PHY and DRAM ODT enable masks
to separate definitions.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 arch/arm/include/asm/arch-rockchip/sdram_common.h | 13 +++++++++++--
 1 file changed, 11 insertions(+), 2 deletions(-)

diff --git a/arch/arm/include/asm/arch-rockchip/sdram_common.h b/arch/arm/include/asm/arch-
rockchip/sdram_common.h
index e53e5a9ca58..84fbbeb8d8d 100644
--- a/arch/arm/include/asm/arch-rockchip/sdram_common.h
+++ b/arch/arm/include/asm/arch-rockchip/sdram_common.h
@@ -45,6 +45,12 @@
 #define PLL_SSMOD_SPREAD(n)            (((n) >> 1) & 0xff)
 #define PLL_SSMOD_DIV(n)               (((n) >> 9) & 0xff)
 #define PLL_SSMOD_DOWNSPREAD(n)                (((n) >> 17) & 0x3)
+#define DIS_TRAIN_PRINT(n)             (((n) >> 19) & 0x1)
+#define DDR_DERATE_EN(n)               (((n) >> 26) & 0x1)
+#define DDR_PER_BANK_REF_EN(n)         (((n) >> 27) & 0x1)
+#define DDR_EXT_TEMP_REF(n)            (((n) >> 29) & 0x3)
+
+#define DDR_PAGECLOSE_EN(n)            (((n) >> 3) & 0x1)

 /* sdram_head_info_v2 define */
 /* for *_drv_odten and *_drv_odtoff */
@@ -69,10 +75,13 @@

 /* for *odt_en_freq; */
 #define DRAM_ODT_EN_FREQ_SHIFT         0
+#define DRAM_ODT_EN_FREQ_MASK          0xfff
 #define PHY_ODT_EN_FREQ_SHIFT          12
+#define PHY_ODT_EN_FREQ_MASK           0xfff
 #define DRAMODT_EN_FREQ(n)             (((n) >> DRAM_ODT_EN_FREQ_SHIFT) & \
-                                        0xfff)
-#define PHYODT_EN_FREQ(n)              (((n) >> PHY_ODT_EN_FREQ_SHIFT) & 0xfff)
+                                        DRAM_ODT_EN_FREQ_MASK)
+#define PHYODT_EN_FREQ(n)              (((n) >> PHY_ODT_EN_FREQ_SHIFT) & \
+                                        PHY_ODT_EN_FREQ_MASK)

 #define PHY_DQ_SR_SHIFT                        0
 #define PHY_CA_SR_SHIFT                        8

--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

* **[PATCH RFC 11/18] ram: rockchip: Add SDRAM initialization support for rk3568**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
                  ` (9 preceding siblings ...)
  2026-05-17 19:24 ` [PATCH RFC 10/18] ram: rockchip: Add missing bits to common code for sdram driver Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 12/18] clk: rockchip: rk3568: Reduce code size for TPL build Pavel Golikov

```
                    ` (6 subsequent siblings)
  17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

SDRAM is ought to be initialized at TPL stage. Only LPDDR4
initialization code is confirmed to be working for now.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 .../include/asm/arch-rockchip/sdram_phy_rk3568.h   |  549 +++
 .../asm/arch-rockchip/sdram_phy_ron_rtt_rk3568.h   |  209 +
 arch/arm/include/asm/arch-rockchip/sdram_rk3568.h  |  197 +
 drivers/ram/rockchip/Makefile                      |    2 +-
 drivers/ram/rockchip/sdram_rk3568.c                | 4384 +++++++++++++++++++-
 5 files changed, 5336 insertions(+), 5 deletions(-)

diff --git a/arch/arm/include/asm/arch-rockchip/sdram_phy_rk3568.h b/arch/arm/include/asm/arch-
rockchip/sdram_phy_rk3568.h
new file mode 100644
index 00000000000..d936d545386
--- /dev/null
+++ b/arch/arm/include/asm/arch-rockchip/sdram_phy_rk3568.h
@@ -0,0 +1,549 @@
+/* SPDX-License-Identifier:     GPL-2.0+ */
+/*
+ * Copyright (c) 2024-2026, Pavel Golikov <paullo612@ya.ru>
+ */
+
+#ifndef _ASM_ARCH_SDRAM_PHY_RK3568_H
+#define _ASM_ARCH_SDRAM_PHY_RK3568_H
+
+#include <asm/arch-rockchip/sdram_phy_ron_rtt_rk3568.h>
+
+/* DDRPHY_REG0 */
+#define CHANNEL_EN_MASK                        (0x1f << 8)
+#define CHANNEL_EN_SHIFT               8
+#define RANK4_EN_MASK                  BIT(20)
+#define RANK4_EN_SHIFT                 20
+
+/* DDRPHY_REG1 */
+#define START_CALIB                    BIT(0)
+#define CALCS_SEL_MASK                 (0xf << 2)
+#define CALCS_SEL_SHIFT                        2
+#define WL_ENABLE                      BIT(6)
+#define WLCS_SEL_MASK                  (0xf << 8)
+#define WLCS_SEL_SHIFT                 8
+#define WL_LOADMODE_MASK               (0xffff << 16)
+#define WL_LOADMODE_SHIFT              16
+
+/* DDRPHY_REG3 */
+#define CL_FRE_OPN_MASK(n)             (0x1f << (24 - 8 * (n)))
+#define CL_FRE_OPN_SHIFT(n)            (24 - 8 * (n))
+
+/* DDRPHY_REG4 */
+#define CWL_FRE_OPN_MASK(n)            (0x1f << (24 - 8 * (n)))
+#define CWL_FRE_OPN_SHIFT(n)           (24 - 8 * (n))
+
+/* DDRPHY_REG5 */
```

```
+#define FB1XCLK_INVDELA_MASK           (0x1f << 24)
+#define FB1XCLK_INVDELA_SHIFT          24
+
+/* DDRPHY_REG8 */
+#define CAT_RANK_NUM_TWO_RANKS         0xc
+#define CAT_RANK_NUM_RANK0             0xe
+#define CAT_RANK_NUM_RANK1             0xd
+
+#define CAT_BP_RANK_SEL_RANK0          0xe
+#define CAT_BP_RANK_SEL_RANK1          0xd
+
+#define CAT_ENABLE                     BIT(0)
+#define CAT_START                      BIT(1)
+#define CLK_DIV_CNT_MASK               (0x1f << 8)
+#define CLK_DIV_CNT_SHIFT              8
+#define CAT_RANK_NUM_MASK              (0xf << 15)
+#define CAT_RANK_NUM_SHIFT             15
+#define CAT_BP_RANK_SEL_MASK           (0xf << 20)
+#define CAT_BP_RANK_SEL_SHIFT          20
+#define CMD_RANK_SWITCH_DIS            BIT(28)
+
+/* DDRPHY_REG9 */
+#define DDRPHY_TCACD_MASK              0x1f
+#define DDRPHY_TCACD_SHIFT             0
+#define DDRPHY_TMRW_MASK               (0x1f << 5)
+#define DDRPHY_TMRW_SHIFT              5
+#define DDRPHY_TDSTRAIN_MASK           (0x1f << 10)
+#define DDRPHY_TDSTRAIN_SHIFT          10
+#define DDRPHY_TCKELCK_MASK            (0x1f << 15)
+#define DDRPHY_TCKELCK_SHIFT           15
+#define DDRPHY_TADR_MASK               (0x1f << 20)
+#define DDRPHY_TADR_SHIFT              20
+#define DDRPHY_TXCBT_MASK              (0x1f << 25)
+#define DDRPHY_TXCBT_SHIFT             25
+
+/* DDRPHY_REG10_0 */
+#define DDRPHY_TFC_MASK                        0xff
+#define DDRPHY_TFC_SHIFT               0
+#define DDRPHY_TCAENT_MASK             (0xff << 8)
+#define DDRPHY_TCAENT_SHIFT            8
+#define DDRPHY_TVREFCA_LONG_MASK       (0xff << 16)
+#define DDRPHY_TVREFCA_LONG_SHIFT      16
+
+/* DDRPHY_REGA */
+#define DDRPHY_MR11_MASK               0xff
+#define DDRPHY_MR11_SHIFT              0
+#define DDRPHY_MR3_MASK                        (0xff << 8)
+#define DDRPHY_MR3_SHIFT               8
+#define DDRPHY_MR2_MASK                        (0xff << 16)
+#define DDRPHY_MR2_SHIFT               16
+#define DDRPHY_MR1_MASK                        (0xff << 24)
+#define DDRPHY_MR1_SHIFT               24
+
+/* DDRPHY_REGB */
+#define DDRPHY_MR22_MASK               0xff
+#define DDRPHY_MR22_SHIFT              0
+#define DDRPHY_MR14_MASK               (0xff << 8)
+#define DDRPHY_MR14_SHIFT              8
+#define DDRPHY_MR13_MASK               (0xff << 16)
+#define DDRPHY_MR13_SHIFT              16
+
+/* DDRPHY_REGC */
+#define CAT_VREF_SCAN_DIS              BIT(30)
+
+/* DDRPHY_REGD */
```

```
+#define CA_PERBIT_SKEW_UPDATE          BIT(0)
+#define CMD_PERBIT_SKEW_BP             BIT(1)
+#define CAT_SKIP_FSPY                  BIT(6)
+#define LPDDR_CA_ODT_SEL_MASK          BIT(11)
+#define LPDDR_CA_ODT_SEL_SHIFT         11
+#define LPDDR_CA_ODT0_MASK             (0x3 << 12)
+#define LPDDR_CA_ODT0_SHIFT            12
+#define LPDDR_CA_ODT1_MASK             (0x3 << 14)
+#define LPDDR_CA_ODT1_SHIFT            14
+
+/* DDRPHY_REG10_1 */
+#define CALIB_MODE_SEL                 BIT(24)
+#define FREQ_CHOOSE_T_MASK             (0x3 << 30)
+#define FREQ_CHOOSE_T_SHIFT            30
+
+/* DDRPHY_REG20 */
+#define LP_BYPASS                      BIT(15)
+#define TRAIN_REG_UPDATE_EN            BIT(18)
+
+/* DDRPHY_REG22 */
+#define ZQCALI_BYPASS                  BIT(1)
+#define ZQCALI_CLEAR                   BIT(2)
+#define PD_ZQCALI                      BIT(3)
+
+/* DDRPHY_REG24 */
+#define DQ_RD_TRAIN_EN                 BIT(0)
+#define RD_TRAIN_FREQ_UPDATE           BIT(2)
+#define RD_TRAIN_CHECK_VALUE_EN                BIT(3)
+#define RX_VREF_VALUE_UPDATE           BIT(7)
+#define RDTRAIN_CS_SEL_MASK            (0xf << 8)
+#define RDTRAIN_CS_SEL_SHIFT           8
+#define RD_TRAIN_PREDEF_EN             BIT(14)
+
+/* DDRPHY_REG27 */
+#define DQ_WR_TRAIN_AUTO                       BIT(0)
+#define DQ_WR_TRAIN_EN                         BIT(1)
+#define WR_TRAIN_DQS_DEFAULT_BYPASS            BIT(4)
+#define WRTRAIN_CS_SEL_MASK                    (0xf << 6)
+#define WRTRAIN_CS_SEL_SHIFT                   6
+#define WRTRAIN_CHECK_DATA_VALUE_RANDOM_GEN    BIT(10)
+#define WRTRAIN_LPDDR4_VREF_RANGE_MASK         BIT(24)
+#define WRTRAIN_LPDDR4_VREF_RANGE_SHIFT                24
+#define DM_WR_TRAIN_EN                         BIT(25)
+
+/* DDRPHY_REG28 */
+#define WR_TRAIN_ROW_ADDR_MASK         0xffff
+#define WR_TRAIN_ROW_ADDR_SHIFT                0
+
+/* DDRPHY_REG29 */
+#define PHY_TRFC_MASK                  (0x3ff << 8)
+#define PHY_TRFC_SHIFT                 8
+#define PHY_TREFI_MASK                 (0x1fff << 18)
+#define PHY_TREFI_SHIFT                        18
+
+/* DDRPHY_REG2A */
+#define FREQ_CHOOSE_B_EN               BIT(4)
+
+/* DDRPHY_REG2F */
+#define PVT_COMP_DIS                   BIT(0)
+#define CMD_T2_MODE_MASK               BIT(17)
+#define CMD_T2_MODE_SHIFT              17
+#define CMD_DELAY_ONE_UI_MASK          BIT(18)
+#define CMD_DELAY_ONE_UI_SHIFT         18
+
+/* DDRPHY_REG33 */
```

```
+#define PLL_POSTDIV0_EN_MASK           BIT(3)
+#define PLL_POSTDIV0_EN_SHIFT          3
+#define PLL_POSTDIV0_MASK              (7 << 4)
+#define PLL_POSTDIV0_SHIFT             4
+#define PLL_POSTDIVN_EN_MASK(n)                (PLL_POSTDIV0_EN_MASK << (8 * (n)))
+#define PLL_POSTDIVN_EN_SHIFT(n)       (PLL_POSTDIV0_EN_SHIFT + (8 * (n)))
+#define PLL_POSTDIVN_MASK(n)           (PLL_POSTDIV0_MASK << (8 * (n)))
+#define PLL_POSTDIVN_SHIFT(n)          (PLL_POSTDIV0_SHIFT + (8 * (n)))
+
+/* DDRPHY_REG37 */
+#define CMD_ABUTSLEWPD_MASK            (0x1f << 0)
+#define CMD_ABUTSLEWPD_SHIFT           0
+#define CMD_ABUTSLEWPU_MASK            (0x1f << 8)
+#define CMD_ABUTSLEWPU_SHIFT           8
+
+/* DDRPHY_REG38 */
+#define CMD_ABUTPRCOMP_CK0_MASK                (0x1f << 0)
+#define CMD_ABUTPRCOMP_CK0_SHIFT       0
+#define CMD_ABUTNRCOMP_CK0_MASK                (0x1f << 8)
+#define CMD_ABUTNRCOMP_CK0_SHIFT       8
+#define CMD_ABUTPRCOMP_MASK            (0x1f << 16)
+#define CMD_ABUTPRCOMP_SHIFT           16
+#define CMD_ABUTNRCOMP_MASK            (0x1f << 24)
+#define CMD_ABUTNRCOMP_SHIFT           24
+
+/* DDRPHY_REG3B */
+#define A0_INVDELAYSEL_MASK            (0xff << 24)
+#define A0_INVDELAYSEL_SHIFT           24
+
+/* undocumented0, DDRPHY_COM_REG_0x3e in rk3562 TRM */
+#define CMD_ABUTPRCOMP_SPECIAL_MASK    0x1f
+#define CMD_ABUTPRCOMP_SPECIAL_SHIFT   0
+#define CMD_ABUTNRCOMP_SPECIAL_MASK    (0x1f << 8)
+#define CMD_ABUTNRCOMP_SPECIAL_SHIFT   8
+#define RAM_VREF1_MARGSEL_MASK         (0x1ff << 16)
+#define RAM_VREF1_MARGSEL_SHIFT                16
+
+/* DDRPHY_REG41 */
+#define CK_INVDELAYSEL_MASK            (0xff << 16)
+#define CK_INVDELAYSEL_SHIFT           16
+
+/*
+ * DDRPHY_REG54, DDRPHY_REG57, DDRPHY_REG5A, DDRPHY_REG5D, DDRPHY_REG60, DDRPHY_REG63,
DDRPHY_REG66,
+ * DDRPHY_REG69
+ */
+#define DQ_TRAIN_CHECK_DATA_VALUE_MASK(n)      (0xff << ((n) * 8))
+#define DQ_TRAIN_CHECK_DATA_VALUE_SHIFT(n)     ((n) * 8)
+
+/* DDRPHY_REG6C, CMD_INVDELAYSEL_SEL_* description is based on rk3562 TRM. */
+#define CMD_INVDELAYSEL_SEL_A_CA0_CS0          0
+#define CMD_INVDELAYSEL_SEL_A_CA1_CS0          1
+#define CMD_INVDELAYSEL_SEL_A_CA2_CS0          2
+#define CMD_INVDELAYSEL_SEL_A_CA3_CS0          3
+#define CMD_INVDELAYSEL_SEL_A_CA4_CS0          4
+#define CMD_INVDELAYSEL_SEL_A_CA5_CS0          5
+#define CMD_INVDELAYSEL_SEL_B_CA0_CS0          6
+#define CMD_INVDELAYSEL_SEL_B_CA1_CS0          7
+#define CMD_INVDELAYSEL_SEL_B_CA2_CS0          8
+#define CMD_INVDELAYSEL_SEL_B_CA3_CS0          9
+#define CMD_INVDELAYSEL_SEL_B_CA4_CS0          10
+#define CMD_INVDELAYSEL_SEL_B_CA5_CS0          11
+#define CMD_INVDELAYSEL_SEL_A_CA0_CS1          12
+#define CMD_INVDELAYSEL_SEL_A_CA1_CS1          13
+#define CMD_INVDELAYSEL_SEL_A_CA2_CS1          14
```

```
+#define CMD_INVDELAYSEL_SEL_A_CA3_CS1          15
+#define CMD_INVDELAYSEL_SEL_A_CA4_CS1          16
+#define CMD_INVDELAYSEL_SEL_A_CA5_CS1          17
+#define CMD_INVDELAYSEL_SEL_B_CA0_CS1          18
+#define CMD_INVDELAYSEL_SEL_B_CA1_CS1          19
+#define CMD_INVDELAYSEL_SEL_B_CA2_CS1          20
+#define CMD_INVDELAYSEL_SEL_B_CA3_CS1          21
+#define CMD_INVDELAYSEL_SEL_B_CA4_CS1          22
+#define CMD_INVDELAYSEL_SEL_B_CA5_CS1          23
+#define CMD_INVDELAYSEL_SEL_B_CA5_CS1          23
+#define CMD_INVDELAYSEL_SEL_A_CK               24
+#define CMD_INVDELAYSEL_SEL_A_CKB              25
+#define CMD_INVDELAYSEL_SEL_A_CKE0             26
+#define CMD_INVDELAYSEL_SEL_A_CKE1             27
+#define CMD_INVDELAYSEL_SEL_A_CSB0             28
+#define CMD_INVDELAYSEL_SEL_A_CSB1             29
+#define CMD_INVDELAYSEL_SEL_A_ODT0             30
+#define CMD_INVDELAYSEL_SEL_A_ODT1             31
+#define CMD_INVDELAYSEL_SEL_B_CK               32
+#define CMD_INVDELAYSEL_SEL_B_CKB              33
+#define CMD_INVDELAYSEL_SEL_B_CKE0             34
+#define CMD_INVDELAYSEL_SEL_B_CKE1             35
+#define CMD_INVDELAYSEL_SEL_B_CSB0             36
+#define CMD_INVDELAYSEL_SEL_B_CSB1             37
+#define CMD_INVDELAYSEL_SEL_B_ODT0             38
+#define CMD_INVDELAYSEL_SEL_B_ODT1             39
+
+#define RDTRAIN_WAIT_VERF_VALID_CNT_MASK       0x1ff
+#define RDTRAIN_WAIT_VERF_VALID_CNT_SHIFT      0
+#define CMD_INVDELAYSEL_SEL_MASK               (0x3f << 10)
+#define CMD_INVDELAYSEL_SEL_SHIFT              10
+
+/* DDRPHY_REG7C */
+#define TRAIN_TRUE_DONE                        BIT(0)
+#define TRAIN_STEP3_ERROR              BIT(1)
+#define TRAIN_STEP2_ERROR              BIT(2)
+#define TRAIN_STEP1_ERROR              BIT(3)
+#define TRAIN_ALL_STEP_DONE            BIT(7)
+
+/* DDRPHY_REG7D */
+#define HALFUI_LOCK_CODE_TO_REG_MASK   (0xff << 24)
+#define HALFUI_LOCK_CODE_TO_REG_SHIFT  24
+
+/* DDRPHY_REG83 */
+#define CALIB_DONE_BYTE_MASK           0x1f
+#define CALIB_ERROR                    BIT(5)
+#define WL_DONE_BYTE_MASK              (0x1f << 8)
+#define WL_DONE_BYTE_SHIFT             8
+
+/* DDRPHY_REG85 */
+#define CHB_CAT_DONE                   BIT(2)
+#define CHA_CAT_DONE                   BIT(3)
+
+/* undocumented1, DDRPHY_COM_REG_0x8c in rk3562 TRM */
+#define WRTRAIN_VREF_MIN_VALUE_MASK    0xf7
+#define WRTRAIN_VREF_MIN_VALUE_SHIFT   0
+#define WRTRAIN_VREF_MAX_VALUE_MASK    (0xf7 << 8)
+#define WRTRAIN_VREF_MAX_VALUE_SHIFT   8
+#define CMD_INVDELAYSEL_MASK           (0xff << 16)
+#define CMD_INVDELAYSEL_SHIFT          16
+
+/* DDRPHY_REGAD */
+#define TRAIN_ERROR_FOR_RD_BYTE_MASK   (0x1f << 10)
+#define TRAIN_ERROR_FOR_RD_BYTE_SHIFT  10
+
```

```
+/* The next ones are totally missing from rk3568 TRM. Based on rk3562 TRM. */
+
+/* DDRPHY_BYTE_REG_0x0 */
+#define ABC_LH_ENB_LP4MODE_MASK                BIT(7)
+#define ABC_LH_ENB_LP4MODE_SHIFT       7
+#define ABC_LH_ABUTSLEWPU_MASK         (0x1f << 8)
+#define ABC_LH_ABUTSLEWPU_SHIFT                8
+#define ABC_LH_VREF1_MARGSEL_MASK      (0x1ff << 23)
+#define ABC_LH_VREF1_MARGSEL_SHIFT     23
+
+/* DDRPHY_BYTE_REG_0x1 */
+#define ABC_LH_ABUTODTPU_MASK          (0x1f << 0)
+#define ABC_LH_ABUTODTPU_SHIFT         0
+#define ABC_LH_ABUTODTPD_MASK          (0x1f << 8)
+#define ABC_LH_ABUTODTPD_SHIFT         8
+#define ABC_LH_ABUTPRCOMP_MASK         (0x1f << 16)
+#define ABC_LH_ABUTPRCOMP_SHIFT                16
+#define ABC_LH_ABUTNRCOMP_MASK         (0x1f << 24)
+#define ABC_LH_ABUTNRCOMP_SHIFT                24
+
+/* DDRPHY_BYTE_REG_0x2 */
+#define ABC_LH_DQSWEAKP_PULL_UP                0
+#define ABC_LH_DQSWEAKP_MIDDLE         1
+#define ABC_LH_DQSWEAKP_HIGH_Z         2
+#define ABC_LH_DQSWEAKP_PULL_DN                3
+
+#define ABC_LH_DQSWEAKP_MASK           (3 << 5)
+#define ABC_LH_DQSWEAKP_SHIFT          5
+#define ABC_LH_LP4X_EN                 BIT(8)
+#define ABC_LH_RXEN_LP4                        BIT(9)
+
+/* DDRPHY_BYTE_REG_0xf, DDRPHY_BYTE_REG_0x13, DDRPHY_BYTE_REG_0x42, DDRPHY_BYTE_REG_0x46 */
+#define ABC_LH_CSN_DQSB_INVDELAYSELRX_MASK     (0x7f << 8)
+#define ABC_LH_CSN_DQSB_INVDELAYSELRX_SHIFT    8
+#define ABC_LH_CSN_DQS_INVDELAYSELRX_MASK      (0x7f << 24)
+#define ABC_LH_CSN_DQS_INVDELAYSELRX_SHIFT     24
+
+/* DDRPHY_BYTE_REG_0xc, DDRPHY_BYTE_REG_0x14, DDRPHY_BYTE_REG_0x43, DDRPHY_BYTE_REG_0x4b */
+#define ABC_LH_CSN_LOOP_INVDELAYSEL_MASK       (0x1f << 24)
+#define ABC_LH_CSN_LOOP_INVDELAYSEL_SHIFT      24
+
+/* DDRPHY_BYTE_REG_0x18 */
+#define ABC_LH_TRAIN_DQS_DEFAULT_MASK          (0xff << 16)
+#define ABC_LH_TRAIN_DQS_DEFAULT_SHIFT         16
+#define ABC_LH_RD_TRAIN_DQS_DEFAULT_MASK       (0x1f << 24)
+#define ABC_LH_RD_TRAIN_DQS_DEFAULT_SHIFT      24
+
+/* DDRPHY_BYTE_REG_0x19 */
+#define ABC_LH_RDTRAIN_CHECK_WRAP1_MASK                Oxff
+#define ABC_LH_RDTRAIN_CHECK_WRAP1_SHIFT       0
+#define ABC_LH_RDTRAIN_CHECK_WRAP0_MASK                (0xff << 8)
+#define ABC_LH_RDTRAIN_CHECK_WRAP0_SHIFT       8
+
+/* DDRPHY_BYTE_REG_0x1c */
+#define ABC_LH_TDQS_INVDELAYSEL1_MASK          0xff
+#define ABC_LH_TDQS_INVDELAYSEL1_SHIFT         0
+#define ABC_LH_TDQS_INVDELAYSEL0_MASK          (0xff << 16)
+#define ABC_LH_TDQS_INVDELAYSEL0_SHIFT         16
+
+/* DDRPHY_BYTE_REG_0x1e */
+#define ABC_LH_CALIB_RESULT_CS0_MASK   (0x7ff << 16)
+#define ABC_LH_CALIB_RESULT_CS0_SHIFT  16
+
+/* DDRPHY_BYTE_REG_0x1f, DDRPHY_BYTE_REG_0x20, DDRPHY_BYTE_REG_0x54, DDRPHY_BYTE_REG_0x55 */
+#define ABC_LH_CSN_VALUE_DQX_INVDELAYSEL_MASK  0xff
```

```
+#define ABC_LH_CSN_VALUE_DQX_INVDELAYSEL_SHIFT  0
+
+/* DDRPHY_BYTE_REG_0x27 */
+#define ABC_LH_TRAIN_RESULT_FOR_RD_BASE_DQS_MASK       (0x7f << 24)
+#define ABC_LH_TRAIN_RESULT_FOR_RD_BASE_DQS_SHIFT      24
+
+struct rk3568_ddr_phy_regs {
+       u32 phy[8][2];
+};
+
+struct rk3568_ddrphy_invdelaysel_regs {
+       u32 reg3b;
+       u32 reg3c;
+       u32 reg3d;
+       u32 reg3e;
+       u32 reg3f;
+       u32 reg40;
+       u32 reg41;
+       u32 reg42;
+};
+
+struct rk3568_ddrphy_dq_train_check_data_regs {
+       u32 reg54;
+       u32 reg55;
+       u32 reg56;
+       u32 reg57;
+       u32 reg58;
+       u32 reg59;
+       u32 undocumented0[(0x1b0 - 0x164) / 4 - 1]; /* documented in rk3562 TRM */
+};
+
+struct rk3568_ddrphy_pad_group {
+       u32 reg0;
+       u32 reg1;
+       u32 reg2;
+       u32 reg3;
+       u32 reg4;
+       u32 reg5;
+       u32 reg6;
+       u32 reg7;
+       u32 reg8;
+       u32 reg9;
+       u32 rega;
+       u32 regb;
+       u32 regc;
+       u32 regd;
+       u32 rege;
+       u32 regf;
+       u32 reg10;
+       u32 reg11;
+       u32 reg12;
+       u32 reg13;
+       u32 reg14;
+       u32 reg15;
+       u32 reg16;
+       u32 reg17;
+       u32 reg18;
+       u32 reg19;
+       u32 reserved0[(0x70 - 0x64) / 4 - 1];
+       u32 tdqs_invdelaysel12;
+       u32 reg1d;
+       u32 reg1e;
+       u32 reg1f;
+       u32 reg20;
+       u32 train_for_rd[6];
```

```
+       u32 reg27;
+       u32 rd_train_readback_data[4];
+       u32 train_for_wr[6];
+       u32 reg32;
+       u32 reserved1[(0x100 - 0xc8) / 4 - 1];
+       u32 reg40;
+       u32 reg41;
+       u32 reg42;
+       u32 reg43;
+       u32 reg44;
+       u32 reg45;
+       u32 reg46;
+       u32 reg47;
+       u32 reg48;
+       u32 reg49;
+       u32 reg4a;
+       u32 reg4b;
+       u32 reg4c;
+       u32 reg4d;
+       u32 reg4e;
+       u32 reg4f;
+       u32 reg50;
+       u32 reg51;
+       u32 reg52;
+       u32 tdqs_invdelaysel23;
+       u32 reg54;
+       u32 reg55;
+       u32 reg56;
+       u32 reserved2[(0x180 - 0x158) / 4 - 1];
+};
+
+struct rk3568_ddrphy {
+       u32 reg0;
+       u32 reg1;
+       u32 reg2;
+       u32 reg3;
+       u32 reg4;
+       u32 reg5;
+       u32 reg6;
+       u32 reg7;
+       u32 reg8;
+       u32 reg9;
+       u32 reg10_0;
+       u32 rega;
+       u32 regb;
+       u32 regc;
+       u32 regd;
+       u32 rege;
+       u32 regf;
+       u32 reg10_1;
+       u32 reg11;
+       u32 reg12;
+       u32 reg13;
+       u32 reg14;
+       u32 reg15;
+       u32 reg16;
+       u32 reg17;
+       u32 reg18;
+       u32 reg19;
+       u32 reg1a;
+       u32 reg1b;
+       u32 reg1c;
+       u32 reg1d;
+       u32 reg1e;
+       u32 reg1f;
```

```
+       u32 reg20;
+       u32 reg21;
+       u32 reg22;
+       u32 reg23;
+       u32 reg24;
+       u32 reg25;
+       u32 reg26;
+       u32 reg27;
+       u32 reg28;
+       u32 reg29;
+       u32 reg2a;
+       u32 reg2b;
+       u32 reg2c;
+       u32 reg2d;
+       u32 reg2e;
+       u32 reg2f;
+       u32 reg30;
+       u32 reg31;
+       u32 reg32;
+       u32 reg33;
+       u32 reg34;
+       u32 reg35;
+       u32 reg36;
+       u32 reserved0[(0x0f0 - 0x0dc) / 4 - 1];
+       u32 reg37;
+       u32 reg38;
+       u32 undocumented0; /* documented in rk3562 TRM */
+       u32 reserved1[(0x104 - 0x0f8) / 4 - 1];
+       union {
+               struct rk3568_ddrphy_invdelaysel_regs invdelaysel_regs;
+               u32 reg_n_invdelaysel[8];
+       };
+       u32 reserved2[(0x150 - 0x120) / 4 - 1];
+       union {
+               struct rk3568_ddrphy_dq_train_check_data_regs dq_train_check_data_regs;
+               u32 reg_n_dq_train_check_data[8][3];
+       };
+       u32 reg6c;
+       u32 reserved4;
+       u32 reg6e;
+       u32 reserved5[(0x1f0 - 0x1b8) / 4 - 1];
+       u32 reg7c;
+       u32 reg7d;
+       u32 reg7e;
+       u32 reg7f;
+       u32 reserved6[(0x20c - 0x1fc) / 4 - 1];
+       u32 reg83;
+       u32 reg84;
+       u32 reg85;
+       u32 reserved7[(0x230 - 0x214) / 4 - 1];
+       u32 undocumented1; /* documented in rk3562 TRM */
+       u32 cmd_training_result[2][7];
+       u32 reserved8[(0x2b4 - 0x268) / 4 - 1];
+       u32 regad;
+       u32 reserved9[(0x300 - 0x2b4) / 4 - 1];
+       /* 2 * x8 for channel A, 2 * x8 for channel B and x8 for channel C (ECC) */
+       struct rk3568_ddrphy_pad_group pad_group[5];
+};
+
+#endif /* _ASM_ARCH_SDRAM_PHY_RK3568_H */
diff --git a/arch/arm/include/asm/arch-rockchip/sdram_phy_ron_rtt_rk3568.h b/arch/arm/include/
asm/arch-rockchip/sdram_phy_ron_rtt_rk3568.h
new file mode 100644
index 00000000000..3361547b504
--- /dev/null
```

```
+++ b/arch/arm/include/asm/arch-rockchip/sdram_phy_ron_rtt_rk3568.h
@@ -0,0 +1,209 @@
+/* SPDX-License-Identifier:     GPL-2.0+ */
+/*
+ * Copyright (c) 2024-2026, Pavel Golikov <paullo612@ya.ru>
+ */
+
+#ifndef _ASM_ARCH_SDRAM_PHY_RON_RTT_RK3568_H
+#define _ASM_ARCH_SDRAM_PHY_RON_RTT_RK3568_H
+
+#define PHY_DDR3_RON_DISABLE           (0)
+#define PHY_DDR3_RON_500ohm            (1)
+#define PHY_DDR3_RON_250ohm            (2)
+#define PHY_DDR3_RON_167ohm            (3)
+#define PHY_DDR3_RON_125ohm            (4)
+#define PHY_DDR3_RON_100ohm            (5)
+#define PHY_DDR3_RON_83ohm             (6)
+#define PHY_DDR3_RON_71ohm             (7)
+#define PHY_DDR3_RON_63ohm             (8)
+#define PHY_DDR3_RON_56ohm             (9)
+#define PHY_DDR3_RON_50ohm             (10)
+#define PHY_DDR3_RON_45ohm             (11)
+#define PHY_DDR3_RON_41ohm             (12)
+#define PHY_DDR3_RON_38ohm             (13)
+#define PHY_DDR3_RON_36ohm             (14)
+#define PHY_DDR3_RON_33ohm             (15)
+#define PHY_DDR3_RON_31ohm             (24)
+#define PHY_DDR3_RON_29ohm             (25)
+#define PHY_DDR3_RON_28ohm             (26)
+#define PHY_DDR3_RON_26ohm             (27)
+#define PHY_DDR3_RON_25ohm             (28)
+#define PHY_DDR3_RON_24ohm             (29)
+#define PHY_DDR3_RON_23ohm             (30)
+#define PHY_DDR3_RON_22ohm             (31)
+
+#define PHY_DDR3_RTT_DISABLE           (0)
+#define PHY_DDR3_RTT_500ohm            (1)
+#define PHY_DDR3_RTT_250ohm            (2)
+#define PHY_DDR3_RTT_167ohm            (3)
+#define PHY_DDR3_RTT_125ohm            (4)
+#define PHY_DDR3_RTT_100ohm            (5)
+#define PHY_DDR3_RTT_83ohm             (6)
+#define PHY_DDR3_RTT_71ohm             (7)
+#define PHY_DDR3_RTT_63ohm             (8)
+#define PHY_DDR3_RTT_56ohm             (9)
+#define PHY_DDR3_RTT_50ohm             (10)
+#define PHY_DDR3_RTT_45ohm             (11)
+#define PHY_DDR3_RTT_41ohm             (12)
+#define PHY_DDR3_RTT_38ohm             (13)
+#define PHY_DDR3_RTT_36ohm             (14)
+#define PHY_DDR3_RTT_33ohm             (15)
+#define PHY_DDR3_RTT_31ohm             (24)
+#define PHY_DDR3_RTT_29ohm             (25)
+#define PHY_DDR3_RTT_28ohm             (26)
+#define PHY_DDR3_RTT_26ohm             (27)
+#define PHY_DDR3_RTT_25ohm             (28)
+#define PHY_DDR3_RTT_24ohm             (29)
+#define PHY_DDR3_RTT_23ohm             (30)
+#define PHY_DDR3_RTT_22ohm             (31)
+
+#define PHY_LPDDR4_RON_DISABLE         (0)
+#define PHY_LPDDR4_RON_576ohm          (1)
+#define PHY_LPDDR4_RON_289ohm          (2)
+#define PHY_LPDDR4_RON_192ohm          (3)
+#define PHY_LPDDR4_RON_144ohm          (4)
```

```
+#define PHY_LPDDR4_RON_115ohm          (5)
+#define PHY_LPDDR4_RON_96ohm           (6)
+#define PHY_LPDDR4_RON_82ohm           (7)
+#define PHY_LPDDR4_RON_72ohm           (8)
+#define PHY_LPDDR4_RON_64ohm           (9)
+#define PHY_LPDDR4_RON_57ohm           (10)
+#define PHY_LPDDR4_RON_52ohm           (11)
+#define PHY_LPDDR4_RON_48ohm           (12)
+#define PHY_LPDDR4_RON_44ohm           (13)
+#define PHY_LPDDR4_RON_41ohm           (14)
+#define PHY_LPDDR4_RON_38ohm           (15)
+#define PHY_LPDDR4_RON_36ohm           (24)
+#define PHY_LPDDR4_RON_34ohm           (25)
+#define PHY_LPDDR4_RON_32ohm           (26)
+#define PHY_LPDDR4_RON_30ohm           (27)
+#define PHY_LPDDR4_RON_28ohm           (28)
+#define PHY_LPDDR4_RON_27ohm           (29)
+#define PHY_LPDDR4_RON_26ohm           (30)
+#define PHY_LPDDR4_RON_25ohm           (31)
+
+#define PHY_LPDDR4_RTT_DISABLE         (0)
+#define PHY_LPDDR4_RTT_576ohm          (1)
+#define PHY_LPDDR4_RTT_289ohm          (2)
+#define PHY_LPDDR4_RTT_192ohm          (3)
+#define PHY_LPDDR4_RTT_144ohm          (4)
+#define PHY_LPDDR4_RTT_115ohm          (5)
+#define PHY_LPDDR4_RTT_96ohm           (6)
+#define PHY_LPDDR4_RTT_82ohm           (7)
+#define PHY_LPDDR4_RTT_72ohm           (8)
+#define PHY_LPDDR4_RTT_64ohm           (9)
+#define PHY_LPDDR4_RTT_57ohm           (10)
+#define PHY_LPDDR4_RTT_52ohm           (11)
+#define PHY_LPDDR4_RTT_48ohm           (12)
+#define PHY_LPDDR4_RTT_44ohm           (13)
+#define PHY_LPDDR4_RTT_41ohm           (14)
+#define PHY_LPDDR4_RTT_38ohm           (15)
+#define PHY_LPDDR4_RTT_36ohm           (24)
+#define PHY_LPDDR4_RTT_34ohm           (25)
+#define PHY_LPDDR4_RTT_32ohm           (26)
+#define PHY_LPDDR4_RTT_30ohm           (27)
+#define PHY_LPDDR4_RTT_28ohm           (28)
+#define PHY_LPDDR4_RTT_27ohm           (29)
+#define PHY_LPDDR4_RTT_26ohm           (30)
+#define PHY_LPDDR4_RTT_25ohm           (31)
+
+#define PHY_LPDDR4X_RON_DISABLE                (0)
+#define PHY_LPDDR4X_RON_646ohm         (1)
+#define PHY_LPDDR4X_RON_323ohm         (2)
+#define PHY_LPDDR4X_RON_215ohm         (3)
+#define PHY_LPDDR4X_RON_162ohm         (4)
+#define PHY_LPDDR4X_RON_129ohm         (5)
+#define PHY_LPDDR4X_RON_108ohm         (6)
+#define PHY_LPDDR4X_RON_92ohm          (7)
+#define PHY_LPDDR4X_RON_81ohm          (8)
+#define PHY_LPDDR4X_RON_72ohm          (9)
+#define PHY_LPDDR4X_RON_65ohm          (10)
+#define PHY_LPDDR4X_RON_59ohm          (11)
+#define PHY_LPDDR4X_RON_54ohm          (12)
+#define PHY_LPDDR4X_RON_50ohm          (13)
+#define PHY_LPDDR4X_RON_46ohm          (14)
+#define PHY_LPDDR4X_RON_43ohm          (15)
+#define PHY_LPDDR4X_RON_40ohm          (24)
+#define PHY_LPDDR4X_RON_38ohm          (25)
+#define PHY_LPDDR4X_RON_36ohm          (26)
+#define PHY_LPDDR4X_RON_34ohm          (27)
```

```
+#define PHY_LPDDR4X_RON_32ohm          (28)
+#define PHY_LPDDR4X_RON_31ohm          (29)
+#define PHY_LPDDR4X_RON_29ohm          (30)
+#define PHY_LPDDR4X_RON_28ohm          (31)
+
+#define PHY_LPDDR4X_RTT_DISABLE                (0)
+#define PHY_LPDDR4X_RTT_646ohm         (1)
+#define PHY_LPDDR4X_RTT_323ohm         (2)
+#define PHY_LPDDR4X_RTT_215ohm         (3)
+#define PHY_LPDDR4X_RTT_162ohm         (4)
+#define PHY_LPDDR4X_RTT_129ohm         (5)
+#define PHY_LPDDR4X_RTT_108ohm         (6)
+#define PHY_LPDDR4X_RTT_92ohm          (7)
+#define PHY_LPDDR4X_RTT_81ohm          (8)
+#define PHY_LPDDR4X_RTT_72ohm          (9)
+#define PHY_LPDDR4X_RTT_65ohm          (10)
+#define PHY_LPDDR4X_RTT_59ohm          (11)
+#define PHY_LPDDR4X_RTT_54ohm          (12)
+#define PHY_LPDDR4X_RTT_50ohm          (13)
+#define PHY_LPDDR4X_RTT_46ohm          (14)
+#define PHY_LPDDR4X_RTT_43ohm          (15)
+#define PHY_LPDDR4X_RTT_40ohm          (24)
+#define PHY_LPDDR4X_RTT_38ohm          (25)
+#define PHY_LPDDR4X_RTT_36ohm          (26)
+#define PHY_LPDDR4X_RTT_34ohm          (27)
+#define PHY_LPDDR4X_RTT_32ohm          (28)
+#define PHY_LPDDR4X_RTT_31ohm          (29)
+#define PHY_LPDDR4X_RTT_29ohm          (30)
+#define PHY_LPDDR4X_RTT_28ohm          (31)
+
+#define PHY_DDR4_LPDDR3_RON_DISABLE    (0)
+#define PHY_DDR4_LPDDR3_RON_556ohm     (1)
+#define PHY_DDR4_LPDDR3_RON_279ohm     (2)
+#define PHY_DDR4_LPDDR3_RON_185ohm     (3)
+#define PHY_DDR4_LPDDR3_RON_139ohm     (4)
+#define PHY_DDR4_LPDDR3_RON_111ohm     (5)
+#define PHY_DDR4_LPDDR3_RON_93ohm      (6)
+#define PHY_DDR4_LPDDR3_RON_79ohm      (7)
+#define PHY_DDR4_LPDDR3_RON_69ohm      (8)
+#define PHY_DDR4_LPDDR3_RON_62ohm      (9)
+#define PHY_DDR4_LPDDR3_RON_55ohm      (10)
+#define PHY_DDR4_LPDDR3_RON_50ohm      (11)
+#define PHY_DDR4_LPDDR3_RON_46ohm      (12)
+#define PHY_DDR4_LPDDR3_RON_42ohm      (13)
+#define PHY_DDR4_LPDDR3_RON_39ohm      (14)
+#define PHY_DDR4_LPDDR3_RON_37ohm      (15)
+#define PHY_DDR4_LPDDR3_RON_34ohm      (24)
+#define PHY_DDR4_LPDDR3_RON_32ohm      (25)
+#define PHY_DDR4_LPDDR3_RON_31ohm      (26)
+#define PHY_DDR4_LPDDR3_RON_29ohm      (27)
+#define PHY_DDR4_LPDDR3_RON_27ohm      (28)
+#define PHY_DDR4_LPDDR3_RON_26ohm      (29)
+#define PHY_DDR4_LPDDR3_RON_25ohm      (30)
+#define PHY_DDR4_LPDDR3_RON_24ohm      (31)
+
+#define PHY_DDR4_LPDDR3_RTT_DISABLE    (0)
+#define PHY_DDR4_LPDDR3_RTT_556ohm     (1)
+#define PHY_DDR4_LPDDR3_RTT_279ohm     (2)
+#define PHY_DDR4_LPDDR3_RTT_185ohm     (3)
+#define PHY_DDR4_LPDDR3_RTT_139ohm     (4)
+#define PHY_DDR4_LPDDR3_RTT_111ohm     (5)
+#define PHY_DDR4_LPDDR3_RTT_93ohm      (6)
+#define PHY_DDR4_LPDDR3_RTT_79ohm      (7)
+#define PHY_DDR4_LPDDR3_RTT_69ohm      (8)
+#define PHY_DDR4_LPDDR3_RTT_62ohm      (9)
```

```
+#define PHY_DDR4_LPDDR3_RTT_55ohm      (10)
+#define PHY_DDR4_LPDDR3_RTT_50ohm      (11)
+#define PHY_DDR4_LPDDR3_RTT_46ohm      (12)
+#define PHY_DDR4_LPDDR3_RTT_42ohm      (13)
+#define PHY_DDR4_LPDDR3_RTT_39ohm      (14)
+#define PHY_DDR4_LPDDR3_RTT_37ohm      (15)
+#define PHY_DDR4_LPDDR3_RTT_34ohm      (24)
+#define PHY_DDR4_LPDDR3_RTT_32ohm      (25)
+#define PHY_DDR4_LPDDR3_RTT_31ohm      (26)
+#define PHY_DDR4_LPDDR3_RTT_29ohm      (27)
+#define PHY_DDR4_LPDDR3_RTT_27ohm      (28)
+#define PHY_DDR4_LPDDR3_RTT_26ohm      (29)
+#define PHY_DDR4_LPDDR3_RTT_25ohm      (30)
+#define PHY_DDR4_LPDDR3_RTT_24ohm      (31)
+
+#endif /* _ASM_ARCH_SDRAM_PHY_RON_RTT_RK3568_H */
diff --git a/arch/arm/include/asm/arch-rockchip/sdram_rk3568.h b/arch/arm/include/asm/arch-
rockchip/sdram_rk3568.h
new file mode 100644
index 00000000000..f858d3f8bc7
--- /dev/null
+++ b/arch/arm/include/asm/arch-rockchip/sdram_rk3568.h
@@ -0,0 +1,197 @@
+/* SPDX-License-Identifier:     GPL-2.0+ */
+/*
+ * Copyright (C) 2020 Rockchip Electronics Co., Ltd
+ * Copyright (c) 2024-2026, Pavel Golikov <paullo612@ya.ru>
+ */
+
+#ifndef _ASM_ARCH_SDRAM_RK3568_H
+#define _ASM_ARCH_SDRAM_RK3568_H
+
+#include <asm/arch-rockchip/dram_spec_timing.h>
+#include <asm/arch-rockchip/sdram.h>
+#include <asm/arch-rockchip/sdram_common.h>
+#include <asm/arch-rockchip/sdram_msch.h>
+#include <asm/arch-rockchip/sdram_pctl_px30.h>
+#include <asm/arch-rockchip/sdram_phy_rk3568.h>
+
+#define PATTERN                                (0x5aa5f00f)
+
+#define DPLL_MODE(n)                   ((0x3 << (2 + 16)) | ((n) << 2))
+
+/* CRU_CLKSEL_CON10 */
+#define CLK_MSCH_DIV_MASK              (3)
+#define CLK_MSCH_DIV_SHIFT             0
+
+/* CRU_DPLL_CON0 */
+#define RK3036_PLLCON0_FBDIV_MASK      0xfff
+#define RK3036_PLLCON0_FBDIV_SHIFT     0
+#define RK3036_PLLCON0_POSTDIV1_MASK   (0x7 << 12)
+#define RK3036_PLLCON0_POSTDIV1_SHIFT  12
+
+/* CRU_DPLL_CON1 */
+#define RK3036_PLLCON1_REFDIV_MASK     0x3f
+#define RK3036_PLLCON1_REFDIV_SHIFT    0
+#define RK3036_PLLCON1_POSTDIV2_MASK   (0x7 << 6)
+#define RK3036_PLLCON1_POSTDIV2_SHIFT  6
+#define RK3036_PLLCON1_DSMPD_MASK      (0x1 << 12)
+#define RK3036_PLLCON1_DSMPD_SHIFT     12
+#define RK3036_PLLCON1_PWRDOWN_SHIFT   13
+#define RK3568_PLLCON1_LOCK(n)         (((n) >> 10) & 0x1)
+
+/* CRU_DPLL_CON2 */
+#define RK3036_PLLCON2_FRAC_MASK       0xffffff
```

```
+#define RK3036_PLLCON2_FRAC_SHIFT      0
+
+/* CRU_DPLL_CON3 */
+#define SSMOD_SPREAD(n)                        ((0x1f << (8 + 16)) | ((n) << 8))
+#define SSMOD_DIVVAL(n)                        ((0xf << (4 + 16)) | ((n) << 4))
+#define SSMOD_DOWNSPREAD(n)            ((0x1 << (3 + 16)) | ((n) << 3))
+#define SSMOD_RESET(n)                 ((0x1 << (2 + 16)) | ((n) << 2))
+#define SSMOD_DIS_SSCG(n)              ((0x1 << (1 + 16)) | ((n) << 1))
+#define SSMOD_BP(n)                    ((0x1 << (0 + 16)) | ((n) << 0))
+
+/* DDR_GRF_CON0 */
+#define DFI_INIT_START                 BIT(2)
+#define DFI_INIT_START_BY_GRF_EN       BIT(1)
+
+/* DDR_GRF_SPLIT_CON */
+#define SPLIT_MODE_MASK                        (0x3)
+#define SPLIT_MODE_OFFSET              (9)
+#define SPLIT_BYPASS_MASK              (1)
+#define SPLIT_BYPASS_OFFSET            (8)
+#define SPLIT_SIZE_MASK                        (0xff)
+#define SPLIT_SIZE_OFFSET              (0)
+
+/* SGRF_SOC_CON5 */
+#define SGRF_SOC_CON5                  0x14
+#define UPCTL2_SRSTN_REQ_MASK          BIT(8)
+#define UPCTL2_SRSTN_REQ_SHIFT         8
+#define UPCTL2_ASRSTN_REQ_MASK         BIT(9)
+#define UPCTL2_ASRSTN_REQ_SHIFT                9
+#define UPCTL2_PSRSTN_REQ_MASK         BIT(11)
+#define UPCTL2_PSRSTN_REQ_SHIFT                11
+
+/* SCRU_SOFTRST_CON02 */
+#define SCRU_SOFTRST_CON02             0x208
+#define PRESETN_DDR_UPCTL_MASK         BIT(1)
+#define PRESETN_DDR_UPCTL_SHIFT                1
+
+/* CRU_SOFTRST_CON27 */
+#define PRESETN_DDRPHY_MASK            BIT(7)
+#define PRESETN_DDRPHY_SHIFT           7
+#define RESETN_DDRPHY_MASK             BIT(8)
+#define RESETN_DDRPHY_SHIFT            8
+
+/* DDR_GRF_CON3 */
+#define DQ_SWAP_EN_MASK                        BIT(7)
+#define DQ_SWAP_EN_SHIFT               7
+#define DQ_SWAP_SEL0_MASK              (3 << 8)
+#define DQ_SWAP_SEL0_SHIFT             8
+#define DQ_SWAP_SEL1_MASK              (3 << 10)
+#define DQ_SWAP_SEL1_SHIFT             10
+#define DQ_SWAP_SEL2_MASK              (3 << 12)
+#define DQ_SWAP_SEL2_SHIFT             12
+#define DQ_SWAP_SEL3_MASK              (3 << 14)
+#define DQ_SWAP_SEL3_SHIFT             14
+
+/* PMUGRF */
+#define PMUGRF_CON_DDRPHY_BUFFEREN_MASK                (0x3 << 12)
+#define PMUGRF_CON_DDRPHY_BUFFEREN_SHIFT       12
+#define PMUGRF_CON_DDRPHY_BUFFEREN_EN          0x1
+#define PMUGRF_CON_DDRPHY_BUFFEREN_DIS         0x2
+
+struct rk3568_ddrgrf {
+       u32 ddr_grf_con[5];
+       u32 grf_ddrsplit_con;
+       u32 reserved1[(0x20 - 0x14) / 4 - 1];
+       u32 ddr_grf_lp_con;
```

```
+       u32 reserved2[(0x100 - 0x20) / 4 - 1];
+       u32 ddr_grf_status[13];
+};
+
+struct msch_regs {
+       u32 coreid;
+       u32 revisionid;
+       u32 deviceconf;
+       u32 devicesize;
+       u32 ddrtiminga0;
+       u32 ddrtimingb0;
+       u32 ddrtimingc0;
+       u32 ddr4timing;
+       u32 devtodev0;
+       u32 ddrmode;
+       u32 reserved;
+       u32 agingx0;
+       u32 aging0;
+       u32 aging1;
+       u32 aging2;
+       u32 aging3;
+};
+
+struct sdram_msch_timings {
+       union noc_ddrtiminga0 ddrtiminga0;
+       union noc_ddrtimingb0 ddrtimingb0;
+       union noc_ddrtimingc0 ddrtimingc0;
+       union noc_ddr4timing ddr4timing;
+       union noc_devtodev_rv1126 devtodev0;
+       union noc_ddrmode ddrmode;
+
+       u32 agingx0;
+       u32 aging0;
+       u32 aging1;
+       u32 aging2;
+       u32 aging3;
+};
+
+struct rk3568_sdram_channel {
+       struct sdram_cap_info cap_info;
+       struct sdram_msch_timings noc_timings;
+};
+
+struct rk3568_sdram_params {
+       struct rk3568_sdram_channel ch;
+       struct sdram_base_params base;
+       struct ddr_pctl_regs pctl_regs;
+       struct rk3568_ddr_phy_regs phy_regs;
+};
+
+struct rk3568_fsp_param {
+       u32 flag;
+       u32 freq_mhz;
+
+       /* dram size */
+       u32 dq_odt;
+       u32 ca_odt;
+       u32 ds_pdds;
+       u32 vref_ca[2];
+       u32 vref_dq[2];
+
+       /* phy side */
+       u32 wr_dq_drv;
+       u32 wr_ca_drv;
+       u32 wr_ckcs_drv;
```

```
+       u32 rd_odt;
+       u32 rd_odt_up_en;
+       u32 rd_odt_down_en;
+       u32 vref_inner;
+       u32 vref_out;
+       u32 lp4_drv_pd_en;
+
+       u32 ca_prebit_skew[32];
+       u32 reserved0[28];
+
+       struct sdram_msch_timings noc_timings;
+       u32 reserved1;
+};
+
+#define MAX_IDX                         (4)
+#define FSP_FLAG                (0xfead0002)
+#define SHARE_MEM_BASE          (0x100000)
+/*
+ * Borrow share memory space to temporarily store FSP params.
+ * In the stage of DDR init write FSP params to this space.
+ * In the stage of trust init move FSP params to SRAM space from share memory space.
+ */
+#define FSP_PARAM_STORE_ADDR   (SHARE_MEM_BASE)
+
+#endif /* _ASM_ARCH_SDRAM_RK3568_H */
diff --git a/drivers/ram/rockchip/Makefile b/drivers/ram/rockchip/Makefile
index 27921ae4921..c3a39fce6e4 100644
--- a/drivers/ram/rockchip/Makefile
+++ b/drivers/ram/rockchip/Makefile
@@ -15,7 +15,7 @@ obj-$(CONFIG_ROCKCHIP_RK3328) = sdram_rk3328.o sdram_pctl_px30.o
sdram_phy_px30.
 obj-$(CONFIG_ROCKCHIP_RK3399) += sdram_rk3399.o
 obj-$(CONFIG_ROCKCHIP_RK3506) += sdram_rk3506.o
 obj-$(CONFIG_ROCKCHIP_RK3528) += sdram_rk3528.o
-obj-$(CONFIG_ROCKCHIP_RK3568) += sdram_rk3568.o
+obj-$(CONFIG_ROCKCHIP_RK3568) += sdram_rk3568.o sdram_pctl_px30.o
 obj-$(CONFIG_ROCKCHIP_RK3576) += sdram_rk3576.o
 obj-$(CONFIG_ROCKCHIP_RK3588) += sdram_rk3588.o
 obj-$(CONFIG_ROCKCHIP_RV1126) += sdram_rv1126.o sdram_pctl_px30.o
diff --git a/drivers/ram/rockchip/sdram_rk3568.c b/drivers/ram/rockchip/sdram_rk3568.c
index a252d5c7010..91ec7300af4 100644
--- a/drivers/ram/rockchip/sdram_rk3568.c
+++ b/drivers/ram/rockchip/sdram_rk3568.c
@@ -1,30 +1,4396 @@
 // SPDX-License-Identifier: GPL-2.0+
 /*
  * (C) Copyright 2021 Rockchip Electronics Co., Ltd.
+ * Copyright (c) 2022 Edgeble AI Technologies Pvt. Ltd.
+ * Copyright (c) 2024-2026, Pavel Golikov <paullo612@ya.ru>
  */

 #include <config.h>
+#include <debug_uart.h>
 #include <dm.h>
+#include <dt-structs.h>
+#include <hang.h>
 #include <ram.h>
 #include <syscon.h>
 #include <asm/arch-rockchip/clock.h>
+#include <asm/arch-rockchip/hardware.h>
+#include <asm/arch-rockchip/cru_rk3568.h>
 #include <asm/arch-rockchip/grf_rk3568.h>
-#include <asm/arch-rockchip/sdram.h>
+#include <asm/arch-rockchip/sdram_common.h>
+#include <asm/arch-rockchip/sdram_rk3568.h>
```

```
+#include <linux/delay.h>
+#include <linux/err.h>
+
+/* define training flag */
+#define CA_TRAINING            (0x1 << 0)
+#define READ_GATE_TRAINING     (0x1 << 1)
+#define WRITE_LEVELING         (0x1 << 2)
+#define WRITE_TRAINING         (0x1 << 3)
+#define READ_TRAINING          (0x1 << 4)
+#define FULL_TRAINING          (0xff)
+
+#define DESKEW_MDF_ABS_VAL     (0)
+#define DESKEW_MDF_DIFF_VAL    (1)
+
+#if defined(CONFIG_TPL_BUILD) || (!defined(CONFIG_TPL) && defined(CONFIG_SPL_BUILD))
+struct skew_info {
+       s16 t_dqss_min;
+       s16 t_dqss_max;
+       u32 max_freq;
+       u32 t_dqss2dq;
+       u8 clk_skew;
+       s8 clk_delta;
+       s8 dqs[2][5];
+};
+#endif

 struct dram_info {
+#if defined(CONFIG_TPL_BUILD) || (!defined(CONFIG_TPL) && defined(CONFIG_SPL_BUILD))
+       void __iomem *pctl;
+       void __iomem *phy;
+       void __iomem *sgrf;
+       struct rk3568_cru *cru;
+       struct msch_regs *msch;
+       struct rk3568_ddrgrf *ddrgrf;
+       struct rk3568_grf *grf;
+       u32 sr_idle;
+       u32 pd_idle;
+       u8 ext_temp_ref;
+       u8 derate_en;
+       u8 per_bank_ref_en;
+       u8 cmd_perbit_skew_bp;
+       struct skew_info skew_info;
+       u8 ecc_en;
+#endif
        struct ram_info info;
        struct rk3568_pmugrf *pmugrf;
 };

+#if defined(CONFIG_TPL_BUILD) || (!defined(CONFIG_TPL) && defined(CONFIG_SPL_BUILD))
+
+#if CONFIG_IS_ENABLED(OF_PLATDATA)
+struct rk3568_dmc_plat {
+       struct dtd_rockchip_rk3568_dmc of_plat;
+};
+#endif
+
+#define SCRU_BASE_ADDR 0xfdd10000
+
+struct dram_info dram_info;
+
+struct rk3568_sdram_params sdram_configs[] = {
+#if defined(CONFIG_RAM_ROCKCHIP_LPDDR4)
+# include      "sdram-rk3568-lpddr4-detect-324.inc"
+# include      "sdram-rk3568-lpddr4-detect-396.inc"
+# include      "sdram-rk3568-lpddr4-detect-528.inc"
```

```
+# include      "sdram-rk3568-lpddr4-detect-630.inc"
+# include      "sdram-rk3568-lpddr4-detect-780.inc"
+# include      "sdram-rk3568-lpddr4-detect-920.inc"
+# include      "sdram-rk3568-lpddr4-detect-1056.inc"
+# include      "sdram-rk3568-lpddr4-detect-1184.inc"
+# include      "sdram-rk3568-lpddr4-detect-1332.inc"
+# include      "sdram-rk3568-lpddr4-detect-1560.inc"
+#elif defined(CONFIG_RAM_ROCKCHIP_DDR4)
+# include      "sdram-rk3568-ddr4-detect-324.inc"
+# include      "sdram-rk3568-ddr4-detect-396.inc"
+# include      "sdram-rk3568-ddr4-detect-528.inc"
+# include      "sdram-rk3568-ddr4-detect-630.inc"
+# include      "sdram-rk3568-ddr4-detect-780.inc"
+# include      "sdram-rk3568-ddr4-detect-920.inc"
+# include      "sdram-rk3568-ddr4-detect-1056.inc"
+# include      "sdram-rk3568-ddr4-detect-1184.inc"
+# include      "sdram-rk3568-ddr4-detect-1332.inc"
+# include      "sdram-rk3568-ddr4-detect-1560.inc"
+#else
+#error "Unsupported RAM type"
+#endif
+};
+
+u32 common_info[] = {
+#include       "sdram-rk3568-loader_params.inc"
+};
+
+struct cb_invdelaysel_addr {
+       /* Byte shift from register start, LSB */
+       u8 byte : 2;
+       u8 reg  : 6;
+};
+
+static struct rk3568_fsp_param fsp_param[MAX_IDX];
+
+static u8 lp3_odt_value;
+
+static s16 wrlvl_result[2][5];
+
+/* DDR configuration 0-9 */
+static const u16 ddr_cfg_2_rbc[] = {
+       ((0 << 8) | (5 << 5) | (0 << 4) | (1 << 3) | 2), /* 0 */
+       ((1 << 8) | (5 << 5) | (0 << 4) | (1 << 3) | 1), /* 1 */
+       ((1 << 8) | (4 << 5) | (0 << 4) | (1 << 3) | 2), /* 2 */
+       ((1 << 8) | (3 << 5) | (0 << 4) | (1 << 3) | 3), /* 3 */
+       ((1 << 8) | (2 << 5) | (0 << 4) | (1 << 3) | 4), /* 4 */
+       ((0 << 8) | (2 << 5) | (1 << 4) | (1 << 3) | 4), /* 5 */
+       ((0 << 8) | (4 << 5) | (1 << 4) | (1 << 3) | 1), /* 6 */
+       ((0 << 8) | (4 << 5) | (1 << 4) | (1 << 3) | 2), /* 7 */
+       ((0 << 8) | (3 << 5) | (1 << 4) | (1 << 3) | 3), /* 8 */
+       ((1 << 8) | (5 << 5) | (0 << 4) | (0 << 3) | 2), /* 9 */
+};
+
+/* DDR configuration 10-20 */
+static const u8 ddr4_cfg_2_rbc[] = {
+       ((1 << 7) | (3 << 4) | (0 << 3) | (2 << 1) | 0), /* 10 */
+       ((1 << 7) | (4 << 4) | (0 << 3) | (2 << 1) | 1), /* 11 */
+       ((0 << 7) | (3 << 4) | (1 << 3) | (2 << 1) | 0), /* 12 */
+       ((0 << 7) | (4 << 4) | (1 << 3) | (2 << 1) | 1), /* 13 */
+
+       ((0 << 7) | (4 << 4) | (0 << 3) | (2 << 1) | 0), /* 18 */
+       ((0 << 7) | (4 << 4) | (0 << 3) | (1 << 1) | 0), /* 19 */
+       ((0 << 7) | (5 << 4) | (0 << 3) | (1 << 1) | 1), /* 20 */
+};
+
```

```
+static const u8 d4_rbc_2_d3_rbc[][2] = {
+       { 10, 3  },
+       { 11, 15 },
+       { 12, 8  },
+       { 13, 16 },
+
+       { 18, 2  },
+       { 19, 9  },
+       { 20, 17 },
+};
+
+static const u32 addrmap[21][9] = {
+       { 0x1f1f, 0x00080808, 0x00000000, 0x00000000, 0x00001f1f, 0x07070707, 0x07070707,
+         0x00000707, 0x0000 }, /* 0 */
+       { 0x1f18, 0x00070707, 0x00000000, 0x1f000000, 0x00001f1f, 0x06060606, 0x06060606,
+         0x00000606, 0x0000 }, /* 1 */
+       { 0x1f18, 0x00080808, 0x00000000, 0x00000000, 0x00001f1f, 0x07070707, 0x07070707,
+         0x00000f07, 0x0000 }, /* 2 */
+       { 0x1f18, 0x00090909, 0x00000000, 0x00000000, 0x00001f00, 0x08080808, 0x08080808,
+         0x00000f0f, 0x0000 }, /* 3 */
+       { 0x1f18, 0x000a0a0a, 0x00000000, 0x00000000, 0x00000000, 0x09090909, 0x0f090909,
+         0x00000f0f, 0x0000 }, /* 4 */
+       { 0x1f09, 0x000a0a0a, 0x00000000, 0x00000000, 0x00000000, 0x0a0a0a0a, 0x0f0a0a0a,
+         0x00000f0f, 0x0000 }, /* 5 */
+       { 0x1f06, 0x00070707, 0x00000000, 0x1f000000, 0x00001f1f, 0x07070707, 0x07070707,
+         0x00000f07, 0x0000 }, /* 6 */
+       { 0x1f07, 0x00080808, 0x00000000, 0x00000000, 0x00001f1f, 0x08080808, 0x08080808,
+         0x00000f08, 0x0000 }, /* 7 */
+       { 0x1f08, 0x00090909, 0x00000000, 0x00000000, 0x00001f00, 0x09090909, 0x09090909,
+         0x00000f0f, 0x0000 }, /* 8 */
+       { 0x1f18, 0x00000808, 0x00000000, 0x00000000, 0x00001f1f, 0x06060606, 0x06060606,
+         0x00000606, 0x0000 }, /* 9 */
+
+       { 0x1f18, 0x00000a0a, 0x00000700, 0x00000000, 0x00001f1f, 0x08080808, 0x08080808,
+         0x00000f0f, 0x0801 }, /* 10 */
+       { 0x1f18, 0x00000909, 0x00000700, 0x00000000, 0x00001f1f, 0x07070707, 0x07070707,
+         0x00000f07, 0x3f01 }, /* 11 */
+       { 0x1f08, 0x00000a0a, 0x00000700, 0x00000000, 0x00001f1f, 0x09090909, 0x09090909,
+         0x00000f0f, 0x0801 }, /* 12 */
+       { 0x1f07, 0x00000909, 0x00000700, 0x00000000, 0x00001f1f, 0x08080808, 0x08080808,
+         0x00000f08, 0x3f01 }, /* 13 */
+       { 0x1f15, 0x00060606, 0x00000000, 0x1f1f0000, 0x00001f1f, 0x05050505, 0x05050505,
+         0x00000f0f, 0x0000 }, /* 14 */
+       { 0x1f18, 0x00000909, 0x00000000, 0x00000000, 0x00001f00, 0x07070707, 0x07070707,
+         0x00000f07, 0x0000 }, /* 15 */
+       { 0x1f07, 0x00000909, 0x00000000, 0x00000000, 0x00001f00, 0x08080808, 0x08080808,
+         0x00000f08, 0x0000 }, /* 16 */
+       { 0x1f1f, 0x00090909, 0x00000000, 0x00000000, 0x00001f00, 0x08080808, 0x08080808,
+         0x00000f08, 0x0000 }, /* 17 */
+       { 0x1f18, 0x00000909, 0x00000007, 0x1f000000, 0x00001f1f, 0x07070707, 0x07070707,
+         0x00000f07, 0x0700 }, /* 18 */
+       { 0x1f18, 0x00000808, 0x00000007, 0x1f000000, 0x00001f1f, 0x06060606, 0x06060606,
+         0x00000606, 0x3f00 }, /* 19 */
+
+       { 0x1f1f, 0x00000a0a, 0x00000700, 0x00000000, 0x00001f1f, 0x08080808, 0x08080808,
+         0x00000f08, 0x0801 }, /* 20 */
+};
+
+static const struct cb_invdelaysel_addr cb_ca_sel[] = {
+       { .reg = 2, .byte = 0 }, /* CA0_A */
+       { .reg = 0, .byte = 1 }, /* CA1_A */
+       { .reg = 3, .byte = 0 }, /* CA2_A */
+       { .reg = 3, .byte = 3 }, /* CA3_A */
+       { .reg = 3, .byte = 1 }, /* CA4_A */
+       { .reg = 4, .byte = 3 }, /* CA5_A */
```

```
+       { .reg = 3, .byte = 2 }, /* CA0_B */
+       { .reg = 1, .byte = 1 }, /* CA1_B */
+       { .reg = 4, .byte = 1 }, /* CA2_B */
+       { .reg = 1, .byte = 3 }, /* CA3_B */
+       { .reg = 4, .byte = 0 }, /* CA4_B */
+       { .reg = 1, .byte = 2 }, /* CA5_B */
+};
+
+static const struct cb_invdelaysel_addr cb_ck_sel[] = {
+       { .reg = 6, .byte = 2 }, /* CK_A  */
+       { .reg = 0, .byte = 3 }, /* CK_B  */
+       { .reg = 6, .byte = 3 }, /* CKB_A */
+       { .reg = 2, .byte = 2 }, /* CKB_B */
+};
+
+static const struct cb_invdelaysel_addr cb_cs_sel[][6] = {
+       {
+               { .reg = 5, .byte = 1 }, /* CKE0_A */
+               { .reg = 2, .byte = 1 }, /* CKE0_B */
+               { .reg = 7, .byte = 3 }, /* CSB0_A */
+               { .reg = 6, .byte = 0 }, /* CSB0_B */
+               { .reg = 2, .byte = 3 }, /* ODT0_A */
+               { .reg = 1, .byte = 0 }, /* ODT0_B */
+
+       },
+       {
+               { .reg = 0, .byte = 0 }, /* CKE1_A */
+               { .reg = 7, .byte = 0 }, /* CKE1_B */
+               { .reg = 7, .byte = 2 }, /* CSB1_A */
+               { .reg = 6, .byte = 1 }, /* CSB1_B */
+               { .reg = 5, .byte = 2 }, /* ODT1_A */
+               { .reg = 5, .byte = 3 }, /* ODT1_B */
+       },
+};
+
+static void rkclk_ddr_reset(struct dram_info *dram, u32 ctl_srstn, u32 ctl_psrstn, u32
phy_srstn,
+                           u32 phy_psrstn)
+{
+       rk_clrsetreg(dram->sgrf + SGRF_SOC_CON5,
+                    (UPCTL2_SRSTN_REQ_MASK | UPCTL2_ASRSTN_REQ_MASK | UPCTL2_PSRSTN_REQ_MASK),
+                    (ctl_srstn << UPCTL2_SRSTN_REQ_SHIFT | ctl_srstn << UPCTL2_ASRSTN_REQ_SHIFT
|
+                     ctl_psrstn << UPCTL2_PSRSTN_REQ_SHIFT));
+
+       rk_clrsetreg(SCRU_BASE_ADDR + SCRU_SOFTRST_CON02, PRESETN_DDR_UPCTL_MASK,
+                    ctl_psrstn << PRESETN_DDR_UPCTL_SHIFT);
+
+       rk_clrsetreg(&dram->cru->softrst_con[27], (PRESETN_DDRPHY_MASK | RESETN_DDRPHY_MASK),
+                    (phy_psrstn << PRESETN_DDRPHY_SHIFT | phy_srstn << RESETN_DDRPHY_SHIFT));
+}
+
+static void rkclk_set_dpll(struct dram_info *dram, unsigned int hz)
+{
+       struct sdram_head_info_index_v2 *index = (struct sdram_head_info_index_v2 *)common_info;
+       unsigned int refdiv, postdiv1, postdiv2, fbdiv;
+       struct global_info *gbl_info;
+       u8 msch_clock_div_en;
+       int delay = 1000;
+       u32 ssmod_info;
+       u32 dsmpd = 1;
+       u32 mhz;
+
+       mhz = hz / MHz;
+       msch_clock_div_en = mhz <= 528 ? 0 : 1;
```

```
+
+       gbl_info = (struct global_info *)((void *)common_info + index->global_index.offset * 4);
+       ssmod_info = gbl_info->info_2t;
+       refdiv = 1;
+       if (mhz <= 100) {
+               postdiv1 = 6;
+               postdiv2 = 6;
+       } else if (mhz <= 150) {
+               postdiv1 = 6;
+               postdiv2 = 4;
+       } else if (mhz <= 200) {
+               postdiv1 = 4;
+               postdiv2 = 4;
+       } else if (mhz <= 300) {
+               postdiv1 = 6;
+               postdiv2 = 2;
+       } else if (mhz <= 400) {
+               postdiv1 = 4;
+               postdiv2 = 2;
+       } else if (mhz <= 600) {
+               postdiv1 = 6;
+               postdiv2 = 1;
+       } else if (mhz <= 800) {
+               postdiv1 = 4;
+               postdiv2 = 1;
+       } else {
+               postdiv1 = 2;
+               postdiv2 = 1;
+       }
+       fbdiv = (mhz * refdiv * postdiv1 * postdiv2) / 24;
+
+       /*
+        * See rk3036_pll_set_rate definition in drivers/clk/rockchip/clk_pll.c
+        *
+        * When power on or changing PLL setting,
+        * we must force PLL into slow mode to ensure output stable clock.
+        */
+       writel(DPLL_MODE(RKCLK_PLL_MODE_SLOW), &dram->cru->mode_con00);
+
+       /* Power down */
+       rk_setreg(&dram->cru->pll[DPLL].con1, 1 << RK3036_PLLCON1_PWRDOWN_SHIFT);
+
+       /* Setup memory schedule clock divider */
+       rk_clrsetreg(&dram->cru->clksel_con[10], CLK_MSCH_DIV_MASK,
+                    msch_clock_div_en << CLK_MSCH_DIV_SHIFT);
+
+       /* Setup dpll */
+       rk_clrsetreg(&dram->cru->pll[DPLL].con0,
+                    (RK3036_PLLCON0_POSTDIV1_MASK | RK3036_PLLCON0_FBDIV_MASK),
+                    (postdiv1 << RK3036_PLLCON0_POSTDIV1_SHIFT) |
+                            (fbdiv << RK3036_PLLCON0_FBDIV_SHIFT));
+
+       /* Enable ssmod, if required */
+       if (PLL_SSMOD_SPREAD(ssmod_info)) {
+               dsmpd = 0;
+               writel((readl(&dram->cru->pll[DPLL].con2) & (~RK3036_PLLCON2_FRAC_MASK)) |
+                              (0 << RK3036_PLLCON2_FRAC_SHIFT),
+                      &dram->cru->pll[DPLL].con2);
+
+               writel(SSMOD_SPREAD(PLL_SSMOD_SPREAD(ssmod_info)) |
+                              SSMOD_DIVVAL(PLL_SSMOD_DIV(ssmod_info)) |
+                              SSMOD_DOWNSPREAD(PLL_SSMOD_DOWNSPREAD(ssmod_info)) |
SSMOD_RESET(0) |
+                              SSMOD_DIS_SSCG(0) | SSMOD_BP(0),
+                      &dram->cru->pll[DPLL].con3);
```

```
+       }
+
+       rk_clrsetreg(&dram->cru->pll[DPLL].con1,
+                    RK3036_PLLCON1_DSMPD_MASK | RK3036_PLLCON1_POSTDIV2_MASK |
+                            RK3036_PLLCON1_REFDIV_MASK,
+                    (dsmpd << RK3036_PLLCON1_DSMPD_SHIFT) |
+                            (postdiv2 << RK3036_PLLCON1_POSTDIV2_SHIFT) |
+                            (refdiv << RK3036_PLLCON1_REFDIV_SHIFT));
+
+       /* Power Up */
+       rk_clrreg(&dram->cru->pll[DPLL].con1, 1 << RK3036_PLLCON1_PWRDOWN_SHIFT);
+
+       /* Wait for pll lock */
+       while (delay > 0) {
+               udelay(1);
+               if (RK3568_PLLCON1_LOCK(readl(&dram->cru->pll[DPLL].con1)))
+                       break;
+               delay--;
+       }
+
+       writel(DPLL_MODE(RKCLK_PLL_MODE_NORMAL), &dram->cru->mode_con00);
+}
+
+static void rkclk_configure_ddr(struct dram_info *dram, struct rk3568_sdram_params
*sdram_params)
+{
+       rk_clrreg(&dram->ddrgrf->ddr_grf_con[0], DFI_INIT_START_BY_GRF_EN);
+
+       /* for inno ddr phy need freq / 2 */
+       rkclk_set_dpll(dram, sdram_params->base.ddr_freq * MHZ / 2);
+}
+
+static u8 ext_temp_ref_from_index(struct global_info *gbl_info)
+{
+       u8 ext_temp_ref_index = DDR_EXT_TEMP_REF(gbl_info->info_2t);
+       u8 ext_temp_ref;
+
+       if (ext_temp_ref_index == 0)
+               /* TODO: 2x for 3568M/3568J */
+               ext_temp_ref = 1;
+       else if (ext_temp_ref_index == 1)
+               ext_temp_ref = 2;
+       else if (ext_temp_ref_index == 2)
+               ext_temp_ref = 4;
+       else
+               ext_temp_ref = 1;
+
+       return ext_temp_ref;
+}
+
+static unsigned int calculate_ddrconfig(struct rk3568_sdram_params *sdram_params)
+{
+       struct sdram_cap_info *cap_info = &sdram_params->ch.cap_info;
+       u32 cs, bw, die_bw, col, row, bank;
+       u32 cs1_row;
+       u32 i, tmp;
+       u32 ddrconf = -1;
+       u32 row_3_4;
+
+       cs = cap_info->rank;
+       bw = cap_info->bw;
+       die_bw = cap_info->dbw;
+       col = cap_info->col;
+       row = cap_info->cs0_row;
+       cs1_row = cap_info->cs1_row;
```

```
+       bank = cap_info->bk;
+       row_3_4 = cap_info->row_3_4;
+
+       if (sdram_params->base.dramtype == DDR4) {
+               if (cs == 2 && row == cs1_row) {
+                       tmp = ((row - 13) << 4) | (1 << 3) | (bw << 1) | die_bw;
+                       for (i = 12; i < 14; i++) {
+                               if (((tmp & 0xf) == (ddr4_cfg_2_rbc[i - 10] & 0xf)) &&
+                                   ((tmp & 0x70) <= (ddr4_cfg_2_rbc[i - 10] & 0x70))) {
+                                       ddrconf = i;
+                                       goto out;
+                               }
+                       }
+               }
+
+               tmp = ((cs - 1) << 7) | ((row - 13) << 4) | (bw << 1) | die_bw;
+               for (i = 10; i < 17; i++) {
+                       if (((tmp & 0xf) == (ddr4_cfg_2_rbc[i - 10] & 0xf)) &&
+                           ((tmp & 0x70) <= (ddr4_cfg_2_rbc[i - 10] & 0x70)) &&
+                           ((tmp & 0x80) <= (ddr4_cfg_2_rbc[i - 10] & 0x80))) {
+                               if (i < 4) {
+                                       ddrconf = i;
+                                       goto out;
+                               } else if (i == 14) {
+                                       ddrconf = 20;
+                                       goto out;
+                               } else {
+                                       ddrconf = i + 3;
+                                       goto out;
+                               }
+                       }
+               }
+       } else {
+               if (cs == 2 && row == cs1_row && bank == 3) {
+                       for (i = 5; i < 9; i++) {
+                               if (((bw + col - 10) == (ddr_cfg_2_rbc[i] & 0x7)) &&
+                                   ((row - 13) << 5) <= (ddr_cfg_2_rbc[i] & (0x7 << 5))) {
+                                       ddrconf = i;
+                                       goto out;
+                               }
+                       }
+               }
+
+               tmp = ((cs - 1) << 8) | ((row - 13) << 5) | ((bw + col - 10) << 0);
+               if (bank == 3)
+                       tmp |= (1 << 3);
+
+               for (i = 0; i < 9; i++) {
+                       if (((tmp & 0x1f) == (ddr_cfg_2_rbc[i] & 0x1f)) &&
+                           ((tmp & (7 << 5)) <= (ddr_cfg_2_rbc[i] & (7 << 5))) &&
+                           ((tmp & (1 << 8)) <= (ddr_cfg_2_rbc[i] & (1 << 8)))) {
+                               ddrconf = i;
+                               goto out;
+                       }
+               }
+
+               if (bank == 3 && (col + bw) == 10) {
+                       ddrconf = 14;
+                       goto out;
+               }
+
+               if (cs == 1 && bank == 3 && row <= 17 && (col + bw) == 13)
+                       ddrconf = 17;
+       }
+
```

```
+out:
+	if (ddrconf > 20)
+		printascii("calculate ddrconfig error\n");
+
+	if (sdram_params->base.dramtype == DDR4) {
+		for (i = 0; i < ARRAY_SIZE(d4_rbc_2_d3_rbc); i++) {
+			if (ddrconf == d4_rbc_2_d3_rbc[i][0]) {
+				if ((ddrconf == 10 || ddrconf == 12) && row > 16)
+					printascii("warn:ddrconf row > 16\n");
+				else
+					ddrconf = d4_rbc_2_d3_rbc[i][1];
+				break;
+			}
+		}
+	}
+
+	return ddrconf;
+}
+
+static void sw_set_req(struct dram_info *dram)
+{
+	void __iomem *pctl_base = dram->pctl;
+
+	/* clear sw_done=0 */
+	writel(PCTL2_SW_DONE_CLEAR, pctl_base + DDR_PCTL2_SWCTL);
+}
+
+static void sw_set_ack(struct dram_info *dram)
+{
+	void __iomem *pctl_base = dram->pctl;
+
+	/* set sw_done=1 */
+	writel(PCTL2_SW_DONE, pctl_base + DDR_PCTL2_SWCTL);
+	while (1) {
+		/* wait programming done */
+		if (readl(pctl_base + DDR_PCTL2_SWSTAT) & PCTL2_SW_DONE_ACK)
+			break;
+	}
+}
+
+static void set_ctl_address_map(struct dram_info *dram, struct rk3568_sdram_params
*sdram_params)
+{
+	struct sdram_cap_info *cap_info = &sdram_params->ch.cap_info;
+	void __iomem *pctl_base = dram->pctl;
+	u32 ddrconf = cap_info->ddrconfig;
+	u32 cs_pst, bg, i, row;
+
+	if (sdram_params->base.dramtype == DDR4)
+		/*
+		 * DDR4 8bit dram BG = 2(4bank groups),
+		 * 16bit dram BG = 1 (2 bank groups)
+		 */
+		bg = (cap_info->dbw == 0) ? 2 : 1;
+	else
+		bg = 0;
+
+	row = cap_info->cs0_row;
+	if (sdram_params->base.dramtype == DDR4) {
+		for (i = 0; i < ARRAY_SIZE(d4_rbc_2_d3_rbc); i++) {
+			if (ddrconf == d4_rbc_2_d3_rbc[i][1]) {
+				ddrconf = d4_rbc_2_d3_rbc[i][0];
+				break;
+			}
+		}
```

```
+       }
+
+       if (ddrconf >= ARRAY_SIZE(addrmap)) {
+               printascii("set ctl address map fail\n");
+               return;
+       }
+
+       sdram_copy_to_reg((u32 *)(pctl_base + DDR_PCTL2_ADDRMAP0), &addrmap[ddrconf][0],
+                         ARRAY_SIZE(addrmap[ddrconf]) * 4);
+
+       /* unused row set to 0xf */
+       for (i = 17; i >= row; i--)
+               setbits_le32(pctl_base + DDR_PCTL2_ADDRMAP6 + ((i - 12) * 8 / 32) * 4,
+                            0xf << ((i - 12) * 8 % 32));
+
+       if (sdram_params->base.dramtype == LPDDR3 && cap_info->row_3_4)
+               setbits_le32(pctl_base + DDR_PCTL2_ADDRMAP6, 1 << 31);
+       if (sdram_params->base.dramtype == DDR4 && cap_info->bw == 0x1)
+               setbits_le32(pctl_base + DDR_PCTL2_PCCFG, 1 << 8);
+
+       if (cap_info->rank == 1) {
+               clrsetbits_le32(pctl_base + DDR_PCTL2_ADDRMAP0, 0x1f, 0x1f);
+       } else if (cap_info->rank == 2 && cap_info->cs0_row != cap_info->cs1_row) {
+               cs_pst = cap_info->bw + cap_info->col + bg + cap_info->bk + cap_info->cs0_row;
+               clrsetbits_le32(pctl_base + DDR_PCTL2_ADDRMAP0, 0x1f, cs_pst - (6 + 2));
+       }
+}
+
+static void phy_pll_set(struct dram_info *dram, u32 freq, u32 dst_fsp)
+{
+       struct rk3568_ddrphy *phy = dram->phy;
+       u32 postdiv, postdiv_en;
+
+       freq /= MHz;
+
+       if (freq <= 50) {
+               postdiv = 5;
+               postdiv_en = 1;
+       } else if (freq <= 100) {
+               postdiv = 4;
+               postdiv_en = 1;
+       } else if (freq <= 200) {
+               postdiv = 3;
+               postdiv_en = 1;
+       } else if (freq <= 400) {
+               postdiv = 2;
+               postdiv_en = 1;
+       } else if (freq <= 800) {
+               postdiv = 1;
+               postdiv_en = 1;
+       } else {
+               postdiv = 0;
+               postdiv_en = 0;
+       }
+
+       clrsetbits_le32(&phy->reg33, PLL_POSTDIVN_EN_MASK(dst_fsp) | PLL_POSTDIVN_MASK(dst_fsp),
+                       (postdiv_en << PLL_POSTDIVN_EN_SHIFT(dst_fsp)) |
+                               (postdiv << PLL_POSTDIVN_SHIFT(dst_fsp)));
+}
+
+static const u16 d3_phy_drv_2_ohm[][2] = {
+       { PHY_DDR3_RON_500ohm, 500 },
+       { PHY_DDR3_RON_250ohm, 250 },
+       { PHY_DDR3_RON_167ohm, 167 },
+       { PHY_DDR3_RON_125ohm, 125 },
```

```
+       { PHY_DDR3_RON_100ohm, 100 },
+       { PHY_DDR3_RON_83ohm,  83  },
+       { PHY_DDR3_RON_71ohm,  71  },
+       { PHY_DDR3_RON_63ohm,  63  },
+       { PHY_DDR3_RON_56ohm,  56  },
+       { PHY_DDR3_RON_50ohm,  50  },
+       { PHY_DDR3_RON_45ohm,  45  },
+       { PHY_DDR3_RON_41ohm,  41  },
+       { PHY_DDR3_RON_38ohm,  38  },
+       { PHY_DDR3_RON_36ohm,  36  },
+       { PHY_DDR3_RON_33ohm,  33  },
+       { PHY_DDR3_RON_31ohm,  31  },
+       { PHY_DDR3_RON_29ohm,  29  },
+       { PHY_DDR3_RON_28ohm,  28  },
+       { PHY_DDR3_RON_26ohm,  26  },
+       { PHY_DDR3_RON_25ohm,  25  },
+       { PHY_DDR3_RON_24ohm,  24  },
+       { PHY_DDR3_RON_23ohm,  23  },
+       { PHY_DDR3_RON_22ohm,  22  },
+};
+
+static const u16 d3_phy_odt_2_ohm[][2] = {
+       { PHY_DDR3_RTT_DISABLE, 0   },
+       { PHY_DDR3_RTT_500ohm,  500 },
+       { PHY_DDR3_RTT_250ohm,  250 },
+       { PHY_DDR3_RTT_167ohm,  167 },
+       { PHY_DDR3_RTT_125ohm,  125 },
+       { PHY_DDR3_RTT_100ohm,  100 },
+       { PHY_DDR3_RTT_83ohm,   83  },
+       { PHY_DDR3_RTT_71ohm,   71  },
+       { PHY_DDR3_RTT_63ohm,   63  },
+       { PHY_DDR3_RTT_56ohm,   56  },
+       { PHY_DDR3_RTT_50ohm,   50  },
+       { PHY_DDR3_RTT_45ohm,   45  },
+       { PHY_DDR3_RTT_41ohm,   41  },
+       { PHY_DDR3_RTT_38ohm,   38  },
+       { PHY_DDR3_RTT_36ohm,   36  },
+       { PHY_DDR3_RTT_33ohm,   33  },
+       { PHY_DDR3_RTT_31ohm,   31  },
+       { PHY_DDR3_RTT_29ohm,   29  },
+       { PHY_DDR3_RTT_28ohm,   28  },
+       { PHY_DDR3_RTT_26ohm,   26  },
+       { PHY_DDR3_RTT_25ohm,   25  },
+       { PHY_DDR3_RTT_24ohm,   24  },
+       { PHY_DDR3_RTT_23ohm,   23  },
+       { PHY_DDR3_RTT_22ohm,   22  },
+};
+
+static const u16 lp4_phy_drv_2_ohm[][2] = {
+       { PHY_LPDDR4_RON_576ohm, 576 },
+       { PHY_LPDDR4_RON_289ohm, 289 },
+       { PHY_LPDDR4_RON_192ohm, 192 },
+       { PHY_LPDDR4_RON_144ohm, 144 },
+       { PHY_LPDDR4_RON_115ohm, 115 },
+       { PHY_LPDDR4_RON_96ohm,  96  },
+       { PHY_LPDDR4_RON_82ohm,  82  },
+       { PHY_LPDDR4_RON_72ohm,  72  },
+       { PHY_LPDDR4_RON_64ohm,  64  },
+       { PHY_LPDDR4_RON_57ohm,  57  },
+       { PHY_LPDDR4_RON_52ohm,  52  },
+       { PHY_LPDDR4_RON_48ohm,  48  },
+       { PHY_LPDDR4_RON_44ohm,  44  },
+       { PHY_LPDDR4_RON_41ohm,  41  },
+       { PHY_LPDDR4_RON_38ohm,  38  },
+       { PHY_LPDDR4_RON_36ohm,  36  },
```

```
+       { PHY_LPDDR4_RON_34ohm,  34  },
+       { PHY_LPDDR4_RON_32ohm,  32  },
+       { PHY_LPDDR4_RON_30ohm,  30  },
+       { PHY_LPDDR4_RON_28ohm,  28  },
+       { PHY_LPDDR4_RON_27ohm,  27  },
+       { PHY_LPDDR4_RON_26ohm,  26  },
+       { PHY_LPDDR4_RON_25ohm,  25  },
+};
+
+static const u16 lp4_phy_odt_2_ohm[][2] = {
+       { PHY_LPDDR4_RTT_DISABLE, 0   },
+       { PHY_LPDDR4_RTT_576ohm,  576 },
+       { PHY_LPDDR4_RTT_289ohm,  289 },
+       { PHY_LPDDR4_RTT_192ohm,  192 },
+       { PHY_LPDDR4_RTT_144ohm,  144 },
+       { PHY_LPDDR4_RTT_115ohm,  115 },
+       { PHY_LPDDR4_RTT_96ohm,   96  },
+       { PHY_LPDDR4_RTT_82ohm,   82  },
+       { PHY_LPDDR4_RTT_72ohm,   72  },
+       { PHY_LPDDR4_RTT_64ohm,   64  },
+       { PHY_LPDDR4_RTT_57ohm,   57  },
+       { PHY_LPDDR4_RTT_52ohm,   52  },
+       { PHY_LPDDR4_RTT_48ohm,   48  },
+       { PHY_LPDDR4_RTT_44ohm,   44  },
+       { PHY_LPDDR4_RTT_41ohm,   41  },
+       { PHY_LPDDR4_RTT_38ohm,   38  },
+       { PHY_LPDDR4_RTT_36ohm,   36  },
+       { PHY_LPDDR4_RTT_34ohm,   34  },
+       { PHY_LPDDR4_RTT_32ohm,   32  },
+       { PHY_LPDDR4_RTT_30ohm,   30  },
+       { PHY_LPDDR4_RTT_28ohm,   28  },
+       { PHY_LPDDR4_RTT_27ohm,   27  },
+       { PHY_LPDDR4_RTT_26ohm,   26  },
+       { PHY_LPDDR4_RTT_25ohm,   25  },
+};
+
+static const u16 lp4x_phy_drv_2_ohm[][2] = {
+       { PHY_LPDDR4X_RON_646ohm, 646 },
+       { PHY_LPDDR4X_RON_323ohm, 323 },
+       { PHY_LPDDR4X_RON_215ohm, 215 },
+       { PHY_LPDDR4X_RON_162ohm, 162 },
+       { PHY_LPDDR4X_RON_129ohm, 129 },
+       { PHY_LPDDR4X_RON_108ohm, 108 },
+       { PHY_LPDDR4X_RON_92ohm,  92  },
+       { PHY_LPDDR4X_RON_81ohm,  81  },
+       { PHY_LPDDR4X_RON_72ohm,  72  },
+       { PHY_LPDDR4X_RON_65ohm,  65  },
+       { PHY_LPDDR4X_RON_59ohm,  59  },
+       { PHY_LPDDR4X_RON_54ohm,  54  },
+       { PHY_LPDDR4X_RON_50ohm,  50  },
+       { PHY_LPDDR4X_RON_46ohm,  46  },
+       { PHY_LPDDR4X_RON_43ohm,  43  },
+       { PHY_LPDDR4X_RON_40ohm,  40  },
+       { PHY_LPDDR4X_RON_38ohm,  38  },
+       { PHY_LPDDR4X_RON_36ohm,  36  },
+       { PHY_LPDDR4X_RON_34ohm,  34  },
+       { PHY_LPDDR4X_RON_32ohm,  32  },
+       { PHY_LPDDR4X_RON_31ohm,  31  },
+       { PHY_LPDDR4X_RON_29ohm,  29  },
+       { PHY_LPDDR4X_RON_28ohm,  28  },
+};
+
+static const u16 lp4x_phy_odt_2_ohm[][2] = {
+       { PHY_LPDDR4X_RTT_DISABLE, 0   },
+       { PHY_LPDDR4X_RTT_646ohm,  646 },
```

```
+       { PHY_LPDDR4X_RTT_323ohm,  323 },
+       { PHY_LPDDR4X_RTT_215ohm,  215 },
+       { PHY_LPDDR4X_RTT_162ohm,  162 },
+       { PHY_LPDDR4X_RTT_129ohm,  129 },
+       { PHY_LPDDR4X_RTT_108ohm,  108 },
+       { PHY_LPDDR4X_RTT_92ohm,   92  },
+       { PHY_LPDDR4X_RTT_81ohm,   81  },
+       { PHY_LPDDR4X_RTT_72ohm,   72  },
+       { PHY_LPDDR4X_RTT_65ohm,   65  },
+       { PHY_LPDDR4X_RTT_59ohm,   59  },
+       { PHY_LPDDR4X_RTT_54ohm,   54  },
+       { PHY_LPDDR4X_RTT_50ohm,   50  },
+       { PHY_LPDDR4X_RTT_46ohm,   46  },
+       { PHY_LPDDR4X_RTT_43ohm,   43  },
+       { PHY_LPDDR4X_RTT_40ohm,   40  },
+       { PHY_LPDDR4X_RTT_38ohm,   38  },
+       { PHY_LPDDR4X_RTT_36ohm,   36  },
+       { PHY_LPDDR4X_RTT_34ohm,   34  },
+       { PHY_LPDDR4X_RTT_32ohm,   32  },
+       { PHY_LPDDR4X_RTT_31ohm,   31  },
+       { PHY_LPDDR4X_RTT_29ohm,   29  },
+       { PHY_LPDDR4X_RTT_28ohm,   28  },
+};
+
+static const u16 d4lp3_phy_drv_2_ohm[][2] = {
+       { PHY_DDR4_LPDDR3_RON_556ohm, 556 },
+       { PHY_DDR4_LPDDR3_RON_279ohm, 279 },
+       { PHY_DDR4_LPDDR3_RON_185ohm, 185 },
+       { PHY_DDR4_LPDDR3_RON_139ohm, 139 },
+       { PHY_DDR4_LPDDR3_RON_111ohm, 111 },
+       { PHY_DDR4_LPDDR3_RON_93ohm,  93  },
+       { PHY_DDR4_LPDDR3_RON_79ohm,  79  },
+       { PHY_DDR4_LPDDR3_RON_69ohm,  69  },
+       { PHY_DDR4_LPDDR3_RON_62ohm,  62  },
+       { PHY_DDR4_LPDDR3_RON_55ohm,  55  },
+       { PHY_DDR4_LPDDR3_RON_50ohm,  50  },
+       { PHY_DDR4_LPDDR3_RON_46ohm,  46  },
+       { PHY_DDR4_LPDDR3_RON_42ohm,  42  },
+       { PHY_DDR4_LPDDR3_RON_39ohm,  39  },
+       { PHY_DDR4_LPDDR3_RON_37ohm,  37  },
+       { PHY_DDR4_LPDDR3_RON_34ohm,  34  },
+       { PHY_DDR4_LPDDR3_RON_32ohm,  32  },
+       { PHY_DDR4_LPDDR3_RON_31ohm,  31  },
+       { PHY_DDR4_LPDDR3_RON_29ohm,  29  },
+       { PHY_DDR4_LPDDR3_RON_27ohm,  27  },
+       { PHY_DDR4_LPDDR3_RON_26ohm,  26  },
+       { PHY_DDR4_LPDDR3_RON_25ohm,  25  },
+       { PHY_DDR4_LPDDR3_RON_24ohm,  24  },
+};
+
+static const u16 d4lp3_phy_odt_2_ohm[][2] = {
+       { PHY_DDR4_LPDDR3_RTT_DISABLE, 0   },
+       { PHY_DDR4_LPDDR3_RTT_556ohm,  556 },
+       { PHY_DDR4_LPDDR3_RTT_279ohm,  279 },
+       { PHY_DDR4_LPDDR3_RTT_185ohm,  185 },
+       { PHY_DDR4_LPDDR3_RTT_139ohm,  139 },
+       { PHY_DDR4_LPDDR3_RTT_111ohm,  111 },
+       { PHY_DDR4_LPDDR3_RTT_93ohm,   93  },
+       { PHY_DDR4_LPDDR3_RTT_79ohm,   79  },
+       { PHY_DDR4_LPDDR3_RTT_69ohm,   69  },
+       { PHY_DDR4_LPDDR3_RTT_62ohm,   62  },
+       { PHY_DDR4_LPDDR3_RTT_55ohm,   55  },
+       { PHY_DDR4_LPDDR3_RTT_50ohm,   50  },
+       { PHY_DDR4_LPDDR3_RTT_46ohm,   46  },
+       { PHY_DDR4_LPDDR3_RTT_42ohm,   42  },
```

```
+       { PHY_DDR4_LPDDR3_RTT_39ohm,   39  },
+       { PHY_DDR4_LPDDR3_RTT_37ohm,   37  },
+       { PHY_DDR4_LPDDR3_RTT_34ohm,   34  },
+       { PHY_DDR4_LPDDR3_RTT_32ohm,   32  },
+       { PHY_DDR4_LPDDR3_RTT_31ohm,   31  },
+       { PHY_DDR4_LPDDR3_RTT_29ohm,   29  },
+       { PHY_DDR4_LPDDR3_RTT_27ohm,   27  },
+       { PHY_DDR4_LPDDR3_RTT_26ohm,   26  },
+       { PHY_DDR4_LPDDR3_RTT_25ohm,   25  },
+       { PHY_DDR4_LPDDR3_RTT_24ohm,   24  },
+};
+
+static u32 lp4_odt_calc(u32 odt_ohm)
+{
+       u32 odt;
+
+       if (odt_ohm == 0)
+               odt = LPDDR4_DQODT_DIS;
+       else if (odt_ohm <= 40)
+               odt = LPDDR4_DQODT_40;
+       else if (odt_ohm <= 48)
+               odt = LPDDR4_DQODT_48;
+       else if (odt_ohm <= 60)
+               odt = LPDDR4_DQODT_60;
+       else if (odt_ohm <= 80)
+               odt = LPDDR4_DQODT_80;
+       else if (odt_ohm <= 120)
+               odt = LPDDR4_DQODT_120;
+       else
+               odt = LPDDR4_DQODT_240;
+
+       return odt;
+}
+
+static void *get_ddr_drv_odt_info(u32 dramtype)
+{
+       struct sdram_head_info_index_v2 *index = (struct sdram_head_info_index_v2 *)common_info;
+       void *ddr_info = 0;
+
+       if (dramtype == DDR4)
+               ddr_info = (void *)common_info + index->ddr4_index.offset * 4;
+       else if (dramtype == DDR3)
+               ddr_info = (void *)common_info + index->ddr3_index.offset * 4;
+       else if (dramtype == LPDDR3)
+               ddr_info = (void *)common_info + index->lp3_index.offset * 4;
+       else if (dramtype == LPDDR4)
+               ddr_info = (void *)common_info + index->lp4_index.offset * 4;
+       else if (dramtype == LPDDR4X)
+               ddr_info = (void *)common_info + index->lp4x_index.offset * 4;
+       else
+               printascii("unsupported dram type\n");
+       return ddr_info;
+}
+
+static void set_lp4_vref(struct dram_info *dram, struct lp4_info *lp4_info, u32 freq_mhz,
+                        u32 dst_fsp, u32 dramtype)
+{
+       void __iomem *pctl_base = dram->pctl;
+       u32 ca_vref, dq_vref;
+
+       if (freq_mhz <= LP4_CA_ODT_EN_FREQ(lp4_info->ca_odten_freq))
+               ca_vref = LP4_CA_VREF(lp4_info->vref_when_odtoff);
+       else
+               ca_vref = LP4_CA_VREF(lp4_info->vref_when_odten);
+
```

```
+       if (freq_mhz <= LP4_DQ_ODT_EN_FREQ(lp4_info->dq_odten_freq))
+               dq_vref = LP4_DQ_VREF(lp4_info->vref_when_odtoff);
+       else
+               dq_vref = LP4_DQ_VREF(lp4_info->vref_when_odten);
+
+       if (dramtype == LPDDR4) {
+               if (ca_vref < 100)
+                       ca_vref = 100;
+               if (ca_vref > 420)
+                       ca_vref = 420;
+
+               if (ca_vref <= 300)
+                       ca_vref = (0 << 6) | (ca_vref - 100) / 4;
+               else
+                       ca_vref = (1 << 6) | (ca_vref - 220) / 4;
+
+               if (dq_vref < 100)
+                       dq_vref = 100;
+               if (dq_vref > 420)
+                       dq_vref = 420;
+
+               if (dq_vref <= 300)
+                       dq_vref = (0 << 6) | (dq_vref - 100) / 4;
+               else
+                       dq_vref = (1 << 6) | (dq_vref - 220) / 4;
+       } else {
+               ca_vref = ca_vref * 11 / 6;
+               if (ca_vref < 150)
+                       ca_vref = 150;
+               if (ca_vref > 629)
+                       ca_vref = 629;
+
+               if (ca_vref <= 449)
+                       ca_vref = (0 << 6) | (ca_vref - 150) / 4;
+               else
+                       ca_vref = (1 << 6) | (ca_vref - 329) / 4;
+
+               if (dq_vref < 150)
+                       dq_vref = 150;
+               if (dq_vref > 629)
+                       dq_vref = 629;
+
+               if (dq_vref <= 449)
+                       dq_vref = (0 << 6) | (dq_vref - 150) / 6;
+               else
+                       dq_vref = (1 << 6) | (dq_vref - 329) / 6;
+       }
+       sw_set_req(dram);
+       clrsetbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT6,
+                       PCTL2_MR_MASK << PCTL2_LPDDR4_MR12_SHIFT,
+                       ca_vref << PCTL2_LPDDR4_MR12_SHIFT);
+
+       clrsetbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT7,
+                       PCTL2_MR_MASK << PCTL2_LPDDR4_MR14_SHIFT,
+                       dq_vref << PCTL2_LPDDR4_MR14_SHIFT);
+       sw_set_ack(dram);
+}
+
+static void set_ds_odt(struct dram_info *dram, struct rk3568_sdram_params *sdram_params,
+                      u32 dst_fsp)
+{
+       struct rk3568_ddrphy *phy = dram->phy;
+       void __iomem *pctl_base = dram->pctl;
+       u32 dramtype = sdram_params->base.dramtype;
+       struct ddr2_3_4_lp2_3_info *ddr_info;
```

```
+       struct lp4_info *lp4_info;
+       u32 i, tmp;
+       const u16(*p_drv)[2];
+       const u16(*p_odt)[2];
+       u32 dram_odten_freq, phy_odten_freq;
+       u32 drv_info, sr_info;
+       u32 phy_dq_drv_ohm, phy_clk_drv_ohm, phy_ca_drv_ohm, dram_drv_ohm;
+       u32 phy_odt_ohm, dram_odt_ohm;
+       u32 lp4_pu_cal, phy_lp4_drv_pd_en;
+       u32 phy_odt_up_en, phy_odt_dn_en;
+       u32 sr_dq, sr_ca, sr_clk;
+       u32 freq = sdram_params->base.ddr_freq;
+       u32 mr1_mr3, mr6, mr11, mr22, vref_out, vref_inner;
+       u32 phy_clk_drv = 0, phy_odt = 0, phy_ca_drv = 0, dram_caodt_ohm = 0;
+       u32 phy_dq_drv = 0;
+       u32 phy_odt_up = 0, phy_odt_dn = 0;
+
+       ddr_info = get_ddr_drv_odt_info(dramtype);
+       lp4_info = (void *)ddr_info;
+
+       if (!ddr_info)
+               return;
+
+       dram_odten_freq = DRAMODT_EN_FREQ(ddr_info->odten_freq);
+       if (dram->cmd_perbit_skew_bp && dram_odten_freq > 700) {
+               dram_odten_freq = 700;
+               /* TODO: Needed? */
+               ddr_info->odten_freq &= ~(DRAM_ODT_EN_FREQ_MASK << DRAM_ODT_EN_FREQ_SHIFT);
+               ddr_info->odten_freq |= (dram_odten_freq << DRAM_ODT_EN_FREQ_SHIFT);
+       }
+
+       /* dram odt en freq control phy drv, dram odt and phy sr */
+       if (freq <= dram_odten_freq) {
+               drv_info = ddr_info->drv_when_odtoff;
+               sr_info  = ddr_info->sr_when_odtoff;
+
+               dram_odt_ohm = 0;
+
+               phy_lp4_drv_pd_en = PHY_LP4_DRV_PULLDOWN_EN_ODTOFF(lp4_info->odt_info);
+       } else {
+               drv_info = ddr_info->drv_when_odten;
+               sr_info  = ddr_info->sr_when_odten;
+
+               dram_odt_ohm = ODT_INFO_DRAM_ODT(ddr_info->odt_info);
+
+               phy_lp4_drv_pd_en = PHY_LP4_DRV_PULLDOWN_EN_ODTEN(lp4_info->odt_info);
+       }
+       phy_dq_drv_ohm  = DRV_INFO_PHY_DQ_DRV(drv_info);
+       phy_clk_drv_ohm = DRV_INFO_PHY_CLK_DRV(drv_info);
+       phy_ca_drv_ohm  = DRV_INFO_PHY_CA_DRV(drv_info);
+
+       sr_dq  = DQ_SR_INFO(sr_info);
+       sr_ca  = CA_SR_INFO(sr_info);
+       sr_clk = CLK_SR_INFO(sr_info);
+
+       phy_odten_freq = PHYODT_EN_FREQ(ddr_info->odten_freq);
+       if (dram->cmd_perbit_skew_bp && phy_odten_freq > 700) {
+               phy_odten_freq = 700;
+               /* TODO: Needed? */
+               ddr_info->odten_freq &= ~(PHY_ODT_EN_FREQ_MASK << PHY_ODT_EN_FREQ_SHIFT);
+               ddr_info->odten_freq |= (phy_odten_freq << PHY_ODT_EN_FREQ_SHIFT);
+       }
+
+       /* phy odt en freq control dram drv and phy odt */
+       if (freq <= phy_odten_freq) {
```

```
+                lp4_pu_cal = LP4_DRV_PU_CAL_ODTOFF(lp4_info->odt_info);
+
+                dram_drv_ohm = DRV_INFO_DRAM_DQ_DRV(ddr_info->drv_when_odtoff);
+
+                phy_odt_ohm   = 0;
+                phy_odt_up_en = 0;
+                phy_odt_dn_en = 0;
+
+                if (dramtype == LPDDR4 || dramtype == LPDDR4X) {
+                        for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i)
+                                clrbits_le32(&phy->pad_group[i].reg2, ABC_LH_RXEN_LP4);
+                }
+        } else {
+                lp4_pu_cal = LP4_DRV_PU_CAL_ODTEN(lp4_info->odt_info);
+
+                dram_drv_ohm = DRV_INFO_DRAM_DQ_DRV(ddr_info->drv_when_odten);
+
+                phy_odt_ohm   = ODT_INFO_PHY_ODT(ddr_info->odt_info);
+                phy_odt_up_en = ODT_INFO_PULLUP_EN(ddr_info->odt_info);
+                phy_odt_dn_en = ODT_INFO_PULLDOWN_EN(ddr_info->odt_info);
+
+                if (dramtype == LPDDR4 || dramtype == LPDDR4X) {
+                        for (i = 0; i < 20; ++i)
+                                setbits_le32(&phy->pad_group[i].reg2, ABC_LH_RXEN_LP4);
+                }
+        }
+
+        if (dramtype == LPDDR4 || dramtype == LPDDR4X) {
+                if (phy_odt_ohm) {
+                        phy_odt_up_en = 0;
+                        phy_odt_dn_en = 1;
+                }
+                if (freq <= LP4_CA_ODT_EN_FREQ(lp4_info->ca_odten_freq))
+                        dram_caodt_ohm = 0;
+                else
+                        dram_caodt_ohm = ODT_INFO_LP4_CA_ODT(lp4_info->odt_info);
+        }
+
+        /* Enable ZQCALIB bypass mode */
+        setbits_le32(&phy->reg22, ZQCALI_BYPASS);
+        clrbits_le32(&phy->reg22, PD_ZQCALI);
+        setbits_le32(&phy->reg22, PD_ZQCALI);
+        clrbits_le32(&phy->reg22, ZQCALI_BYPASS);
+
+        if (dramtype == DDR3) {
+                p_drv = d3_phy_drv_2_ohm;
+                p_odt = d3_phy_odt_2_ohm;
+        } else if (dramtype == LPDDR4) {
+                p_drv = lp4_phy_drv_2_ohm;
+                p_odt = lp4_phy_odt_2_ohm;
+        } else if (dramtype == LPDDR4X) {
+                p_drv = lp4x_phy_drv_2_ohm;
+                p_odt = lp4x_phy_odt_2_ohm;
+        } else {
+                p_drv = d4lp3_phy_drv_2_ohm;
+                p_odt = d4lp3_phy_odt_2_ohm;
+        }
+
+        for (i = ARRAY_SIZE(d3_phy_drv_2_ohm) - 1; i >= 0; --i) {
+                if (phy_dq_drv_ohm <= p_drv[i][1]) {
+                        phy_dq_drv = p_drv[i][0];
+                        break;
+                }
+        }
+
```

```
+	for (i = ARRAY_SIZE(d3_phy_drv_2_ohm) - 1; i >= 0; --i) {
+		if (phy_clk_drv_ohm <= p_drv[i][1]) {
+			phy_clk_drv = p_drv[i][0];
+			break;
+		}
+	}
+
+	for (i = ARRAY_SIZE(d3_phy_drv_2_ohm) - 1; i >= 0; --i) {
+		if (phy_ca_drv_ohm <= p_drv[i][1]) {
+			phy_ca_drv = p_drv[i][0];
+			break;
+		}
+	}
+
+	if (!phy_odt_ohm)
+		phy_odt = 0;
+	else
+		for (i = ARRAY_SIZE(d4lp3_phy_odt_2_ohm) - 1; i >= 0; --i) {
+			if (phy_odt_ohm <= p_odt[i][1]) {
+				phy_odt = p_odt[i][0];
+				break;
+			}
+		}
+
+	if (dramtype != LPDDR4 && dramtype != LPDDR4X) {
+		if (!phy_odt_ohm || (phy_odt_up_en && phy_odt_dn_en))
+			vref_inner = 0x100;
+		else if (phy_odt_up_en)
+			vref_inner = (2 * dram_drv_ohm + phy_odt_ohm) * 256 /
+				     (dram_drv_ohm + phy_odt_ohm);
+		else
+			vref_inner = phy_odt_ohm * 256 / (phy_odt_ohm + dram_drv_ohm);
+
+		if (dramtype != DDR3 && dram_odt_ohm)
+			vref_out = (2 * phy_dq_drv_ohm + dram_odt_ohm) * 256 /
+				   (phy_dq_drv_ohm + dram_odt_ohm);
+		else
+			vref_out = 0x100;
+
+		phy_lp4_drv_pd_en = 0;
+	} else {
+		/* for lp4 and lp4x*/
+		vref_inner = phy_odt_ohm
+			? (PHY_LP4_DQ_VREF(lp4_info->vref_when_odten)  * 512) / 1000
+			: (PHY_LP4_DQ_VREF(lp4_info->vref_when_odtoff) * 512) / 1000;
+
+		vref_out = dram_odt_ohm
+			? (LP4_DQ_VREF(lp4_info->vref_when_odten)  * 512) / 1000
+			: (LP4_DQ_VREF(lp4_info->vref_when_odtoff) * 512) / 1000;
+	}
+
+	clrsetbits_le32(&phy->reg38,
+			CMD_ABUTNRCOMP_MASK | CMD_ABUTPRCOMP_MASK | CMD_ABUTNRCOMP_CK0_MASK |
+				CMD_ABUTPRCOMP_CK0_MASK,
+			(phy_ca_drv << CMD_ABUTNRCOMP_SHIFT) |
+				(phy_ca_drv << CMD_ABUTPRCOMP_SHIFT) |
+				(phy_clk_drv << CMD_ABUTNRCOMP_CK0_SHIFT) |
+				(phy_clk_drv << CMD_ABUTPRCOMP_CK0_SHIFT));
+
+	if (dramtype == LPDDR4 || dramtype == LPDDR4X)
+		clrsetbits_le32(&phy->undocumented0,
+				CMD_ABUTNRCOMP_SPECIAL_MASK | CMD_ABUTPRCOMP_SPECIAL_MASK,
+				(phy_clk_drv << CMD_ABUTNRCOMP_SPECIAL_SHIFT) |
+					(phy_clk_drv << CMD_ABUTPRCOMP_SPECIAL_SHIFT));
+	else
```

```
+               clrsetbits_le32(&phy->undocumented0,
+                               CMD_ABUTNRCOMP_SPECIAL_MASK | CMD_ABUTPRCOMP_SPECIAL_MASK,
+                               (phy_ca_drv << CMD_ABUTNRCOMP_SPECIAL_SHIFT) |
+                                       (phy_ca_drv << CMD_ABUTPRCOMP_SPECIAL_SHIFT));
+
+       /* clk / cmd slew rate */
+       clrsetbits_le32(&phy->reg37, CMD_ABUTSLEWPU_MASK | CMD_ABUTSLEWPD_MASK,
+                       (sr_ca << CMD_ABUTSLEWPU_SHIFT) | (sr_clk << CMD_ABUTSLEWPD_SHIFT));
+
+       phy_lp4_drv_pd_en = (~phy_lp4_drv_pd_en) & 1;
+       if (phy_odt_up_en)
+               phy_odt_up = phy_odt;
+       if (phy_odt_dn_en)
+               phy_odt_dn = phy_odt;
+
+       for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i) {
+               clrsetbits_le32(&phy->pad_group[i].reg1,
+                               ABC_LH_ABUTNRCOMP_MASK | ABC_LH_ABUTPRCOMP_MASK |
+                                       ABC_LH_ABUTODTPD_MASK | ABC_LH_ABUTODTPU_MASK,
+                               (phy_dq_drv << ABC_LH_ABUTNRCOMP_SHIFT) |
+                                       (phy_dq_drv << ABC_LH_ABUTPRCOMP_SHIFT) |
+                                       (phy_odt_dn << ABC_LH_ABUTODTPD_SHIFT) |
+                                       (phy_odt_up << ABC_LH_ABUTODTPU_SHIFT));
+
+               clrsetbits_le32(&phy->pad_group[i].reg0,
+                               ABC_LH_VREF1_MARGSEL_MASK | ABC_LH_ABUTSLEWPU_MASK |
+                                       ABC_LH_ENB_LP4MODE_MASK,
+                               (vref_inner << ABC_LH_VREF1_MARGSEL_SHIFT) |
+                                       (sr_dq << ABC_LH_ABUTSLEWPU_SHIFT) |
+                                       (phy_lp4_drv_pd_en << ABC_LH_ENB_LP4MODE_SHIFT));
+       }
+
+       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG)) {
+               printf("PHY drv:clk:%d,ca:%d,DQ:%d,odt:%d\n", phy_clk_drv_ohm, phy_ca_drv_ohm,
+                      phy_dq_drv_ohm, phy_odt_ohm);
+               printf("vrefinner:%d%%, vrefout:%d%%\n", vref_inner * 100 / 512,
+                      vref_out * 100 / 512);
+               printf("dram drv:%d,odt:%d\n", dram_drv_ohm, dram_odt_ohm);
+       }
+
+       /* reg_rx_vref_value_update */
+       setbits_le32(&phy->reg24, RX_VREF_VALUE_UPDATE);
+       clrbits_le32(&phy->reg24, RX_VREF_VALUE_UPDATE);
+
+       if (dramtype == DDR4) {
+               mr6 = vref_out * 10000 / 512;
+
+               if (mr6 >= 7500)
+                       mr6 = (mr6 - 6000) / 65;
+               else
+                       mr6 = (1 << 6) | (mr6 - 4500) / 65;
+
+               sw_set_req(dram);
+               clrsetbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT7,
+                               PCTL2_MR_MASK << PCTL2_DDR4_MR6_SHIFT, mr6 <<
PCTL2_DDR4_MR6_SHIFT);
+               sw_set_ack(dram);
+               vref_out = 0x100;
+       }
+
+       /* RAM VREF */
+       clrsetbits_le32(&phy->undocumented0, RAM_VREF1_MARGSEL_MASK,
+                       vref_out << RAM_VREF1_MARGSEL_SHIFT);
+       if (dramtype == LPDDR3)
+               udelay(100);
```

```
+
+       if (dramtype == LPDDR4 || dramtype == LPDDR4X)
+               set_lp4_vref(dram, lp4_info, freq, dst_fsp, dramtype);
+
+       if (dramtype == DDR3 || dramtype == DDR4) {
+               mr1_mr3 = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT3);
+               mr1_mr3 = mr1_mr3 >> PCTL2_DDR34_MR1_SHIFT & PCTL2_MR_MASK;
+       } else {
+               mr1_mr3 = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT4);
+               mr1_mr3 = mr1_mr3 >> PCTL2_LPDDR234_MR3_SHIFT & PCTL2_MR_MASK;
+       }
+
+       if (dramtype == DDR3) {
+               mr1_mr3 &= ~(DDR3_DS_MASK | DDR3_RTT_NOM_MASK);
+               if (dram_drv_ohm == 34)
+                       mr1_mr3 |= DDR3_DS_34;
+
+               if (dram_odt_ohm == 0)
+                       mr1_mr3 |= DDR3_RTT_NOM_DIS;
+               else if (dram_odt_ohm <= 40)
+                       mr1_mr3 |= DDR3_RTT_NOM_40;
+               else if (dram_odt_ohm <= 60)
+                       mr1_mr3 |= DDR3_RTT_NOM_60;
+               else
+                       mr1_mr3 |= DDR3_RTT_NOM_120;
+
+       } else if (dramtype == DDR4) {
+               mr1_mr3 &= ~(DDR4_DS_MASK | DDR4_RTT_NOM_MASK);
+               if (dram_drv_ohm == 48)
+                       mr1_mr3 |= DDR4_DS_48;
+
+               if (dram_odt_ohm == 0)
+                       mr1_mr3 |= DDR4_RTT_NOM_DIS;
+               else if (dram_odt_ohm <= 34)
+                       mr1_mr3 |= DDR4_RTT_NOM_34;
+               else if (dram_odt_ohm <= 40)
+                       mr1_mr3 |= DDR4_RTT_NOM_40;
+               else if (dram_odt_ohm <= 48)
+                       mr1_mr3 |= DDR4_RTT_NOM_48;
+               else if (dram_odt_ohm <= 60)
+                       mr1_mr3 |= DDR4_RTT_NOM_60;
+               else
+                       mr1_mr3 |= DDR4_RTT_NOM_120;
+
+       } else if (dramtype == LPDDR3) {
+               if (dram_drv_ohm <= 34)
+                       mr1_mr3 |= LPDDR3_DS_34;
+               else if (dram_drv_ohm <= 40)
+                       mr1_mr3 |= LPDDR3_DS_40;
+               else if (dram_drv_ohm <= 48)
+                       mr1_mr3 |= LPDDR3_DS_48;
+               else if (dram_drv_ohm <= 60)
+                       mr1_mr3 |= LPDDR3_DS_60;
+               else if (dram_drv_ohm <= 80)
+                       mr1_mr3 |= LPDDR3_DS_80;
+
+               if (dram_odt_ohm == 0)
+                       lp3_odt_value = LPDDR3_ODT_DIS;
+               else if (dram_odt_ohm <= 60)
+                       lp3_odt_value = LPDDR3_ODT_60;
+               else if (dram_odt_ohm <= 120)
+                       lp3_odt_value = LPDDR3_ODT_120;
+               else
+                       lp3_odt_value = LPDDR3_ODT_240;
+       } else { /* for lpddr4 and lpddr4x */
```

```
+               if (dram_odt_ohm) {
+                       /* TODO: Needed? */
+                       sw_set_req(dram);
+                       clrsetbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) +
DDR_PCTL2_RANKCTL,
+                                       PCTL2_DIFF_RANK_WR_GAP_MASK,
+                                       5 << PCTL2_DIFF_RANK_WR_GAP_SHIFT);
+                       sw_set_ack(dram);
+               }
+               /* MR3 for lp4 PU-CAL and PDDS */
+               mr1_mr3 &= ~(LPDDR4_PDDS_MASK | LPDDR4_PU_CAL_MASK);
+               mr1_mr3 |= lp4_pu_cal;
+
+               tmp = lp4_odt_calc(dram_drv_ohm);
+               if (!tmp)
+                       tmp = LPDDR4_PDDS_240;
+               mr1_mr3 |= (tmp << LPDDR4_PDDS_SHIFT);
+
+               /* MR11 for lp4 ca odt, dq odt set */
+               mr11 = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT6);
+               mr11 = mr11 >> PCTL2_LPDDR4_MR11_SHIFT & PCTL2_MR_MASK;
+
+               mr11 &= ~(LPDDR4_DQODT_MASK | LPDDR4_CAODT_MASK);
+
+               tmp = lp4_odt_calc(dram_odt_ohm);
+               mr11 |= (tmp << LPDDR4_DQODT_SHIFT);
+
+               tmp = lp4_odt_calc(dram_caodt_ohm);
+               mr11 |= (tmp << LPDDR4_CAODT_SHIFT);
+               sw_set_req(dram);
+               clrsetbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT6,
+                               PCTL2_MR_MASK << PCTL2_LPDDR4_MR11_SHIFT,
+                               mr11 << PCTL2_LPDDR4_MR11_SHIFT);
+               sw_set_ack(dram);
+
+               /* MR22 for soc odt/odt-ck/odt-cs/odt-ca */
+               mr22 = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT7);
+               mr22 = mr22 >> PCTL2_LPDDR4_MR22_SHIFT & PCTL2_MR_MASK;
+               mr22 &= ~LPDDR4_SOC_ODT_MASK;
+
+               tmp = lp4_odt_calc(phy_odt_ohm);
+               mr22 |= tmp;
+               mr22 = mr22 |
+                      (LP4_ODTE_CK_EN(lp4_info->cs_drv_ca_odt_info) << LPDDR4_ODTE_CK_SHIFT) |
+                      (LP4_ODTE_CS_EN(lp4_info->cs_drv_ca_odt_info) << LPDDR4_ODTE_CS_SHIFT) |
+                      (LP4_ODTD_CA_EN(lp4_info->cs_drv_ca_odt_info) << LPDDR4_ODTD_CA_SHIFT);
+
+               /* TODO: Needed? */
+               if (!sdram_params->ch.cap_info.dbw)
+                       mr22 |= 0x80;
+
+               sw_set_req(dram);
+               clrsetbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT7,
+                               PCTL2_MR_MASK << PCTL2_LPDDR4_MR22_SHIFT,
+                               mr22 << PCTL2_LPDDR4_MR22_SHIFT);
+               sw_set_ack(dram);
+       }
+
+       if (dramtype == DDR4 || dramtype == DDR3) {
+               sw_set_req(dram);
+               clrsetbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT3,
+                               PCTL2_MR_MASK << PCTL2_DDR34_MR1_SHIFT,
+                               mr1_mr3 << PCTL2_DDR34_MR1_SHIFT);
+               sw_set_ack(dram);
+       } else {
```

```
+               sw_set_req(dram);
+               clrsetbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT4,
+                               PCTL2_MR_MASK << PCTL2_LPDDR234_MR3_SHIFT,
+                               mr1_mr3 << PCTL2_LPDDR234_MR3_SHIFT);
+               sw_set_ack(dram);
+       }
+}
+
+static void phy_set_fb1xclk_invdela(struct dram_info *dram, unsigned int freq)
+{
+       struct rk3568_ddrphy *phy = dram->phy;
+       u8 fb1xclk_invdela;
+
+       if (freq <= 1066)
+               fb1xclk_invdela = 23;
+       else
+               fb1xclk_invdela = 16;
+
+       clrsetbits_le32(&phy->reg5, FB1XCLK_INVDELA_MASK, fb1xclk_invdela <<
FB1XCLK_INVDELA_SHIFT);
+}
+
+static u8 sdram_cmd_dq_map_get(struct rk3568_sdram_params *sdram_params)
+{
+       struct sdram_head_info_index_v2 *index = (struct sdram_head_info_index_v2 *)common_info;
+       u32 dramtype = sdram_params->base.dramtype;
+       struct dq_map_info *map_info;
+       u8 dq_map = 0;
+
+       /* Same configuration for LPDDR4 and LPDDR4X */
+       if (dramtype == LPDDR4X)
+               dramtype = LPDDR4;
+
+       if (dramtype <= LPDDR4) {
+               map_info = (struct dq_map_info *)((void *)common_info +
+                                                 index->dq_map_index.offset * 4);
+               dq_map = (map_info->byte_map[dramtype / 4] >> ((dramtype % 4) * 8)) & 0xff;
+       }
+
+       return dq_map;
+}
+
+static void sdram_cmd_dq_path_remap(struct dram_info *dram, u8 dq_map)
+{
+       u8 dq_swap_en;
+
+       if (dq_map != 0) {
+               /*
+                * There is no sense in enabling swapping support on hardware level if dqs are
not
+                * configured to be swapped.
+                */
+               dq_swap_en = dq_map != ((0x3 << 6) | (0x2 << 4) | (0x1 << 2) | (0x0 << 0)) ? 1 :
0;
+
+               rk_clrsetreg(&dram->ddrgrf->ddr_grf_con[3],
+                            (DQ_SWAP_SEL3_MASK | DQ_SWAP_SEL2_MASK | DQ_SWAP_SEL1_MASK |
+                             DQ_SWAP_SEL0_MASK | DQ_SWAP_EN_MASK),
+                            (dq_map << DQ_SWAP_SEL0_SHIFT) | (dq_swap_en << DQ_SWAP_EN_SHIFT));
+       } else {
+               rk_clrreg(&dram->ddrgrf->ddr_grf_con[3], DQ_SWAP_EN_MASK);
+       }
+}
+
+static void phy_cfg(struct dram_info *dram, struct rk3568_sdram_params *sdram_params, u32
```

```
post_init)
+{
+       struct sdram_cap_info *cap_info = &sdram_params->ch.cap_info;
+       u32 dramtype = sdram_params->base.dramtype;
+       struct rk3568_ddrphy *phy = dram->phy;
+       void __iomem *phy_base = dram->phy;
+       u32 byte1 = 0, byte0 = 0;
+       u8 dq_map, rank4_enable;
+       u32 i, dq_enable;
+
+       phy_set_fb1xclk_invdela(dram, sdram_params->base.ddr_freq);
+
+       dq_map = sdram_cmd_dq_map_get(sdram_params);
+
+       /* Configure DQ pad groups mapping */
+       sdram_cmd_dq_path_remap(dram, dq_map);
+
+       phy_pll_set(dram, sdram_params->base.ddr_freq * MHZ, 0);
+       for (i = 0; sdram_params->phy_regs.phy[i][0] != 0xffffffff; i++) {
+               if (sdram_params->phy_regs.phy[i][0] == 0xc)
+                       clrsetbits_le32(phy_base + sdram_params->phy_regs.phy[i][0], 0x3f << 24,
+                                       sdram_params->phy_regs.phy[i][1]);
+               else
+                       writel(sdram_params->phy_regs.phy[i][1],
+                              phy_base + sdram_params->phy_regs.phy[i][0]);
+       }
+
+       /* Enable signals of DQ pad groups */
+       dq_enable = 0;
+
+       if (cap_info->bw == 2) {
+               /* Bus width is 32, just enable all four pad groups */
+               dq_enable = (1 << 3) | (1 << 2) | (1 << 1) | (1 << 0);
+       } else if (dq_map != 0) {
+               /*
+                * Bus width is lower than 32. Let's find appropriate pad groups to enable in DQ
+                * map configuration.
+                */
+               for (i = 0; i < 4; i++) {
+                       if (((dq_map >> (i * 2)) & 0x3) == 0)
+                               byte0 = i;
+                       if (((dq_map >> (i * 2)) & 0x3) == 1)
+                               byte1 = i;
+               }
+
+               if (cap_info->bw == 1)
+                       dq_enable |= ((1 << byte0) | (1 << byte1));
+               else
+                       dq_enable |= (1 << byte0);
+       }
+
+       if (dram->ecc_en)
+               dq_enable |= (1 << 4);
+
+       rank4_enable = (cap_info->rank == 4) || (post_init == 0 && dramtype == LPDDR3);
+
+       clrsetbits_le32(&phy->reg0, CHANNEL_EN_MASK | RANK4_EN_MASK,
+                       (dq_enable << CHANNEL_EN_SHIFT) | (rank4_enable << RANK4_EN_SHIFT));
+
+       if (dram->cmd_perbit_skew_bp)
+               setbits_le32(&phy->regd, CMD_PERBIT_SKEW_BP);
+
+       /* lpddr4 odt control by phy, enable cs0 odt */
+       if (sdram_params->base.dramtype == LPDDR4)
+               clrsetbits_le32(&phy->regd,
```

```
+                               LPDDR_CA_ODT_SEL_MASK | LPDDR_CA_ODT0_MASK | LPDDR_CA_ODT1_MASK,
+                               (1 << LPDDR_CA_ODT_SEL_SHIFT) | (1 << LPDDR_CA_ODT0_SHIFT) |
+                                       (1 << LPDDR_CA_ODT1_SHIFT));
+
+       if (dramtype == DDR3 || dramtype == LPDDR3 || dramtype == DDR4) {
+               for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i)
+                       clrbits_le32(&phy->pad_group[i].reg2, ABC_LH_RXEN_LP4);
+       } else if (dramtype == LPDDR4) {
+               for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i) {
+                       clrsetbits_le32(&phy->pad_group[i].reg0, ABC_LH_DQSWEAKP_MASK,
+                                       ABC_LH_DQSWEAKP_HIGH_Z << ABC_LH_DQSWEAKP_SHIFT);
+               }
+       } else if (dramtype == LPDDR4X) {
+               for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i) {
+                       clrsetbits_le32(&phy->pad_group[i].reg0, ABC_LH_DQSWEAKP_MASK,
+                                       ABC_LH_DQSWEAKP_HIGH_Z << ABC_LH_DQSWEAKP_SHIFT);
+                       setbits_le32(&phy->pad_group[i].reg2, ABC_LH_LP4X_EN);
+               }
+       }
+
+       /* Disable pvt compensation */
+       setbits_le32(&phy->reg2f, PVT_COMP_DIS);
+
+       /* Enable t2, if requested */
+       if (((sdram_params->pctl_regs.pctl[0][1] >> 10) & 1) != 0) {
+               clrsetbits_le32(&phy->reg2f, CMD_T2_MODE_MASK | CMD_DELAY_ONE_UI_MASK,
+                               (1 << CMD_T2_MODE_SHIFT) | (1 << CMD_DELAY_ONE_UI_SHIFT));
+       }
+
+       setbits_le32(&phy->reg2a, FREQ_CHOOSE_B_EN);
+       clrsetbits_le32(&phy->reg10_1, FREQ_CHOOSE_T_MASK, 0 << FREQ_CHOOSE_T_SHIFT);
+}
+
+static int update_refresh_reg(struct dram_info *dram)
+{
+       void __iomem *pctl_base = dram->pctl;
+       u32 ret;
+
+       ret = readl(pctl_base + DDR_PCTL2_RFSHCTL3) ^ (1 << 1);
+       writel(ret, pctl_base + DDR_PCTL2_RFSHCTL3);
+
+       return 0;
+}
+
+/*
+ * rank = 1: cs0
+ * rank = 2: cs1
+ */
+int read_mr(struct dram_info *dram, u32 rank, u32 mr_num, u32 dramtype)
+{
+       struct sdram_head_info_index_v2 *index = (struct sdram_head_info_index_v2 *)common_info;
+       void __iomem *pctl_base = dram->pctl;
+       struct dq_map_info *map_info;
+       u32 i, temp;
+       u32 dqmap;
+       u32 ret;
+
+       map_info = (struct dq_map_info *)((void *)common_info + index->dq_map_index.offset * 4);
+
+       if (dramtype == LPDDR2)
+               dqmap = map_info->lp2_dq0_7_map;
+       else
+               dqmap = map_info->lp3_dq0_7_map;
+
+       pctl_read_mr(pctl_base, rank, mr_num);
```

```
+
+       ret = (readl(&dram->ddrgrf->ddr_grf_status[0]) & 0xff);
+
+       if (dramtype != LPDDR4 && dramtype != LPDDR4X) {
+               temp = 0;
+               for (i = 0; i < 8; i++)
+                       temp = temp | (((ret >> i) & 0x1) << ((dqmap >> (i * 4)) & 0xf));
+       } else {
+               temp = ((readl(&dram->ddrgrf->ddr_grf_status[1]) >> 8) & 0xff);
+       }
+
+       return temp;
+}
+
+/* before call this function autorefresh should be disabled */
+void send_a_refresh(struct dram_info *dram)
+{
+       void __iomem *pctl_base = dram->pctl;
+
+       while ((readl(pctl_base + DDR_PCTL2_DBGSTAT) >> 4) & 1)
+               continue;
+
+       writel(0x10, pctl_base + DDR_PCTL2_DBGCMD);
+
+       while ((readl(pctl_base + DDR_PCTL2_DBGSTAT) >> 4) & 1)
+               continue;
+}
+
+static void enter_sr(struct dram_info *dram, u32 en)
+{
+       void __iomem *pctl_base = dram->pctl;
+       u32 stat;
+
+       if (en) {
+               setbits_le32(pctl_base + DDR_PCTL2_PWRCTL, PCTL2_SELFREF_SW);
+               do {
+                       stat = readl(pctl_base + DDR_PCTL2_STAT);
+               } while ((stat & PCTL2_SELFREF_TYPE_MASK) != PCTL2_SELFREF_TYPE_SR_NOT_AUTO ||
+                        (stat & PCTL2_OPERATING_MODE_MASK) != PCTL2_OPERATING_MODE_SR);
+       } else {
+               clrbits_le32(pctl_base + DDR_PCTL2_PWRCTL, PCTL2_SELFREF_SW);
+               do {
+                       stat = readl(pctl_base + DDR_PCTL2_STAT);
+               } while ((stat & PCTL2_OPERATING_MODE_MASK) == PCTL2_OPERATING_MODE_SR);
+       }
+}
+
+static u8 read_ca_prebit(struct rk3568_ddrphy *phy, struct cb_invdelaysel_addr addr)
+{
+       return (readl(&phy->reg_n_invdelaysel[addr.reg]) >> 8 * addr.byte) & 0xff;
+}
+
+static void record_ca_prebit(struct rk3568_ddrphy *phy, u32 dir, int delta,
+                            struct cb_invdelaysel_addr addr)
+{
+       if (dir != DESKEW_MDF_ABS_VAL)
+               delta += read_ca_prebit(phy, addr);
+
+       clrsetbits_le32(&phy->reg_n_invdelaysel[addr.reg], 0xff << 8 * addr.byte,
+                       (delta & 0xff) << 8 * addr.byte);
+}
+
+static void update_ca_prebit(struct rk3568_ddrphy *phy, u32 cs)
+{
+       /* Selecting a rank to apply skew for */
```

```
+       clrsetbits_le32(&phy->reg8, CAT_BP_RANK_SEL_MASK,
+                       ((cs == 0) ? CAT_BP_RANK_SEL_RANK0 : CAT_BP_RANK_SEL_RANK1)
+                               << CAT_BP_RANK_SEL_SHIFT);
+
+       setbits_le32(&phy->regd, CA_PERBIT_SKEW_UPDATE);
+       udelay(1);
+       clrbits_le32(&phy->regd, CA_PERBIT_SKEW_UPDATE);
+}
+
+static inline bool cb_invdelaysel_addr_cmp(struct cb_invdelaysel_addr a,
+                                           struct cb_invdelaysel_addr b)
+{
+       return a.reg == b.reg && a.byte == b.byte;
+}
+
+/*
+ * dir: 0: de-skew = delta_*
+ *     1: de-skew = reg val + delta_*
+ * delta_dif: value for differential signal: clk/
+ * delta_sig: value for single signal: ca/cmd
+ */
+static void modify_ca_deskew(struct dram_info *dram, u32 dir, int delta_dif, int delta_sig, u32
cs,
+                            u32 dramtype)
+{
+       struct rk3568_ddrphy *phy = dram->phy;
+       struct cb_invdelaysel_addr addr;
+       u32 i;
+
+       if ((dramtype == LPDDR4 || dramtype == LPDDR4X) && !dram->cmd_perbit_skew_bp) {
+               if (cs == 0) {
+                       for (i = 0; i < ARRAY_SIZE(cb_ck_sel); ++i)
+                               record_ca_prebit(phy, dir, delta_dif, cb_ck_sel[i]);
+               }
+
+               for (i = 0; i < ARRAY_SIZE(cb_ca_sel); ++i)
+                       record_ca_prebit(phy, dir, delta_sig, cb_ca_sel[i]);
+
+               for (i = 0; i < ARRAY_SIZE(cb_cs_sel[cs != 0]); ++i)
+                       record_ca_prebit(phy, dir, delta_sig, cb_cs_sel[cs != 0][i]);
+
+               update_ca_prebit(phy, cs);
+       } else {
+               for (i = 0; i < 8 * sizeof(u32); ++i) {
+                       addr.reg = i / sizeof(u32);
+                       addr.byte = i % sizeof(u32);
+
+                       if (cb_invdelaysel_addr_cmp(addr, cb_ck_sel[0]) ||
+                           cb_invdelaysel_addr_cmp(addr, cb_ck_sel[1])) {
+                               record_ca_prebit(phy, dir, delta_dif, addr);
+                       } else if ((dramtype == LPDDR4 || dramtype == LPDDR4X) &&
+                                  (cb_invdelaysel_addr_cmp(addr, cb_ck_sel[2]) ||
+                                   cb_invdelaysel_addr_cmp(addr, cb_ck_sel[3]))) {
+                               record_ca_prebit(phy, dir, delta_dif, addr);
+                       } else {
+                               record_ca_prebit(phy, dir, delta_sig, addr);
+                       }
+               }
+       }
+}
+
+static void record_ca_result(struct dram_info *dram, u32 dramtype, u32 rank)
+{
+       struct skew_info *skew_info = &dram->skew_info;
+       struct rk3568_ddrphy *phy = dram->phy;
```

```
+       u32 cmd_invdelaysel_sel;
+       int i, j;
+       u8 delta;
+       u32 cs;
+
+       for (cs = 0; cs < rank; ++cs) {
+               for (i = 0; i < ARRAY_SIZE(cb_ck_sel); ++i) {
+                       cmd_invdelaysel_sel = CMD_INVDELAYSEL_SEL_A_CK + (i % 2) * 8;
+                       clrsetbits_le32(&phy->reg6c, CMD_INVDELAYSEL_SEL_MASK,
+                                       cmd_invdelaysel_sel << CMD_INVDELAYSEL_SEL_SHIFT);
+                       delta = (readl(&phy->undocumented1) & CMD_INVDELAYSEL_MASK) >>
+                               CMD_INVDELAYSEL_SHIFT;
+                       record_ca_prebit(phy, DESKEW_MDF_ABS_VAL, delta, cb_ck_sel[i]);
+               }
+
+               cmd_invdelaysel_sel = rank == 0
+                                     ? CMD_INVDELAYSEL_SEL_A_CA0_CS0
+                                     : CMD_INVDELAYSEL_SEL_A_CA0_CS1;
+               for (i = 0; i < ARRAY_SIZE(cb_ca_sel); ++i) {
+                       clrsetbits_le32(&phy->reg6c, CMD_INVDELAYSEL_SEL_MASK,
+                                       (cmd_invdelaysel_sel + i) << CMD_INVDELAYSEL_SEL_SHIFT);
+                       delta = (readl(&phy->undocumented1) & CMD_INVDELAYSEL_MASK) >>
+                               CMD_INVDELAYSEL_SHIFT;
+                       record_ca_prebit(phy, DESKEW_MDF_ABS_VAL, delta, cb_ca_sel[i]);
+               }
+
+               for (i = 0; i < ARRAY_SIZE(cb_cs_sel); ++i) {
+                       for (j = 0; j < ARRAY_SIZE(cb_cs_sel[i]); ++j) {
+                               cmd_invdelaysel_sel = CMD_INVDELAYSEL_SEL_A_CKE0 + i + (j % 2) *
8;
+                               clrsetbits_le32(&phy->reg6c, CMD_INVDELAYSEL_SEL_MASK,
+                                               cmd_invdelaysel_sel <<
CMD_INVDELAYSEL_SEL_SHIFT);
+                               delta = (readl(&phy->undocumented1) & CMD_INVDELAYSEL_MASK) >>
+                                       CMD_INVDELAYSEL_SHIFT;
+                               record_ca_prebit(phy, DESKEW_MDF_ABS_VAL, delta, cb_cs_sel[i]
[j]);
+                       }
+               }
+
+               modify_ca_deskew(dram, DESKEW_MDF_DIFF_VAL, skew_info->clk_delta,
+                                skew_info->clk_delta, cs, dramtype);
+       }
+}
+
+static u32 low_power_update(struct dram_info *dram, u32 en)
+{
+       struct rk3568_ddrphy *phy = dram->phy;
+       void __iomem *pctl_base = dram->pctl;
+       u32 lp_stat = 0;
+
+       if (en) {
+               clrbits_le32(&phy->reg20, LP_BYPASS);
+               setbits_le32(pctl_base + DDR_PCTL2_PWRCTL, en & 0xf);
+       } else {
+               lp_stat = readl(pctl_base + DDR_PCTL2_PWRCTL) & 0xf;
+               clrbits_le32(pctl_base + DDR_PCTL2_PWRCTL, 0xf);
+               setbits_le32(&phy->reg20, LP_BYPASS);
+       }
+
+       return lp_stat;
+}
+
+static int data_training_ca(struct dram_info *dram, u32 dramtype, u32 rank, u32 freq, u32
dst_fsp)
```

```
+{
+       u32 t_cacd, t_mrw, t_ckelck, t_dstrain, t_xcbt, t_fc;
+       struct rk3568_ddrphy *phy = dram->phy;
+       void __iomem *pctl_base = dram->pctl;
+       u32 timeout_us = 10000;
+       u32 ca_clk_div;
+       u32 vref_ca;
+       int ret;
+
+       if (freq < 400 || rank > 2)
+               return 0;
+
+       vref_ca = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT6) & 0x7f;
+       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG))
+               printf("vref_ca:%x\n", vref_ca);
+
+       /* Configure clock divider for training */
+       if (freq < 600)
+               ca_clk_div = 1;
+       else if (freq < 1200)
+               ca_clk_div = 2;
+       else
+               ca_clk_div = 4;
+
+       clrsetbits_le32(&phy->reg8, CLK_DIV_CNT_MASK, ca_clk_div << CLK_DIV_CNT_SHIFT);
+
+       clrbits_le32(&phy->regd, CAT_SKIP_FSPY);
+
+       /* Configure training timings */
+       t_cacd    = MIN((freq * 20 + 999) / 1000, 62) / 2;
+       t_mrw     = MIN(MAX((freq * 14 + 999) / 1000, 10), 30) / 2;
+       t_dstrain = ((freq * 2 + 999) / 1000) / 2;
+       t_ckelck  = MAX((freq * 7 + freq / 2 + 999) / 1000, 5) / 2; /* MAX(7.5ns, 5nCK) */
+       t_xcbt    = MAX((freq + freq / 2 + 999) / 1000, 2) / 2;     /* MAX(1.5ns, 2nCK) */
+
+       clrsetbits_le32(&phy->reg9,
+                       DDRPHY_TCACD_MASK |
+                               DDRPHY_TMRW_MASK |
+                               DDRPHY_TDSTRAIN_MASK |
+                               DDRPHY_TCKELCK_MASK |
+                               DDRPHY_TADR_MASK |
+                               DDRPHY_TXCBT_MASK,
+                       (t_cacd << DDRPHY_TCACD_SHIFT) |
+                               (t_mrw << DDRPHY_TMRW_SHIFT) |
+                               (t_dstrain << DDRPHY_TDSTRAIN_SHIFT) |
+                               (t_ckelck << DDRPHY_TCKELCK_SHIFT) |
+                               (t_cacd << DDRPHY_TADR_SHIFT) |
+                               (t_xcbt << DDRPHY_TXCBT_SHIFT));
+
+       t_fc = MIN((freq * 250 + 999) / 1000, 510) / 2;
+
+       clrsetbits_le32(&phy->reg10_0,
+                       DDRPHY_TFC_MASK | DDRPHY_TCAENT_MASK | DDRPHY_TVREFCA_LONG_MASK,
+                       (t_fc << DDRPHY_TFC_SHIFT) |
+                               (t_fc << DDRPHY_TCAENT_SHIFT) |
+                               (t_fc << DDRPHY_TVREFCA_LONG_SHIFT));
+
+       /* Choose training ranks */
+       clrsetbits_le32(&phy->reg8, CAT_RANK_NUM_MASK,
+                       (rank == 2 ? CAT_RANK_NUM_TWO_RANKS : CAT_RANK_NUM_RANK0)
+                               << CAT_RANK_NUM_SHIFT);
+
+       /* Setup vref_ca */
+       clrsetbits_le32(&phy->regc, 0x7f | (0x7f << 8) | CAT_VREF_SCAN_DIS,
+                       vref_ca | (vref_ca << 8) | CAT_VREF_SCAN_DIS);
```

```
+
+       /* Start CA training */
+       setbits_le32(&phy->reg8, CAT_ENABLE);
+       setbits_le32(&phy->reg8, CAT_START);
+
+       /* Wait till training ends */
+       while ((readl(&phy->reg85) & (CHA_CAT_DONE | CHB_CAT_DONE)) !=
+              (CHA_CAT_DONE | CHB_CAT_DONE)) {
+               udelay(1);
+
+               if (timeout_us-- == 0) {
+                       printascii("error: CA training timeout\n");
+                       ret = -1;
+                       goto out_cleanup;
+               }
+       }
+
+       ret = 0;
+
+out_cleanup:
+       clrbits_le32(&phy->reg8, CAT_START | CAT_ENABLE);
+
+       return ret;
+}
+
+static int data_training_rg(struct dram_info *dram, u32 cs, u32 dramtype)
+{
+       struct rk3568_ddrphy *phy = dram->phy;
+       u32 dq_enable, reg0, calib_result;
+       u32 dis_auto_zq = 0;
+       u32 ret = 0;
+       u32 result;
+       u32 i;
+
+       dq_enable = (readl(&phy->reg0) & CHANNEL_EN_MASK) >> CHANNEL_EN_SHIFT;
+
+       if (dramtype != LPDDR4 && dramtype != LPDDR4X) {
+               reg0 = readl(&phy->pad_group[0].reg0);
+
+               for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i) {
+                       clrsetbits_le32(&phy->pad_group[i].reg0, ABC_LH_DQSWEAKP_MASK,
+                                       ABC_LH_DQSWEAKP_HIGH_Z << ABC_LH_DQSWEAKP_SHIFT);
+               }
+       }
+
+       dis_auto_zq = pctl_dis_zqcs_aref(dram->pctl);
+
+       if (dramtype == DDR4) {
+               /* Use the read preamble training mode for DDR4 */
+               setbits_le32(&phy->reg10_1, CALIB_MODE_SEL);
+       } else {
+               /* Use normal read mode for data training for other DDR types */
+               clrbits_le32(&phy->reg10_1, CALIB_MODE_SEL);
+       }
+
+       /* Choose training cs */
+       clrsetbits_le32(&phy->reg1, CALCS_SEL_MASK,
+                       (~(1 << cs) << CALCS_SEL_SHIFT) & CALCS_SEL_MASK);
+
+       /* Enable gate training */
+       setbits_le32(&phy->reg1, START_CALIB);
+       /* Wait till training ends */
+       udelay(100);
+       /* Read training result */
+       result = readl(&phy->reg83);
```

```
+
+       /* Disable gate training, give rank control back to DFI */
+       clrbits_le32(&phy->reg1, CALCS_SEL_MASK | START_CALIB);
+
+       pctl_rest_zqcs_aref(dram->pctl, dis_auto_zq);
+
+       /* Check whether calibration was completed successfully for all configured DQ pad groups
*/
+       if (result & CALIB_ERROR || (result & CALIB_DONE_BYTE_MASK) != dq_enable) {
+               if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG))
+                       printascii("read gate training failed\n");
+               ret = -1;
+               goto out_cleanup;
+       }
+
+       if (cs == 0) {
+               for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i) {
+                       if (!(dq_enable & (1 << i))) {
+                               /* Skip this pad group if it is not enabled */
+                               continue;
+                       }
+
+                       calib_result =
+                               (readl(&phy->pad_group[i].reg1e) & ABC_LH_CALIB_RESULT_CS0_MASK)
>>
+                               ABC_LH_CALIB_RESULT_CS0_SHIFT;
+
+                       if ((calib_result >> 8) >= 5) {
+                               printascii("error: rx dqs calibration delay is too high\n");
+                               ret = -1;
+                               goto out_cleanup;
+                       }
+               }
+       }
+
+out_cleanup:
+       if (dramtype != LPDDR4 && dramtype != LPDDR4X) {
+               for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i)
+                       writel(reg0, &phy->pad_group[i].reg0);
+       }
+
+       return ret;
+}
+
+static int data_training_wl(struct dram_info *dram, u32 cs, u32 dramtype, u32 rank)
+{
+       struct rk3568_ddrphy *phy = dram->phy;
+       void __iomem *pctl_base = dram->pctl;
+       u32 timeout_us = 1000;
+       u32 dis_auto_zq = 0;
+       u32 dq_enable;
+       u32 cur_fsp;
+       u32 tmp;
+       int ret;
+
+       dq_enable = (readl(&phy->reg0) & CHANNEL_EN_MASK) >> CHANNEL_EN_SHIFT;
+
+       dis_auto_zq = pctl_dis_zqcs_aref(dram->pctl);
+
+       clrbits_le32(&phy->reg27, DQ_WR_TRAIN_AUTO);
+
+       cur_fsp = readl(pctl_base + DDR_PCTL2_MSTR2) & 0x3;
+       tmp = readl(pctl_base + UMCTL2_REGS_FREQ(cur_fsp) + DDR_PCTL2_INIT3);
+
+       if (dramtype == DDR3 || dramtype == DDR4)
```

```
+               tmp = (tmp & 0x3fff) | 0x4000;
+       else
+               tmp &= 0xff;
+
+       clrsetbits_le32(&phy->reg1, WL_LOADMODE_MASK, tmp << WL_LOADMODE_SHIFT);
+
+       /* Disable another cs's output */
+       if ((dramtype == DDR3 || dramtype == DDR4) && rank == 2)
+               pctl_write_mr(dram->pctl, (cs + 1) & 1, 1, tmp | (1 << 12), dramtype);
+
+       /* Choose training cs */
+       clrsetbits_le32(&phy->reg1, WLCS_SEL_MASK, (~(1 << cs) << WLCS_SEL_SHIFT) &
WLCS_SEL_MASK);
+
+       /* Enable write leveling training */
+       setbits_le32(&phy->reg1, WL_ENABLE);
+
+       /* Wait till training ends */
+       while (dq_enable != (readl(&phy->reg83) & WL_DONE_BYTE_MASK) >> WL_DONE_BYTE_SHIFT) {
+               udelay(1);
+
+               if (timeout_us-- == 0) {
+                       printascii("error: write leveling timeout\n");
+                       ret = -1;
+                       goto out_cleanup;
+               }
+       }
+
+       ret = 0;
+
+out_cleanup:
+       /* Disable write leveling training, give rank control back to DFI */
+       clrbits_le32(&phy->reg1, WL_ENABLE | WLCS_SEL_MASK);
+
+       /* Enable another cs's output */
+       if ((dramtype == DDR3 || dramtype == DDR4) && rank == 2)
+               pctl_write_mr(dram->pctl, (cs + 1) & 1, 1, tmp & ~(1 << 12), dramtype);
+
+       pctl_rest_zqcs_aref(dram->pctl, dis_auto_zq);
+
+       return ret;
+}
+
+char pattern[32] = { 0xaa, 0x55, 0xaa, 0x55, 0x55, 0xaa, 0x55, 0xaa, 0x55, 0xaa, 0x55,
+                    0xaa, 0xaa, 0x55, 0xaa, 0x55, 0x55, 0x55, 0xaa, 0xaa, 0xaa, 0xaa,
+                    0x55, 0x55, 0xaa, 0xaa, 0x55, 0x55, 0x55, 0x55, 0xaa, 0xaa };
+
+static void dram_setup_data_training(struct dram_info *dram, u32 dramtype)
+{
+       void __iomem *pctl_base = dram->pctl;
+       void __iomem *test_addr;
+       u32 test_pattern;
+       u32 cs_add = 0;
+       u64 cs_cap;
+       u32 cs_pst;
+       int i, j;
+       u32 bw;
+
+       bw = 2 >> ((readl(pctl_base + DDR_PCTL2_MSTR) >> 12) & 0x3);
+       cs_pst = (readl(pctl_base + DDR_PCTL2_ADDRMAP0) & 0x1f) + 6 + 2;
+       if (cs_pst > 32)
+               cs_add = 1;
+
+       for (i = 0; i < cs_add + 1; ++i) {
+               cs_cap = (1 << i);
```

```
+                test_addr = (void *)CFG_SYS_SDRAM_BASE + cs_cap;
+
+                j = 0;
+                while (j < ARRAY_SIZE(pattern)) {
+                        if (bw == 2) {
+                                test_pattern = pattern[j] | (pattern[j] << 8) | (pattern[j] <<
16) |
+                                               (pattern[j] << 24);
+                                ++j;
+                        } else if (bw == 1) {
+                                test_pattern = pattern[j] | (pattern[j] << 8) |
+                                               (pattern[j + 1] << 16) | (pattern[j + 1] << 24);
+                                j += 2;
+                        } else {
+                                test_pattern = pattern[j] | (pattern[j + 1] << 8) |
+                                               (pattern[j + 2] << 16) | (pattern[j + 3] << 24);
+                                j += 4;
+                        }
+
+                        writel(test_pattern, test_addr);
+
+                        if (dramtype == DDR4) {
+                                if (j == 24)
+                                        test_addr = (void *)((CFG_SYS_SDRAM_BASE + cs_cap) |
+                                                             (0x1060 >> (2 - bw)));
+                                else if (j == 16)
+                                        test_addr = (void *)((CFG_SYS_SDRAM_BASE + cs_cap) |
+                                                             (0x40 >> (2 - bw)));
+                                else if (j == 8)
+                                        test_addr = (void *)((CFG_SYS_SDRAM_BASE + cs_cap) |
+                                                             (0x1020 >> (2 - bw)));
+                                else
+                                        test_addr += 4;
+                        } else {
+                                test_addr += 4;
+                        }
+                }
+        }
+}
+
+static void phy_setup_data_training(struct dram_info *dram, u32 dramtype)
+{
+        struct rk3568_ddrphy *phy = dram->phy;
+        u8 check_values[4];
+        u8 test_byte;
+        int i, j;
+
+        for (i = 0; i < ARRAY_SIZE(phy->reg_n_dq_train_check_data); ++i) {
+                memset(&check_values, 0, sizeof(check_values));
+
+                for (j = 0; j < ARRAY_SIZE(pattern); ++j) {
+                        test_byte = pattern[j];
+                        check_values[j / 8] |= ((test_byte >> i) & 1) << (j % 4);
+                }
+
+                clrsetbits_le32(&phy->reg_n_dq_train_check_data[i][0],
+                                DQ_TRAIN_CHECK_DATA_VALUE_MASK(0) |
+                                        DQ_TRAIN_CHECK_DATA_VALUE_MASK(1) |
+                                        DQ_TRAIN_CHECK_DATA_VALUE_MASK(2) |
+                                        DQ_TRAIN_CHECK_DATA_VALUE_MASK(3),
+                                (check_values[3] << DQ_TRAIN_CHECK_DATA_VALUE_SHIFT(0)) |
+                                        (check_values[2] << DQ_TRAIN_CHECK_DATA_VALUE_SHIFT(1)) |
+                                        (check_values[1] << DQ_TRAIN_CHECK_DATA_VALUE_SHIFT(2)) |
+                                        (check_values[0] << DQ_TRAIN_CHECK_DATA_VALUE_SHIFT(3)));
+        }
```

```
+}
+
+static void setup_data_training(struct dram_info *dram, u32 dramtype, u32 phy_side)
+{
+       if (phy_side)
+               phy_setup_data_training(dram, dramtype);
+       else
+               dram_setup_data_training(dram, dramtype);
+}
+
+static int data_training_rd(struct dram_info *dram, u32 cs, u32 dramtype, u32 mhz)
+{
+       struct sdram_head_info_index_v2 *index;
+       struct rk3568_ddrphy *phy = dram->phy;
+       void __iomem *pctl_base = dram->pctl;
+       u32 rdtrain_wait_vref_valid_cnt;
+       struct dq_map_info *map_info;
+       u32 dq_enable, dq_error;
+       u32 timeout_us = 1000;
+       u32 trefi_1x, trfc_1x;
+       u32 train_check_flag;
+       u32 dis_auto_zq = 0;
+       u8 pad_dq_map[2];
+       u32 dqs_default;
+       u32 cur_fsp;
+       int ret;
+       u32 i;
+
+       dqs_default = 0x1f;
+       dis_auto_zq = pctl_dis_zqcs_aref(dram->pctl);
+
+       if (dramtype == DDR4 && !dram->ecc_en) {
+               clrsetbits_le32(&phy->reg28, WR_TRAIN_ROW_ADDR_MASK, 0 <<
WR_TRAIN_ROW_ADDR_SHIFT);
+               setup_data_training(dram, dramtype, 1);
+       }
+
+       /* Config refresh timing */
+       cur_fsp = readl(pctl_base + DDR_PCTL2_MSTR2) & 0x3;
+       trefi_1x =
+               ((readl(pctl_base + UMCTL2_REGS_FREQ(cur_fsp) + DDR_PCTL2_RFSHTMG) >> 16) &
0xfff) *
+               32;
+       trfc_1x = readl(pctl_base + UMCTL2_REGS_FREQ(cur_fsp) + DDR_PCTL2_RFSHTMG) & 0x3ff;
+
+       /* reg_phy_trefi, reg_phy_trfc. reg_max_refi_cnt is 8 after reset by default. */
+       clrsetbits_le32(&phy->reg29, PHY_TRFC_MASK | PHY_TREFI_MASK,
+                       (trfc_1x << PHY_TRFC_SHIFT) | (trefi_1x << PHY_TREFI_SHIFT));
+
+       /* Choose training cs */
+       clrsetbits_le32(&phy->reg24, RDTRAIN_CS_SEL_MASK,
+                       (~((1 << cs) << RDTRAIN_CS_SEL_SHIFT)) & RDTRAIN_CS_SEL_MASK);
+
+       if (dramtype == DDR4 && !dram->ecc_en) {
+               /* 800ns / dfi_1xclk */
+               rdtrain_wait_vref_valid_cnt = (((mhz * 800) / 2) + 999) / 1000;
+               clrsetbits_le32(&phy->reg6c, RDTRAIN_WAIT_VERF_VALID_CNT_MASK,
+                               rdtrain_wait_vref_valid_cnt <<
RDTRAIN_WAIT_VERF_VALID_CNT_SHIFT);
+       }
+
+       /* Set dq map for ddr4 */
+       if (dramtype == DDR4 && dram->ecc_en) {
+               index = (struct sdram_head_info_index_v2 *)common_info;
+               map_info = (struct dq_map_info *)((void *)common_info +
```

```
+                                                index->dq_map_index.offset * 4);
+
+               setbits_le32(&phy->reg24, RD_TRAIN_CHECK_VALUE_EN);
+               for (i = 0; i < 4; i++) {
+                       pad_dq_map[0] =
+                               (map_info->ddr4_dq_map[cs * 2 + i / 2] >> (8 + (16 * (i % 2)))) &
+                               0xff;
+                       pad_dq_map[1] = (map_info->ddr4_dq_map[cs * 2 + i / 2] >> (16 * (i % 2)))
&
+                                       0xff;
+                       clrsetbits_le32(&phy->pad_group[i].reg19,
+                                       ABC_LH_RDTRAIN_CHECK_WRAP1_MASK |
+                                               ABC_LH_RDTRAIN_CHECK_WRAP0_MASK,
+                                       (pad_dq_map[0] << ABC_LH_RDTRAIN_CHECK_WRAP1_SHIFT) |
+                                               (pad_dq_map[1] <<
+                                               ABC_LH_RDTRAIN_CHECK_WRAP0_SHIFT));
+               }
+
+               clrsetbits_le32(&phy->pad_group[4].reg19,
+                               ABC_LH_RDTRAIN_CHECK_WRAP1_MASK |
ABC_LH_RDTRAIN_CHECK_WRAP0_MASK,
+                               0x6372);
+       }
+
+       /* Setup rd_train_dqs_default for each pad group */
+       for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i) {
+               clrsetbits_le32(&phy->pad_group[i].reg18, ABC_LH_RD_TRAIN_DQS_DEFAULT_MASK,
+                               dqs_default << ABC_LH_RD_TRAIN_DQS_DEFAULT_SHIFT);
+       }
+
+       if (dramtype != DDR4 || dram->ecc_en) {
+               /* Enable auto train of the read train */
+               setbits_le32(&phy->reg24, DQ_RD_TRAIN_EN);
+               train_check_flag = TRAIN_TRUE_DONE;
+       } else {
+               /* Enable predefined train of the read train */
+               setbits_le32(&phy->reg24, RD_TRAIN_PREDEF_EN);
+               train_check_flag = TRAIN_ALL_STEP_DONE;
+       }
+
+       /* Wait till training ends */
+       while (!(readl(&phy->reg7c) & train_check_flag)) {
+               udelay(1);
+
+               if (timeout_us-- == 0) {
+                       printascii("error: read training timeout\n");
+                       ret = -1;
+                       goto out_cleanup;
+               }
+       }
+
+       /* Check the read train state */
+       if (dramtype == DDR4 && !dram->ecc_en) {
+               if (readl(&phy->reg7c) &
+                   (TRAIN_STEP3_ERROR | TRAIN_STEP2_ERROR | TRAIN_STEP1_ERROR)) {
+                       printascii("error: read training failed\n");
+                       ret = -1;
+                       goto out_cleanup;
+               }
+       } else {
+               dq_enable = (readl(&phy->reg0) & CHANNEL_EN_MASK) >> CHANNEL_EN_SHIFT;
+               dq_error = (readl(&phy->regad) & TRAIN_ERROR_FOR_RD_BYTE_MASK) >>
+                          TRAIN_ERROR_FOR_RD_BYTE_SHIFT;
+
+               if (dq_enable & dq_error) {
```

```
+                       printascii("error: read training failed\n");
+                       ret = -1;
+                       goto out_cleanup;
+               }
+       }
+
+       ret = 0;
+
+out_cleanup:
+       /* Exit the Read Training */
+       if (dramtype != DDR4 || dram->ecc_en)
+               /* Disable auto train of the read train */
+               clrbits_le32(&phy->reg24, DQ_RD_TRAIN_EN);
+       else
+               /* Disable predefined train of the read train */
+               clrbits_le32(&phy->reg24, RD_TRAIN_PREDEF_EN);
+
+       pctl_rest_zqcs_aref(dram->pctl, dis_auto_zq);
+
+       return ret;
+}
+
+static int data_training_wr(struct dram_info *dram, u32 cs, u32 dramtype, u32 mhz, u32 dst_fsp)
+{
+       struct rk3568_ddrphy *phy = dram->phy;
+       void __iomem *pctl_base = dram->pctl;
+       u32 mr_tmp, cl, cwl, phy_fsp;
+       u32 trefi_1x, trfc_1x;
+       u32 timeout_us = 1000;
+       u32 dis_auto_zq = 0;
+       u32 cur_fsp;
+       int ret;
+
+       /* Enable DM write training */
+       setbits_le32(&phy->reg27, DM_WR_TRAIN_EN);
+
+       if (dramtype == LPDDR3 && mhz <= 400) {
+               phy_fsp = (readl(&phy->reg10_1) & FREQ_CHOOSE_T_MASK) >> FREQ_CHOOSE_T_SHIFT;
+               cl = (readl(&phy->reg3) & CL_FRE_OPN_MASK(phy_fsp)) >> CL_FRE_OPN_SHIFT(phy_fsp);
+               cwl = (readl(&phy->reg4) & CWL_FRE_OPN_MASK(phy_fsp)) >>
CWL_FRE_OPN_SHIFT(phy_fsp);
+
+               clrsetbits_le32(&phy->reg3, CL_FRE_OPN_MASK(phy_fsp),
+                               0x8 << CL_FRE_OPN_SHIFT(phy_fsp));
+               clrsetbits_le32(&phy->reg4, CWL_FRE_OPN_MASK(phy_fsp),
+                               0x4 << CWL_FRE_OPN_SHIFT(phy_fsp));
+               pctl_write_mr(dram->pctl, 3, 2, 0x6, dramtype);
+       }
+
+       dis_auto_zq = pctl_dis_zqcs_aref(dram->pctl);
+
+       /*
+        * reg_train_row_addr. reg_train_col_addr and reg_train_ba_addr are set to zero after
reset.
+        */
+       clrsetbits_le32(&phy->reg28, WR_TRAIN_ROW_ADDR_MASK, 0 << WR_TRAIN_ROW_ADDR_SHIFT);
+
+       /* wrtrain_check_data_value_random_gen */
+       setbits_le32(&phy->reg27, WRTRAIN_CHECK_DATA_VALUE_RANDOM_GEN);
+
+       /* Configure refresh timing */
+       cur_fsp = readl(pctl_base + DDR_PCTL2_MSTR2) & 0x3;
+       trefi_1x =
+               ((readl(pctl_base + UMCTL2_REGS_FREQ(cur_fsp) + DDR_PCTL2_RFSHTMG) >> 16) &
0xfff) *
```

```
+               32;
+       trfc_1x = readl(pctl_base + UMCTL2_REGS_FREQ(cur_fsp) + DDR_PCTL2_RFSHTMG) & 0x3ff;
+       /* reg_phy_trefi, reg_phy_trfc. reg_max_refi_cnt is 8 after reset by default. */
+       clrsetbits_le32(&phy->reg29, PHY_TRFC_MASK | PHY_TREFI_MASK,
+                       (trfc_1x << PHY_TRFC_SHIFT) | (trefi_1x << PHY_TREFI_SHIFT));
+
+       /* Choose training cs */
+       clrsetbits_le32(&phy->reg27, WRTRAIN_CS_SEL_MASK,
+                       (~((1 << cs) << WRTRAIN_CS_SEL_SHIFT)) & WRTRAIN_CS_SEL_MASK);
+
+       /*
+        * reg_wr_train_dqs_default_bypass. Only mentioned in app note. Documented for RV1126 phy
+        * variant in RV1126 TRM.
+        */
+       setbits_le32(&phy->reg27, WR_TRAIN_DQS_DEFAULT_BYPASS);
+
+       /* Enable auto train of the write train */
+       setbits_le32(&phy->reg27, DQ_WR_TRAIN_AUTO);
+
+       /* Start write train */
+       setbits_le32(&phy->reg27, DQ_WR_TRAIN_EN);
+
+       /* Wait till training ends */
+       while (!(readl(&phy->reg7c) & TRAIN_ALL_STEP_DONE)) {
+               udelay(1);
+               if (timeout_us-- == 0) {
+                       printascii("error: write training timeout\n");
+                       ret = -1;
+                       goto out_cleanup;
+               }
+       }
+
+       /* Check the write train state */
+       if (readl(&phy->reg7c) & (TRAIN_STEP3_ERROR | TRAIN_STEP2_ERROR | TRAIN_STEP1_ERROR)) {
+               printascii("error: write training failed\n");
+               ret = -1;
+               goto out_cleanup;
+       }
+
+       /* save LPDDR4 write vref to fsp_param for dfs */
+       if (dramtype == LPDDR4) {
+               fsp_param[dst_fsp].vref_dq[cs] =
+                       (((readl(&phy->undocumented1) & WRTRAIN_VREF_MAX_VALUE_MASK) >>
+                         WRTRAIN_VREF_MAX_VALUE_SHIFT) +
+                        ((readl(&phy->undocumented1) & WRTRAIN_VREF_MIN_VALUE_MASK) >>
+                         WRTRAIN_VREF_MIN_VALUE_SHIFT)) /
+                       2;
+               /* add range info */
+               fsp_param[dst_fsp].vref_dq[cs] |=
+                       ((readl(&phy->reg27) & WRTRAIN_LPDDR4_VREF_RANGE_MASK) >>
+                        WRTRAIN_LPDDR4_VREF_RANGE_SHIFT)
+                       << 6;
+       }
+
+       ret = 0;
+
+out_cleanup:
+       /* Exit write training */
+       clrbits_le32(&phy->reg27, DQ_WR_TRAIN_EN);
+
+       pctl_rest_zqcs_aref(dram->pctl, dis_auto_zq);
+
+       if (dramtype == LPDDR3 && mhz <= 400) {
+               clrsetbits_le32(&phy->reg3, CL_FRE_OPN_MASK(phy_fsp),
+                               cl << CL_FRE_OPN_SHIFT(phy_fsp));
```

```
+               clrsetbits_le32(&phy->reg4, CWL_FRE_OPN_MASK(phy_fsp),
+                               cwl << CWL_FRE_OPN_SHIFT(phy_fsp));
+               mr_tmp = readl(pctl_base + UMCTL2_REGS_FREQ(cur_fsp) + DDR_PCTL2_INIT3);
+               pctl_write_mr(dram->pctl, 3, 2, mr_tmp & PCTL2_MR_MASK, dramtype);
+       }
+
+       return ret;
+}
+
+static int data_training(struct dram_info *dram, u32 cs, struct rk3568_sdram_params
*sdram_params,
+                        u32 dst_fsp, u32 training_flag)
+{
+       u32 ret = 0;
+
+       if (training_flag == FULL_TRAINING)
+               training_flag = READ_GATE_TRAINING | WRITE_LEVELING | WRITE_TRAINING |
+                               READ_TRAINING;
+
+       if ((training_flag & WRITE_LEVELING) == WRITE_LEVELING) {
+               ret = data_training_wl(dram, cs, sdram_params->base.dramtype,
+                                      sdram_params->ch.cap_info.rank);
+               if (ret != 0)
+                       goto out;
+       }
+
+       if ((training_flag & READ_GATE_TRAINING) == READ_GATE_TRAINING) {
+               ret = data_training_rg(dram, cs, sdram_params->base.dramtype);
+               if (ret != 0)
+                       goto out;
+       }
+
+       if ((training_flag & READ_TRAINING) == READ_TRAINING) {
+               ret = data_training_rd(dram, cs, sdram_params->base.dramtype,
+                                      sdram_params->base.ddr_freq);
+               if (ret != 0)
+                       goto out;
+       }
+
+       if ((training_flag & WRITE_TRAINING) == WRITE_TRAINING) {
+               ret = data_training_wr(dram, cs, sdram_params->base.dramtype,
+                                      sdram_params->base.ddr_freq, dst_fsp);
+               if (ret != 0)
+                       goto out;
+       }
+
+out:
+       return ret;
+}
+
+static int get_wrlvl_val(struct dram_info *dram, struct rk3568_sdram_params *sdram_params)
+{
+       struct skew_info *skew_info = &dram->skew_info;
+       u32 dramtype = sdram_params->base.dramtype;
+       struct rk3568_ddrphy *phy = dram->phy;
+       s16 clk_skew_a, clk_skew_b;
+       int i, j, ret;
+       u32 dq_enable;
+       u32 lp_stat;
+
+       lp_stat = low_power_update(dram, 0);
+
+       if (dramtype == LPDDR4 || dramtype == LPDDR4X) {
+               clrsetbits_le32(&phy->reg6c, CMD_INVDELAYSEL_SEL_MASK,
+                               CMD_INVDELAYSEL_SEL_A_CK << CMD_INVDELAYSEL_SEL_SHIFT);
```

```
+               clk_skew_a = (readl(&phy->undocumented1) & CMD_INVDELAYSEL_MASK) >>
+                            CMD_INVDELAYSEL_SHIFT;
+
+               clrsetbits_le32(&phy->reg6c, CMD_INVDELAYSEL_SEL_MASK,
+                               CMD_INVDELAYSEL_SEL_B_CK << CMD_INVDELAYSEL_SEL_SHIFT);
+               clk_skew_b = (readl(&phy->undocumented1) & CMD_INVDELAYSEL_MASK) >>
+                            CMD_INVDELAYSEL_SHIFT;
+       } else {
+               clk_skew_a = (readl(&phy->invdelaysel_regs.reg41) & CK_INVDELAYSEL_MASK) >>
+                            CK_INVDELAYSEL_SHIFT;
+               clk_skew_b = clk_skew_a;
+       }
+
+       /* Do a write leveling training */
+       ret = data_training(dram, 0, sdram_params, 0, WRITE_LEVELING);
+       if (ret)
+               goto out_lp_update;
+
+       if (sdram_params->ch.cap_info.rank == 2) {
+               ret = data_training(dram, 1, sdram_params, 0, WRITE_LEVELING);
+               if (ret)
+                       goto out_lp_update;
+       }
+
+       /* Collect training results */
+       for (i = 0; i < 4; ++i) {
+               wrlvl_result[0][i] = (readl(&phy->pad_group[i].tdqs_invdelaysel12) &
+                                     ABC_LH_TDQS_INVDELAYSEL0_MASK) >>
+                                    ABC_LH_TDQS_INVDELAYSEL0_SHIFT;
+               wrlvl_result[1][i] = (readl(&phy->pad_group[i].tdqs_invdelaysel12) &
+                                     ABC_LH_TDQS_INVDELAYSEL1_MASK) >>
+                                    ABC_LH_TDQS_INVDELAYSEL1_SHIFT;
+
+               if (i <= 1) {
+                       wrlvl_result[0][i] -= clk_skew_a;
+                       wrlvl_result[1][i] -= clk_skew_a;
+               } else {
+                       wrlvl_result[0][i] -= clk_skew_b;
+                       wrlvl_result[1][i] -= clk_skew_b;
+               }
+       }
+
+       skew_info->t_dqss_max = SHRT_MIN;
+       skew_info->t_dqss_min = SHRT_MAX;
+
+       dq_enable = (readl(&phy->reg0) & CHANNEL_EN_MASK) >> CHANNEL_EN_SHIFT;
+
+       /* Convert training results to skew in ps */
+       for (i = 0; i < sdram_params->ch.cap_info.rank; ++i) {
+               if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG))
+                       printf("tdqss: cs%d ", i);
+
+               for (j = 0; j < 5; ++j) {
+                       if (!((dq_enable >> j) & 1))
+                               continue;
+
+                       wrlvl_result[i][j] = (wrlvl_result[i][j] * 500000) /
+                                            ((int)sdram_params->base.ddr_freq) / 64;
+
+                       if (wrlvl_result[i][j] < skew_info->t_dqss_min)
+                               skew_info->t_dqss_min = wrlvl_result[i][j];
+
+                       if (wrlvl_result[i][j] > skew_info->t_dqss_max)
+                               skew_info->t_dqss_max = wrlvl_result[i][j];
+
```

```
+                       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG)) {
+                               if (j > 0)
+                                       printascii(", ");
+
+                               printf("dqs%d: %dps", j, wrlvl_result[i][j]);
+                       }
+               }
+
+               if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG))
+                       printascii("\n");
+       }
+
+out_lp_update:
+       low_power_update(dram, lp_stat);
+
+       return ret;
+}
+
+#ifdef CONFIG_RAM_ROCKCHIP_DEBUG
+
+struct train_result_addr {
+       /* Byte shift from register start, LSB */
+       u8 byte : 2;
+       u8 reg  : 6;
+};
+
+struct ca_train_result_addr {
+       struct train_result_addr ca_min_start;
+       struct train_result_addr ca_max_start;
+       struct train_result_addr cs_min;
+       struct train_result_addr cs_max;
+};
+
+static const struct ca_train_result_addr cs_ca_train_result_addr[] = {
+       {
+               .ca_min_start = { .reg = 0, .byte = 3},
+               .ca_max_start = { .reg = 4, .byte = 3},
+               .cs_min       = { .reg = 3, .byte = 3},
+               .cs_max       = { .reg = 3, .byte = 1},
+       },
+       {
+               .ca_min_start = { .reg = 1, .byte = 1},
+               .ca_max_start = { .reg = 5, .byte = 1},
+               .cs_min       = { .reg = 3, .byte = 2},
+               .cs_max       = { .reg = 3, .byte = 0},
+       },
+};
+
+struct wr_train_result_addr {
+       struct train_result_addr dq_min_start;
+       struct train_result_addr dq_max_start;
+       struct train_result_addr dm_min;
+       struct train_result_addr dm_max;
+};
+
+static const struct wr_train_result_addr wr_train_result_addr = {
+       .dq_min_start = { .reg = 1, .byte = 3},
+       .dq_max_start = { .reg = 4, .byte = 3},
+       .dm_min       = { .reg = 0, .byte = 1},
+       .dm_max       = { .reg = 3, .byte = 1},
+};
+
+struct rd_train_result_addr {
+       struct train_result_addr dq_min_start;
+       struct train_result_addr dq_max_start;
```

```
+};
+
+static const struct rd_train_result_addr rd_train_result_addr = {
+       .dq_min_start = { .reg = 0, .byte = 3},
+       .dq_max_start = { .reg = 3, .byte = 3},
+};
+
+enum training_print_mode {
+       TRAINING_PRINT_MODE_MIN   = 0,
+       TRAINING_PRINT_MODE_AVG   = 1,
+       TRAINING_PRINT_MODE_MAX   = 2,
+       TRAINING_PRINT_MODE_RANGE = 3,
+};
+
+static char *training_print_mode_names[] = {
+       "min  ",
+       "avg  ",
+       "max  ",
+       "range",
+};
+
+static void phy_print_training_value(u8 min, u8 max, enum training_print_mode mode)
+{
+       u8 value;
+
+       switch (mode) {
+       case TRAINING_PRINT_MODE_MIN:
+               value = min;
+               break;
+       case TRAINING_PRINT_MODE_AVG:
+               value = (min + max) / 2;
+               break;
+       case TRAINING_PRINT_MODE_MAX:
+               value = max;
+               break;
+       case TRAINING_PRINT_MODE_RANGE:
+               value = max - min;
+               break;
+       }
+
+       if (value < 16)
+               printascii(" ");
+
+       printf(" 0x%x", value);
+}
+
+static inline void train_result_addr_advance(struct train_result_addr *addr)
+{
+       if (addr->byte == 0) {
+               addr->byte = 3;
+               ++addr->reg;
+       } else {
+               --addr->byte;
+       }
+}
+
+static inline u8 phy_cmd_training_result_get_value(struct rk3568_ddrphy *phy, u32 cs,
+                                                   struct train_result_addr addr)
+{
+       return (readl(&phy->cmd_training_result[cs][addr.reg]) >> addr.byte * 8) & 0xff;
+}
+
+static void phy_print_training_ca_result(struct rk3568_ddrphy *phy,
+                                         struct rk3568_sdram_params *sdram_params)
+{
```

```
+       enum training_print_mode print_mode;
+       struct train_result_addr min_addr;
+       struct train_result_addr max_addr;
+       u32 cs, channel, ca;
+       u8 min, max;
+
+       if (sdram_params->base.dramtype != LPDDR4 && sdram_params->base.dramtype != LPDDR4X)
+               return;
+
+       printascii("CA Training result:\n");
+
+       for (cs = 0; cs < sdram_params->ch.cap_info.rank; ++cs) {
+               for (print_mode = 0; print_mode < ARRAY_SIZE(training_print_mode_names);
+                    ++print_mode) {
+                       printf("cs:%d %s:", cs, training_print_mode_names[print_mode]);
+
+                       for (channel = 0; channel < 2; ++channel) {
+                               min_addr = cs_ca_train_result_addr[cs].ca_min_start;
+                               max_addr = cs_ca_train_result_addr[cs].ca_max_start;
+
+                               for (ca = 0; ca < 6; ++ca) {
+                                       min = phy_cmd_training_result_get_value(phy, cs,
min_addr);
+                                       max = phy_cmd_training_result_get_value(phy, cs,
max_addr);
+
+                                       phy_print_training_value(min, max, print_mode);
+
+                                       train_result_addr_advance(&min_addr);
+                                       train_result_addr_advance(&max_addr);
+                               }
+
+                               min_addr = cs_ca_train_result_addr[cs].cs_min;
+                               max_addr = cs_ca_train_result_addr[cs].cs_max;
+
+                               min = phy_cmd_training_result_get_value(phy, cs, min_addr);
+                               max = phy_cmd_training_result_get_value(phy, cs, max_addr);
+
+                               phy_print_training_value(min, max, print_mode);
+
+                               if (channel != 1)
+                                       printascii(",");
+                       }
+
+                       printascii("\n");
+               }
+       }
+}
+
+static inline u8 phy_train_for_wr_get_value(struct rk3568_ddrphy *phy, u32 pad_group,
+                                           struct train_result_addr addr)
+{
+       return (readl(&phy->pad_group[pad_group].train_for_wr[addr.reg]) >> addr.byte * 8) &
0xff;
+}
+
+static void phy_print_training_wr_result(struct rk3568_ddrphy *phy,
+                                         struct rk3568_sdram_params *sdram_params)
+{
+       enum training_print_mode print_mode;
+       struct train_result_addr min_addr;
+       struct train_result_addr max_addr;
+       u32 dq_enable;
+       u8 min, max;
+       int i, dq;
```

```
+
+       printascii("the write training result:\n");
+
+       dq_enable = (readl(&phy->reg0) & CHANNEL_EN_MASK) >> CHANNEL_EN_SHIFT;
+
+       for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i) {
+               if (!(dq_enable & (1 << i)))
+                       continue;
+
+               clrsetbits_le32(&phy->pad_group[i].regc, ABC_LH_CSN_LOOP_INVDELAYSEL_MASK,
+                               0xa << ABC_LH_CSN_LOOP_INVDELAYSEL_SHIFT);
+
+               printf("DQS%d: 0x%x, ", i,
+                      (readl(&phy->pad_group[0].reg1f) & ABC_LH_CSN_VALUE_DQX_INVDELAYSEL_MASK)
>>
+                              ABC_LH_CSN_VALUE_DQX_INVDELAYSEL_SHIFT);
+       }
+
+       printascii("\n");
+
+       for (print_mode = 0; print_mode < ARRAY_SIZE(training_print_mode_names); ++print_mode) {
+               printf("%s:", training_print_mode_names[print_mode]);
+
+               for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i) {
+                       if (!(dq_enable & (1 << i)))
+                               continue;
+
+                       if (i != 0 && i % 2 == 0)
+                               printascii("\n      ");
+
+                       min_addr = wr_train_result_addr.dq_min_start;
+                       max_addr = wr_train_result_addr.dq_max_start;
+
+                       for (dq = 0; dq < 8; ++dq) {
+                               min = phy_train_for_wr_get_value(phy, i, min_addr);
+                               max = phy_train_for_wr_get_value(phy, i, max_addr);
+
+                               phy_print_training_value(min, max, print_mode);
+
+                               train_result_addr_advance(&min_addr);
+                               train_result_addr_advance(&max_addr);
+                       }
+
+                       min_addr = wr_train_result_addr.dm_min;
+                       max_addr = wr_train_result_addr.dm_max;
+
+                       phy_print_training_value(min, max, print_mode);
+
+                       printascii(",");
+               }
+
+               printascii("\n");
+       }
+}
+
+static inline u8 phy_train_for_rd_get_value(struct rk3568_ddrphy *phy, u32 pad_group,
+                                           struct train_result_addr addr)
+{
+       return (readl(&phy->pad_group[pad_group].train_for_rd[addr.reg]) >> addr.byte * 8) &
0x7f;
+}
+
+static void phy_print_training_rd_result(struct rk3568_ddrphy *phy,
+                                         struct rk3568_sdram_params *sdram_params)
+{
```

```
+       enum training_print_mode print_mode;
+       struct train_result_addr min_addr;
+       struct train_result_addr max_addr;
+       u32 dq_enable;
+       u8 min, max;
+       int i, dq;
+
+       printascii("the read training result:\n");
+
+       dq_enable = (readl(&phy->reg0) & CHANNEL_EN_MASK) >> CHANNEL_EN_SHIFT;
+
+       for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i) {
+               if (!(dq_enable & (1 << i)))
+                       continue;
+
+               printf("DQS%d: 0x%x, ", i,
+                      (readl(&phy->pad_group[i].reg27) &
+                       ABC_LH_TRAIN_RESULT_FOR_RD_BASE_DQS_MASK) >>
+                              ABC_LH_TRAIN_RESULT_FOR_RD_BASE_DQS_SHIFT);
+       }
+
+       printascii("\n");
+
+       for (print_mode = 0; print_mode < ARRAY_SIZE(training_print_mode_names); ++print_mode) {
+               printf("%s:", training_print_mode_names[print_mode]);
+
+               for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i) {
+                       if (!(dq_enable & (1 << i)))
+                               continue;
+
+                       if (i != 0 && i % 2 == 0)
+                               printascii("\n      ");
+
+                       min_addr = rd_train_result_addr.dq_min_start;
+                       max_addr = rd_train_result_addr.dq_max_start;
+
+                       for (dq = 0; dq < 8; ++dq) {
+                               min = phy_train_for_rd_get_value(phy, i, min_addr);
+                               max = phy_train_for_rd_get_value(phy, i, max_addr);
+
+                               phy_print_training_value(min, max, print_mode);
+
+                               train_result_addr_advance(&min_addr);
+                               train_result_addr_advance(&max_addr);
+                       }
+
+                       printascii(",");
+               }
+
+               printascii("\n");
+       }
+}
+
+static void phy_print_training_result(struct rk3568_ddrphy *phy,
+                                     struct rk3568_sdram_params *sdram_params, u32
training_flag)
+{
+       if ((training_flag & CA_TRAINING) == CA_TRAINING)
+               phy_print_training_ca_result(phy, sdram_params);
+
+       if ((training_flag & READ_TRAINING) == READ_TRAINING)
+               phy_print_training_rd_result(phy, sdram_params);
+
+       if ((training_flag & WRITE_TRAINING) == WRITE_TRAINING)
+               phy_print_training_wr_result(phy, sdram_params);
```

```
+}
+
+static void print_ddr_info(struct dram_info *dram, struct rk3568_sdram_params *sdram_params)
+{
+       struct rk3568_ddrgrf *ddrgrf = dram->ddrgrf;
+       u32 split;
+
+       if ((readl(&ddrgrf->grf_ddrsplit_con) & (1 << SPLIT_BYPASS_OFFSET)) != 0)
+               split = 0;
+       else
+               split = readl(&ddrgrf->grf_ddrsplit_con) & SPLIT_SIZE_MASK;
+
+       if (dram->ecc_en)
+               printascii("ECC enable\n");
+
+       sdram_print_ddr_info(&sdram_params->ch.cap_info, &sdram_params->base, split);
+}
+
+#else
+
+static inline void phy_print_training_result(struct rk3568_ddrphy *phy,
+                                            struct rk3568_sdram_params *sdram_params,
+                                            u32 training_flag)
+{
+}
+
+static inline void print_ddr_info(struct dram_info *dram, struct rk3568_sdram_params
*sdram_params)
+{
+}
+
+#endif /* CONFIG_RAM_ROCKCHIP_DEBUG */
+
+static int high_freq_training(struct dram_info *dram, struct rk3568_sdram_params *sdram_params,
+                              u32 fsp)
+{
+       struct skew_info *skew_info = &dram->skew_info;
+       struct rk3568_ddrphy *phy = dram->phy;
+       u32 dramtype = sdram_params->base.dramtype;
+       u32 cs, dqs;
+       int ret;
+
+       if (dramtype == LPDDR4 || dramtype == LPDDR4X)
+               send_a_refresh(dram);
+
+       for (cs = 0; cs < sdram_params->ch.cap_info.rank; ++cs) {
+               for (dqs = 0; dqs < ARRAY_SIZE(skew_info->dqs[cs]); ++dqs) {
+                       clrsetbits_le32(&phy->pad_group[dqs].reg18,
ABC_LH_TRAIN_DQS_DEFAULT_MASK,
+                                       skew_info->dqs[cs][dqs] <<
ABC_LH_TRAIN_DQS_DEFAULT_SHIFT);
+               }
+
+               ret = data_training(dram, cs, sdram_params, fsp,
+                                   READ_GATE_TRAINING | READ_TRAINING | WRITE_TRAINING);
+               if (ret)
+                       return ret;
+
+               if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG)) {
+                       printf("cs %d:\n", cs);
+                       phy_print_training_result(phy, sdram_params,
+                                                 READ_TRAINING | WRITE_TRAINING);
+               }
+       }
+
```

```
+       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG))
+               phy_print_training_result(phy, sdram_params, CA_TRAINING);
+
+       return 0;
+}
+
+static void set_ddrconfig(struct dram_info *dram, u32 ddrconfig)
+{
+       writel(ddrconfig, &dram->msch->deviceconf);
+}
+
+static void update_noc_timing(struct dram_info *dram, struct rk3568_sdram_params *sdram_params)
+{
+       void __iomem *pctl_base = dram->pctl;
+       u32 bw, bl;
+
+       bw = 8 << sdram_params->ch.cap_info.bw;
+       bl = ((readl(pctl_base + DDR_PCTL2_MSTR) >> 16) & 0xf) * 2;
+
+       /* update the noc timing related to data bus width */
+       if ((bw / 8 * bl) <= 16)
+               sdram_params->ch.noc_timings.ddrmode.b.burstsize = 0;
+       else if ((bw / 8 * bl) == 32)
+               sdram_params->ch.noc_timings.ddrmode.b.burstsize = 1;
+       else if ((bw / 8 * bl) == 64)
+               sdram_params->ch.noc_timings.ddrmode.b.burstsize = 2;
+       else
+               sdram_params->ch.noc_timings.ddrmode.b.burstsize = 3;
+
+       sdram_params->ch.noc_timings.ddrtimingc0.b.burstpenalty =
+               (bl * bw / 8) > 16 ? (bl / 4) : (16 / (bl * bw / 8)) * bl / 4;
+
+       if (sdram_params->base.dramtype == LPDDR4) {
+               sdram_params->ch.noc_timings.ddrmode.b.mwrsize = (bw == 16) ? 0x1 : 0x2;
+               sdram_params->ch.noc_timings.ddrtimingc0.b.wrtomwr =
+                       3 * sdram_params->ch.noc_timings.ddrtimingc0.b.burstpenalty;
+       }
+
+       writel(sdram_params->ch.noc_timings.ddrtiminga0.d32, &dram->msch->ddrtiminga0);
+       writel(sdram_params->ch.noc_timings.ddrtimingb0.d32, &dram->msch->ddrtimingb0);
+       writel(sdram_params->ch.noc_timings.ddrtimingc0.d32, &dram->msch->ddrtimingc0);
+       writel(sdram_params->ch.noc_timings.devtodev0.d32,   &dram->msch->devtodev0);
+       writel(sdram_params->ch.noc_timings.ddrmode.d32,     &dram->msch->ddrmode);
+       writel(sdram_params->ch.noc_timings.ddr4timing.d32,  &dram->msch->ddr4timing);
+}
+
+static void dram_all_config(struct dram_info *dram, struct rk3568_sdram_params *sdram_params,
+                           u32 post_init)
+{
+       struct sdram_cap_info *cap_info = &sdram_params->ch.cap_info;
+       u32 dram_type = sdram_params->base.dramtype;
+       void __iomem *pctl_base = dram->pctl;
+       u32 sys_reg2 = 0;
+       u32 sys_reg3 = 0;
+       u32 devicesize;
+       u64 cs_cap[2];
+       u32 cs_pst;
+
+       set_ddrconfig(dram, cap_info->ddrconfig);
+       sdram_org_config(cap_info, &sdram_params->base, &sys_reg2, &sys_reg3, 0);
+       writel(sys_reg2, &dram->pmugrf->pmu_os_reg2);
+       writel(sys_reg3, &dram->pmugrf->pmu_os_reg3);
+
+       cs_cap[0] = sdram_get_cs_cap(cap_info, 0, dram_type);
+       cs_cap[1] = sdram_get_cs_cap(cap_info, 1, dram_type);
```

```
+
+       if (cap_info->row_3_4) {
+               cs_cap[0] = cs_cap[0] * 3 / 4;
+               cs_cap[1] = cs_cap[1] * 3 / 4;
+       }
+
+       cs_pst = (readl(pctl_base + DDR_PCTL2_ADDRMAP0) & 0x1f) + 6 + 2;
+
+       if (cap_info->rank == 2) {
+               if (cs_pst > 28)
+                       cs_cap[0] = 1ull << cs_pst;
+       }
+
+       if (post_init == 0 || cs_cap[0] + cs_cap[1] != 4ull * 1024 * 1024 * 1024) {
+               if (cs_pst != 39 && cs_pst > 28)
+                       cs_cap[0] = 1ull << ((addrmap[cap_info->ddrconfig][0] & 0x1f) + 8);
+
+               devicesize = ((((cs_cap[1] >> 20) / 64) & 0xff) << 8) |
+                            (((cs_cap[0] >> 20) / 64) & 0xff);
+       } else if (cap_info->rank == 1) {
+               if (cap_info->ddrconfig == 0 || cap_info->ddrconfig == 17)
+                       devicesize = ((8ull * 1024 * 1024 * 1024 >> 20) / 64) & 0xff;
+               else
+                       devicesize = ((((4ull * 1024 * 1024 * 1024 >> 20) / 64) & 0xff) << 8) |
+                                    (((4ull * 1024 * 1024 * 1024 >> 20) / 64) & 0xff);
+       } else if (cs_pst < 28) {
+               devicesize = ((((4ull * 1024 * 1024 * 1024 >> 20) / 64) & 0xff) << 8) |
+                            (((4ull * 1024 * 1024 * 1024 >> 20) / 64) & 0xff);
+       }
+
+       writel(devicesize, &dram->msch->devicesize);
+       update_noc_timing(dram, sdram_params);
+}
+
+static void enable_low_power(struct dram_info *dram, struct rk3568_sdram_params *sdram_params)
+{
+       void __iomem *pctl_base = dram->pctl;
+
+       writel(0x1f1f0617, &dram->ddrgrf->ddr_grf_con[1]);
+
+       /* enable sr, pd */
+       if (dram->pd_idle == 0)
+               clrbits_le32(pctl_base + DDR_PCTL2_PWRCTL, (1 << 1));
+       else
+               setbits_le32(pctl_base + DDR_PCTL2_PWRCTL, (1 << 1));
+       if (dram->sr_idle == 0)
+               clrbits_le32(pctl_base + DDR_PCTL2_PWRCTL, 1);
+       else
+               setbits_le32(pctl_base + DDR_PCTL2_PWRCTL, 1);
+       setbits_le32(pctl_base + DDR_PCTL2_PWRCTL, (1 << 3));
+}
+
+static void setup_default_dqs_delays(struct dram_info *dram,
+                                    struct rk3568_sdram_params *sdram_params)
+{
+       struct rk3568_ddrphy *phy = dram->phy;
+       u32 *vdelay_sel;
+
+       u32 rank = sdram_params->ch.cap_info.rank;
+       u32 freq = sdram_params->base.ddr_freq;
+       u32 i, j, dll_lock, dqs_delay;
+
+       /* Compute dealy */
+       dll_lock = (readl(&phy->reg7d) & HALFUI_LOCK_CODE_TO_REG_MASK) >>
+                  HALFUI_LOCK_CODE_TO_REG_SHIFT;
```

```
+
+       dqs_delay = 500000 / freq;
+
+       if (dqs_delay)
+               dqs_delay = 10000 / dqs_delay;
+
+       if (dll_lock <= 64)
+               dqs_delay = (dqs_delay + 1) * -64 + 3200;
+       else
+               dqs_delay = (50 - (dqs_delay + 1)) * dll_lock;
+
+       dqs_delay /= 100;
+
+       /* Set dqs and dqsb delays for each pad group's rank */
+       for (i = 0; i < ARRAY_SIZE(phy->pad_group); ++i) {
+               for (j = 0; j < rank; ++j) {
+                       if (j == 0)
+                               vdelay_sel = &phy->pad_group[i].rege;
+                       if (j == 1)
+                               vdelay_sel = &phy->pad_group[i].reg17;
+                       else
+                               break;
+
+                       clrsetbits_le32(&vdelay_sel,
+                                       ABC_LH_CSN_DQS_INVDELAYSELRX_MASK |
+                                               ABC_LH_CSN_DQSB_INVDELAYSELRX_MASK,
+                                       (dqs_delay << ABC_LH_CSN_DQS_INVDELAYSELRX_SHIFT) |
+                                               (dqs_delay <<
ABC_LH_CSN_DQSB_INVDELAYSELRX_SHIFT));
+               }
+       }
+
+       /* Finally, ask hardware to apply just set values */
+       setbits_le32(&phy->reg24, RD_TRAIN_FREQ_UPDATE);
+       clrbits_le32(&phy->reg24, RD_TRAIN_FREQ_UPDATE);
+}
+
+static int sdram_init_(struct dram_info *dram, struct rk3568_sdram_params *sdram_params,
+                      u32 post_init)
+{
+       struct sdram_head_info_index_v2 *index = (struct sdram_head_info_index_v2 *)common_info;
+       u32 dramtype = sdram_params->base.dramtype;
+       void __iomem *pctl_base = dram->pctl;
+       struct global_info *gbl_info;
+       u32 mr_tmp;
+
+       gbl_info = (struct global_info *)((void *)common_info + index->global_index.offset * 4);
+
+       rkclk_configure_ddr(dram, sdram_params);
+
+       rkclk_ddr_reset(dram, 1, 1, 1, 1);
+       udelay(10);
+
+       rkclk_ddr_reset(dram, 1, 1, 1, 0);
+       phy_cfg(dram, sdram_params, post_init);
+
+       rkclk_ddr_reset(dram, 1, 1, 0, 0);
+       rkclk_ddr_reset(dram, 1, 0, 0, 0);
+       pctl_cfg(dram->pctl, &sdram_params->pctl_regs, dram->sr_idle, dram->pd_idle);
+
+       if ((dramtype == LPDDR4 || dramtype == LPDDR4X) && sdram_params->ch.cap_info.bw == 0)
+               setbits_le32(pctl_base + DDR_PCTL2_ZQCTL0, PCTL2_ZQ_RESISTOR_SHARED);
+
+       /* Enable ordered read/writes for port 0 */
+       setbits_le32(pctl_base + DDR_PCTL2_PCFGR_n, 1 << 16);
```

```
+
+       if (dram->ecc_en && post_init)
+               clrsetbits_le32(pctl_base + DDR_PCTL2_ECCCFG0, PCTL2_ECC_MODE_MASK,
+                               PCTL2_ECC_MODE_EN << PCTL2_ECC_MODE_SHIFT);
+
+       /* Use target_frequency's register set */
+       setbits_le32(pctl_base + DDR_PCTL2_MSTR, 0x1 << 29);
+       /* Select Frequency 0 as target_frequency register set */
+       clrsetbits_le32(pctl_base + DDR_PCTL2_MSTR2, 0x3, 0);
+
+       set_ds_odt(dram, sdram_params, 0);
+       sdram_params->ch.cap_info.ddrconfig = calculate_ddrconfig(sdram_params);
+       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG) && post_init)
+               printf("ddrconfig: %d\n", sdram_params->ch.cap_info.ddrconfig);
+
+       set_ctl_address_map(dram, sdram_params);
+
+       if (dram->ecc_en && post_init)
+               /* TODO: Configure ECC poisoning */
+               ;
+
+       /* Enable pageclose if configured to do so */
+       if (DDR_PAGECLOSE_EN(gbl_info->reserved[1]))
+               setbits_le32(pctl_base + DDR_PCTL2_SCHED, PCTL2_PAGECLOSE_EN);
+
+       setbits_le32(pctl_base + DDR_PCTL2_DFIMISC, (1 << 5) | (1 << 4));
+
+       rkclk_ddr_reset(dram, 0, 0, 0, 0);
+
+       while ((readl(pctl_base + DDR_PCTL2_STAT) & PCTL2_OPERATING_MODE_MASK) ==
+              PCTL2_OPERATING_MODE_INIT)
+               continue;
+
+       /* Setup default delays for dqs before doing gate training. */
+       setup_default_dqs_delays(dram, sdram_params);
+
+       if (dramtype == LPDDR3) {
+               pctl_write_mr(dram->pctl, 3, 11, lp3_odt_value, dramtype);
+       } else if (dramtype == LPDDR4) {
+               mr_tmp = readl(pctl_base + DDR_PCTL2_INIT6);
+               /* MR11 */
+               pctl_write_mr(dram->pctl, 0xf, 11,
+                             mr_tmp >> PCTL2_LPDDR4_MR11_SHIFT & PCTL2_MR_MASK, dramtype);
+               /* MR12 */
+               pctl_write_mr(dram->pctl, 0xf, 12,
+                             mr_tmp >> PCTL2_LPDDR4_MR12_SHIFT & PCTL2_MR_MASK, dramtype);
+
+               mr_tmp = readl(pctl_base + DDR_PCTL2_INIT7);
+               /* MR22 */
+               pctl_write_mr(dram->pctl, 0xf, 22,
+                             mr_tmp >> PCTL2_LPDDR4_MR22_SHIFT & PCTL2_MR_MASK, dramtype);
+               send_a_refresh(dram);
+       } else if (dramtype == DDR4) {
+               mr_tmp = readl(pctl_base + DDR_PCTL2_INIT7);
+               /* MR6 */
+               pctl_write_mr(dram->pctl, 3, 6,
+                             (mr_tmp >> PCTL2_DDR4_MR6_SHIFT & PCTL2_MR_MASK) | 0x80, dramtype);
+               pctl_write_mr(dram->pctl, 3, 6,
+                             (mr_tmp >> PCTL2_DDR4_MR6_SHIFT & PCTL2_MR_MASK) | 0x80, dramtype);
+               pctl_write_mr(dram->pctl, 3, 6, (mr_tmp >> PCTL2_DDR4_MR6_SHIFT & PCTL2_MR_MASK),
+                             dramtype);
+       }
+
+       if (data_training(dram, 0, sdram_params, 0, READ_GATE_TRAINING) != 0) {
+               if (post_init != 0)
```

```
+                       printascii("DTT cs0 error\n");
+               return -1;
+       }
+
+       if (dramtype == LPDDR4 || dramtype == LPDDR4X) {
+               mr_tmp = read_mr(dram, 1, 14, dramtype);
+
+               if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG) && post_init)
+                       printf("LP4 MR14: 0x%x\n", mr_tmp);
+
+               if (mr_tmp != 0x4d)
+                       return -1;
+       }
+
+       if (dramtype == LPDDR4 || dramtype == LPDDR4X) {
+               mr_tmp = readl(pctl_base + DDR_PCTL2_INIT7);
+               /* MR14 */
+               pctl_write_mr(dram->pctl, 3, 14, mr_tmp >> PCTL2_LPDDR4_MR14_SHIFT &
PCTL2_MR_MASK,
+                             dramtype);
+       }
+       if (post_init != 0 && sdram_params->ch.cap_info.rank == 2) {
+               if (data_training(dram, 1, sdram_params, 0, READ_GATE_TRAINING) != 0) {
+                       printascii("DTT cs1 error\n");
+                       return -1;
+               }
+       }
+
+       dram_all_config(dram, sdram_params, post_init);
+       enable_low_power(dram, sdram_params);
+
+       return 0;
+}
+
+static u64 dram_detect_cap(struct dram_info *dram, struct rk3568_sdram_params *sdram_params,
+                          unsigned char channel)
+{
+       struct sdram_cap_info *cap_info = &sdram_params->ch.cap_info;
+       u32 dram_type = sdram_params->base.dramtype;
+       struct rk3568_ddrphy *phy = dram->phy;
+       void __iomem *pctl_base = dram->pctl;
+       u32 pwrctl;
+       u32 coltmp;
+       u32 rowtmp;
+       u32 bktmp;
+       u32 mr8;
+       u32 cs;
+
+       cap_info->bw = dram_type == DDR3 ? 0 : 1;
+
+       if (dram_type != LPDDR4 && dram_type != LPDDR4X) {
+               if (dram_type != DDR4) {
+                       coltmp = 12;
+                       bktmp = 3;
+                       if (dram_type == LPDDR2)
+                               rowtmp = 15;
+                       else
+                               rowtmp = 16;
+
+                       if (sdram_detect_col(cap_info, coltmp) != 0)
+                               goto cap_err;
+
+                       sdram_detect_bank(cap_info, coltmp, bktmp);
+                       if (dram_type != LPDDR3)
+                               sdram_detect_dbw(cap_info, dram_type);
```

```
+               } else {
+                       coltmp = 10;
+                       bktmp = 4;
+                       rowtmp = 17;
+
+                       cap_info->col = 10;
+                       cap_info->bk = 2;
+                       sdram_detect_bg(cap_info, coltmp);
+               }
+
+               if (sdram_detect_row(cap_info, coltmp, bktmp, rowtmp) != 0)
+                       goto cap_err;
+
+               sdram_detect_row_3_4(cap_info, coltmp, bktmp);
+       } else {
+               cap_info->col = 10;
+               cap_info->bk = 3;
+               mr8 = read_mr(dram, 1, 8, dram_type);
+               cap_info->dbw = ((mr8 >> 6) & 0x3) == 0 ? 1 : 0;
+               mr8 = (mr8 >> 2) & 0xf;
+
+               cap_info->cs0_row = 14 + (mr8 + 1) / 2;
+               if (cap_info->dbw == 0)
+                       cap_info->cs0_row++;
+
+               cap_info->row_3_4 = mr8 % 2 == 1 ? 1 : 0;
+               cap_info->bw = 2;
+       }
+
+       pwrctl = readl(pctl_base + DDR_PCTL2_PWRCTL);
+       writel(0, pctl_base + DDR_PCTL2_PWRCTL);
+
+       if (data_training(dram, 1, sdram_params, 0, READ_GATE_TRAINING) == 0)
+               cs = 1;
+       else
+               cs = 0;
+
+       /* TODO: drop me. We won't support 4 ranks for now.*/
+       if (cs == 1) {
+               if (dram_type == LPDDR3) {
+                       if (data_training(dram, 3, sdram_params, 0, READ_GATE_TRAINING) == 0)
+                               cs = 3;
+               } else if (dram_type == LPDDR4 || dram_type == LPDDR4X) {
+                       if (data_training(dram, 3, sdram_params, 0, READ_GATE_TRAINING) == 0) {
+                               dram->cmd_perbit_skew_bp = true;
+                               cs = 3;
+                       } else if (data_training(dram, 2, sdram_params, 0, READ_GATE_TRAINING) ==
+                                  0) {
+                               dram->cmd_perbit_skew_bp = true;
+                       }
+               }
+       }
+       cap_info->rank = cs + 1;
+
+       if (dram_type == LPDDR4X) {
+               clrsetbits_le32(&phy->reg0, CHANNEL_EN_MASK,
+                               ((1 << 3) | (1 << 2) | (1 << 1) | (1 << 0)) << CHANNEL_EN_SHIFT);
+               if (data_training(dram, 0, sdram_params, 0, READ_GATE_TRAINING) == 0)
+                       cap_info->bw = 2;
+               else
+                       cap_info->bw = 1;
+       }
+
+       writel(pwrctl, pctl_base + DDR_PCTL2_PWRCTL);
+
```

```
+       cap_info->cs0_high16bit_row = cap_info->cs0_row;
+       if (cs) {
+               cap_info->cs1_row = cap_info->cs0_row;
+               cap_info->cs1_high16bit_row = cap_info->cs0_row;
+       } else {
+               cap_info->cs1_row = 0;
+               cap_info->cs1_high16bit_row = 0;
+       }
+
+       if (dram_type == LPDDR3)
+               sdram_detect_dbw(cap_info, dram_type);
+
+       return 0;
+cap_err:
+       return -1;
+}
+
+static int dram_detect_cs1_row(struct dram_info *dram, struct rk3568_sdram_params *sdram_params,
+                               unsigned char channel)
+{
+       struct sdram_cap_info *cap_info = &sdram_params->ch.cap_info;
+       void __iomem *pctl_base = dram->pctl;
+       u32 row, bktmp, coltmp, bw;
+       void __iomem *test_addr;
+       u32 cs_add = 0;
+       u32 byte_mask;
+       u32 ret = 0;
+       u32 max_row;
+       u64 cs0_cap;
+       u32 cs_pst;
+
+       if (cap_info->rank == 2) {
+               cs_pst = (readl(pctl_base + DDR_PCTL2_ADDRMAP0) & 0x1f) + 6 + 2;
+               if (cs_pst < 28)
+                       cs_add = 1;
+
+               cs0_cap = 1 << cs_pst;
+
+               if (sdram_params->base.dramtype == DDR4) {
+                       if (cap_info->dbw == 0)
+                               bktmp = cap_info->bk + 2;
+                       else
+                               bktmp = cap_info->bk + 1;
+               } else {
+                       bktmp = cap_info->bk;
+               }
+               bw = cap_info->bw;
+               coltmp = cap_info->col;
+
+               if (bw == 2)
+                       byte_mask = 0xffff;
+               else
+                       byte_mask = 0xff;
+
+               max_row = (cs_pst == 31) ? 30 : 31;
+
+               max_row = max_row - bktmp - coltmp - bw - cs_add + 1;
+
+               row = (cap_info->cs0_row > max_row) ? max_row : cap_info->cs0_row;
+
+               for (; row > 12; row--) {
+                       test_addr = (void __iomem *)(CFG_SYS_SDRAM_BASE + cs0_cap +
+                                                    (1ul << (row + bktmp + coltmp + cs_add + bw
-
+                                                             1ul)));
```

```
+
+                       writel(0, CFG_SYS_SDRAM_BASE + cs0_cap);
+                       writel(PATTERN, test_addr);
+
+                       if (((readl(test_addr) & byte_mask) == (PATTERN & byte_mask)) &&
+                           ((readl(CFG_SYS_SDRAM_BASE + cs0_cap) & byte_mask) == 0)) {
+                               ret = row;
+                               break;
+                       }
+               }
+       }
+
+       return ret;
+}
+
+/* return: 0 = success, other = fail */
+static int sdram_init_detect(struct dram_info *dram, struct rk3568_sdram_params *sdram_params)
+{
+       struct sdram_head_info_index_v2 *index = (struct sdram_head_info_index_v2 *)common_info;
+       struct sdram_cap_info *cap_info = &sdram_params->ch.cap_info;
+       struct dq_map_info *map_info;
+       u32 ecc_detected;
+       u32 sys_reg3 = 0;
+       u32 sys_reg = 0;
+       u32 ret;
+
+       map_info = (struct dq_map_info *)((void *)common_info + index->dq_map_index.offset * 4);
+
+       if (sdram_params->base.dramtype != DDR3 || sdram_params->base.dramtype != DDR4) {
+               ecc_detected = false;
+       } else {
+               dram_info.ecc_en = true;
+               ret = sdram_init_(dram, sdram_params, 0);
+               ecc_detected = ret == 0;
+       }
+
+       dram_info.cmd_perbit_skew_bp = false;
+       dram_info.ecc_en = false;
+
+       if (sdram_init_(dram, sdram_params, 0))
+               return -1;
+
+       if (sdram_params->base.dramtype == DDR3) {
+               writel(PATTERN, CFG_SYS_SDRAM_BASE);
+               if (readl(CFG_SYS_SDRAM_BASE) != PATTERN)
+                       return -1;
+       }
+
+       if ((sdram_params->base.dramtype == LPDDR4 || sdram_params->base.dramtype == LPDDR4X) &&
+           sdram_params->base.ddr_freq >= 334) {
+               data_training(dram, 0, sdram_params, 0, FULL_TRAINING);
+       }
+
+       if (dram_detect_cap(dram, sdram_params, 0) != 0)
+               return -1;
+
+       pctl_remodify_sdram_params(&sdram_params->pctl_regs, cap_info, sdram_params-
>base.dramtype);
+       dram_info.ecc_en = ecc_detected;
+       ret = sdram_init_(dram, sdram_params, 1);
+       if (ret != 0)
+               goto out;
+
+       if (!ecc_detected) {
+               cap_info->cs1_row = dram_detect_cs1_row(dram, sdram_params, 0);
```

```
+               if (cap_info->cs1_row) {
+                       sys_reg = readl(&dram->pmugrf->pmu_os_reg2);
+                       sys_reg3 = readl(&dram->pmugrf->pmu_os_reg3);
+                       SYS_REG_ENC_CS1_ROW(cap_info->cs1_row, sys_reg, sys_reg3, 0);
+                       writel(sys_reg, &dram->pmugrf->pmu_os_reg2);
+                       writel(sys_reg3, &dram->pmugrf->pmu_os_reg3);
+               }
+
+               sdram_detect_high_row(cap_info);
+       }
+out:
+       return ret;
+}
+
+struct rk3568_sdram_params *get_default_sdram_config(u32 freq_mhz)
+{
+       struct ddr2_3_4_lp2_3_info *ddr_info;
+       u32 offset = 0;
+       u32 i;
+
+       if (!freq_mhz) {
+               ddr_info = get_ddr_drv_odt_info(sdram_configs[0].base.dramtype);
+               if (ddr_info)
+                       freq_mhz = (ddr_info->ddr_freq0_1 >> DDR_FREQ_F0_SHIFT) & DDR_FREQ_MASK;
+               else
+                       freq_mhz = 0;
+       }
+
+       for (i = 0; i < ARRAY_SIZE(sdram_configs); i++) {
+               if (sdram_configs[i].base.ddr_freq == 0 ||
+                   freq_mhz < sdram_configs[i].base.ddr_freq)
+                       break;
+       }
+       offset = i == 0 ? 0 : i - 1;
+
+       return &sdram_configs[offset];
+}
+
+static void pctl_derate_en(struct dram_info *dram, struct rk3568_sdram_params *sdram_params,
+                          u32 dst_fsp)
+{
+       u32 dramtype = sdram_params->base.dramtype;
+       u32 freq = sdram_params->base.ddr_freq;
+       void __iomem *pctl_base = dram->pctl;
+       u32 tmp, mr_tmp, cur_fsp;
+
+       if (!dram->derate_en || (dramtype != LPDDR4 && dramtype != LPDDR4X))
+               return;
+
+       tmp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_DERATEEN);
+       /* derate_value */
+       tmp &= ~((u32)0x3 << 1);
+       /* derate_byte */
+       tmp &= ~((u32)0xf << 4);
+       /* rc_derate_value */
+       tmp &= ~((u32)0x7 << 8);
+       tmp |= ((((freq * 1875 + 1999999) / 2000000 - 1) & 3) << 1) |
+              ((((freq * 3750 + 1999999) / 2000000 - 1) & 7) << 8);
+       writel(tmp, pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_DERATEEN);
+       setbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_DERATEEN,
+                    PCTL2_DERATE_MR4_TUF_DIS);
+
+       mr_tmp = read_mr(dram, 1, 0, dramtype);
+       if (mr_tmp & 1) {
+               tmp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT4);
```

```
+               tmp &= ~((u32)1 << 4);
+               writel(tmp, pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT4);
+       }
+
+       writel((freq * 167) / 2, pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_DERATEINT);
+       setbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_DERATEEN,
+                    PCTL2_DERATE_ENABLE);
+
+       cur_fsp = readl(pctl_base + DDR_PCTL2_MSTR2) & 0x3;
+
+       setbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_DERATEEN,
+                    PCTL2_DERATE_MR4_PAUSE_FC);
+       setbits_le32(pctl_base + UMCTL2_REGS_FREQ(cur_fsp) + DDR_PCTL2_DERATEEN,
+                    PCTL2_DERATE_MR4_PAUSE_FC);
+}
+
+static const u16 pctl_need_update_reg[] = {
+       DDR_PCTL2_RFSHTMG,
+       DDR_PCTL2_INIT3,
+       DDR_PCTL2_INIT4,
+       DDR_PCTL2_INIT6,
+       DDR_PCTL2_INIT7,
+       DDR_PCTL2_RANKCTL,
+       DDR_PCTL2_DRAMTMG0,
+       DDR_PCTL2_DRAMTMG1,
+       DDR_PCTL2_DRAMTMG2,
+       DDR_PCTL2_DRAMTMG3,
+       DDR_PCTL2_DRAMTMG4,
+       DDR_PCTL2_DRAMTMG5,
+       DDR_PCTL2_DRAMTMG6,
+       DDR_PCTL2_DRAMTMG7,
+       DDR_PCTL2_DRAMTMG8,
+       DDR_PCTL2_DRAMTMG9,
+       DDR_PCTL2_DRAMTMG12,
+       DDR_PCTL2_DRAMTMG13,
+       DDR_PCTL2_DRAMTMG14,
+       DDR_PCTL2_ZQCTL0,
+       DDR_PCTL2_DFITMG0,
+       DDR_PCTL2_ODTCFG
+};
+
+static const u16 phy_need_update_reg[] = {
+       0x8,
+       0xc,
+       0x10
+};
+
+static void pre_set_rate(struct dram_info *dram, struct rk3568_sdram_params *sdram_params,
+                        u32 dst_fsp, u32 dst_fsp_lp4)
+{
+       u32 dramtype = sdram_params->base.dramtype;
+       struct rk3568_ddrphy *phy = dram->phy;
+       void __iomem *pctl_base = dram->pctl;
+       void __iomem *phy_base = dram->phy;
+       u32 tmp, mr_tmp, mr_rank;
+       u32 phy_fsp_shift;
+       u32 i, j, find;
+
+       sw_set_req(dram);
+       /* pctl timing update */
+       for (i = 0, find = 0; i < ARRAY_SIZE(pctl_need_update_reg); i++) {
+               for (j = find; sdram_params->pctl_regs.pctl[j][0] != 0xffffffff; j++) {
+                       if (sdram_params->pctl_regs.pctl[j][0] == pctl_need_update_reg[i]) {
+                               writel(sdram_params->pctl_regs.pctl[j][1],
+                                      pctl_base + UMCTL2_REGS_FREQ(dst_fsp) +
```

```
+                                                pctl_need_update_reg[i]);
+                                find = j;
+                                break;
+                        }
+                }
+        }
+
+        pctl_derate_en(dram, sdram_params, dst_fsp);
+        sw_set_ack(dram);
+
+        /* phy timing update */
+        phy_fsp_shift = (dst_fsp * 8);
+
+        /* cl cwl al update */
+        for (i = 0, find = 0; i < ARRAY_SIZE(phy_need_update_reg); i++) {
+                for (j = find; sdram_params->phy_regs.phy[j][0] != 0xffffffff; j++) {
+                        if (sdram_params->phy_regs.phy[j][0] == phy_need_update_reg[i]) {
+                                tmp = readl(phy_base + phy_need_update_reg[i]);
+                                tmp &= ~((u32)(0x1f << (24 - phy_fsp_shift)));
+                                tmp |= ((u32)(sdram_params->phy_regs.phy[j][1] >>
phy_fsp_shift));
+                                writel(tmp, phy_base + phy_need_update_reg[i]);
+                                find = j;
+                                break;
+                        }
+                }
+        }
+
+        if (dramtype == LPDDR4 || dramtype == LPDDR4X) {
+                mr_tmp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT4);
+                mr_tmp = (mr_tmp >> PCTL2_LPDDR4_MR13_SHIFT) & PCTL2_MR_MASK;
+                /* FSP-WR, FSP-OP */
+                mr_tmp &= ~(BIT(7) | BIT(6));
+
+                /* MR13 */
+                pctl_write_mr(dram->pctl, 0xf, 13, mr_tmp | ((0x2 << 6) >> dst_fsp_lp4),
dramtype);
+                clrsetbits_le32(&phy->regb, DDRPHY_MR13_MASK,
+                                (mr_tmp | (dst_fsp_lp4 << 7) | (dst_fsp_lp4 << 6))
+                                        << DDRPHY_MR13_SHIFT);
+        }
+
+        set_ds_odt(dram, sdram_params, dst_fsp);
+
+        if (dramtype == LPDDR4 || dramtype == LPDDR4X) {
+                mr_tmp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT4);
+
+                /* MR3 */
+                pctl_write_mr(dram->pctl, 0xf, 3,
+                              mr_tmp >> PCTL2_LPDDR234_MR3_SHIFT & PCTL2_MR_MASK, dramtype);
+                clrsetbits_le32(&phy->rega, DDRPHY_MR3_MASK,
+                                ((mr_tmp >> PCTL2_LPDDR234_MR3_SHIFT) << DDRPHY_MR3_SHIFT) &
+                                        DDRPHY_MR3_MASK);
+
+                mr_tmp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT3);
+                /* MR1 */
+                pctl_write_mr(dram->pctl, 0xf, 1,
+                              mr_tmp >> PCTL2_LPDDR234_MR1_SHIFT & PCTL2_MR_MASK, dramtype);
+                clrsetbits_le32(&phy->rega, DDRPHY_MR1_MASK,
+                                ((mr_tmp >> PCTL2_LPDDR234_MR1_SHIFT) << DDRPHY_MR1_SHIFT) &
+                                        DDRPHY_MR1_MASK);
+                /* MR2 */
+                pctl_write_mr(dram->pctl, 0xf, 2, mr_tmp & PCTL2_MR_MASK, dramtype);
+                clrsetbits_le32(&phy->rega, DDRPHY_MR2_MASK,
+                                (mr_tmp << DDRPHY_MR2_SHIFT) & DDRPHY_MR2_MASK);
```

```
+
+               mr_tmp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT6);
+               /* MR11 */
+               if (!dram->cmd_perbit_skew_bp || sdram_params->ch.cap_info.rank != 2) {
+                       pctl_write_mr(dram->pctl, 0x5, 11,
+                                     mr_tmp >> PCTL2_LPDDR4_MR11_SHIFT & PCTL2_MR_MASK,
dramtype);
+                       mr_rank = 0xa;
+               } else {
+                       pctl_write_mr(dram->pctl, 0x3, 11,
+                                     mr_tmp >> PCTL2_LPDDR4_MR11_SHIFT & PCTL2_MR_MASK,
dramtype);
+                       mr_rank = 0xc;
+               }
+               pctl_write_mr(dram->pctl, mr_rank, 11,
+                             (mr_tmp >> PCTL2_LPDDR4_MR11_SHIFT & PCTL2_MR_MASK) &
+                                     ~((u32)0x7 << 4),
+                             dramtype);
+               clrsetbits_le32(&phy->rega, DDRPHY_MR11_MASK,
+                               ((mr_tmp >> PCTL2_LPDDR4_MR11_SHIFT) << DDRPHY_MR11_SHIFT) &
+                                       DDRPHY_MR11_MASK);
+               /* MR12 */
+               pctl_write_mr(dram->pctl, 0xf, 12,
+                             mr_tmp >> PCTL2_LPDDR4_MR12_SHIFT & PCTL2_MR_MASK, dramtype);
+
+               mr_tmp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT7);
+               /* MR22 */
+               pctl_write_mr(dram->pctl, 0xf, 22,
+                             mr_tmp >> PCTL2_LPDDR4_MR22_SHIFT & PCTL2_MR_MASK, dramtype);
+               clrsetbits_le32(&phy->regb, DDRPHY_MR22_MASK,
+                               ((mr_tmp >> PCTL2_LPDDR4_MR22_SHIFT) << DDRPHY_MR22_SHIFT) &
+                                       DDRPHY_MR22_MASK);
+               /* MR14 */
+               pctl_write_mr(dram->pctl, 0xf, 14,
+                             mr_tmp >> PCTL2_LPDDR4_MR14_SHIFT & PCTL2_MR_MASK, dramtype);
+               clrsetbits_le32(&phy->regb, DDRPHY_MR14_MASK,
+                               ((mr_tmp >> PCTL2_LPDDR4_MR14_SHIFT) << DDRPHY_MR14_SHIFT) &
+                                       DDRPHY_MR14_MASK);
+       }
+
+       update_noc_timing(dram, sdram_params);
+}
+
+static void save_fsp_param(struct dram_info *dram, u32 dst_fsp,
+                          struct rk3568_sdram_params *sdram_params)
+{
+       struct rk3568_fsp_param *p_fsp_param = &fsp_param[dst_fsp];
+       struct rk3568_ddrphy *phy = dram->phy;
+       void __iomem *pctl_base = dram->pctl;
+       struct ddr2_3_4_lp2_3_info *ddr_info;
+       struct cb_invdelaysel_addr addr;
+       u32 temp;
+       int i;
+
+       ddr_info = get_ddr_drv_odt_info(sdram_params->base.dramtype);
+
+       /* TODO: blob reads current frequency directly from dpll. Must we? */
+       p_fsp_param->freq_mhz = sdram_params->base.ddr_freq;
+
+       if (sdram_params->base.dramtype == LPDDR4 || sdram_params->base.dramtype == LPDDR4X) {
+               p_fsp_param->rd_odt_up_en = 0;
+               p_fsp_param->rd_odt_down_en = 1;
+       } else {
+               p_fsp_param->rd_odt_up_en = ODT_INFO_PULLUP_EN(ddr_info->odt_info);
+               p_fsp_param->rd_odt_down_en = ODT_INFO_PULLDOWN_EN(ddr_info->odt_info);
```

```
+       }
+
+       if (p_fsp_param->rd_odt_up_en)
+               p_fsp_param->rd_odt = (readl(&phy->pad_group[0].reg1) & ABC_LH_ABUTODTPU_MASK) >>
+                                     ABC_LH_ABUTODTPU_SHIFT;
+       else if (p_fsp_param->rd_odt_down_en)
+               p_fsp_param->rd_odt = (readl(&phy->pad_group[0].reg1) & ABC_LH_ABUTODTPD_MASK) >>
+                                     ABC_LH_ABUTODTPD_SHIFT;
+       else
+               p_fsp_param->rd_odt = 0;
+       p_fsp_param->wr_dq_drv = (readl(&phy->pad_group[0].reg1) & ABC_LH_ABUTPRCOMP_MASK) >>
+                                ABC_LH_ABUTPRCOMP_SHIFT;
+       p_fsp_param->wr_ca_drv = (readl(&phy->reg38) & CMD_ABUTPRCOMP_MASK) >>
CMD_ABUTPRCOMP_SHIFT;
+       p_fsp_param->wr_ckcs_drv = (readl(&phy->reg38) & CMD_ABUTPRCOMP_CK0_MASK) >>
+                                  CMD_ABUTPRCOMP_CK0_SHIFT;
+       p_fsp_param->vref_inner = (readl(&phy->pad_group[0].reg0) & ABC_LH_VREF1_MARGSEL_MASK) >>
+                                 ABC_LH_VREF1_MARGSEL_SHIFT;
+       p_fsp_param->vref_out = (readl(&phy->undocumented0) & RAM_VREF1_MARGSEL_MASK) >>
+                               RAM_VREF1_MARGSEL_SHIFT;
+
+       if (sdram_params->base.dramtype == DDR3) {
+               temp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT3);
+               temp = (temp >> PCTL2_DDR34_MR1_SHIFT) & PCTL2_MR_MASK;
+               p_fsp_param->ds_pdds = temp & DDR3_DS_MASK;
+               p_fsp_param->dq_odt = temp & DDR3_RTT_NOM_MASK;
+               p_fsp_param->ca_odt = p_fsp_param->dq_odt;
+       } else if (sdram_params->base.dramtype == DDR4) {
+               temp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT3);
+               temp = (temp >> PCTL2_DDR34_MR1_SHIFT) & PCTL2_MR_MASK;
+               p_fsp_param->ds_pdds = temp & DDR4_DS_MASK;
+               p_fsp_param->dq_odt = temp & DDR4_RTT_NOM_MASK;
+               p_fsp_param->ca_odt = p_fsp_param->dq_odt;
+       } else if (sdram_params->base.dramtype == LPDDR3) {
+               temp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT4);
+               temp = (temp >> PCTL2_LPDDR234_MR3_SHIFT) & PCTL2_MR_MASK;
+               p_fsp_param->ds_pdds = temp & 0xf;
+
+               p_fsp_param->dq_odt = lp3_odt_value;
+               p_fsp_param->ca_odt = p_fsp_param->dq_odt;
+       } else if (sdram_params->base.dramtype == LPDDR4 ||
+                  sdram_params->base.dramtype == LPDDR4X) {
+               temp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT4);
+               temp = (temp >> PCTL2_LPDDR234_MR3_SHIFT) & PCTL2_MR_MASK;
+               p_fsp_param->ds_pdds = temp & LPDDR4_PDDS_MASK;
+
+               temp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT6);
+               temp = (temp >> PCTL2_LPDDR4_MR11_SHIFT) & PCTL2_MR_MASK;
+               p_fsp_param->dq_odt = temp & LPDDR4_DQODT_MASK;
+               p_fsp_param->ca_odt = temp & LPDDR4_CAODT_MASK;
+
+               p_fsp_param->lp4_drv_pd_en =
+                       (readl(&phy->pad_group[0].reg0) & ABC_LH_ENB_LP4MODE_MASK) >>
+                       ABC_LH_ENB_LP4MODE_SHIFT;
+       }
+
+       if (!(readl(&phy->regd) & CMD_PERBIT_SKEW_BP)) {
+               for (i = 0; i < 8 * sizeof(u32); ++i) {
+                       addr.reg = i / sizeof(u32);
+                       addr.byte = (sizeof(u32) - 1) - i % sizeof(u32);
+
+                       p_fsp_param->ca_prebit_skew[i] = read_ca_prebit(phy, addr);
+               }
+       }
+
```

```
+       p_fsp_param->noc_timings.ddrtiminga0 = sdram_params->ch.noc_timings.ddrtiminga0;
+       p_fsp_param->noc_timings.ddrtimingb0 = sdram_params->ch.noc_timings.ddrtimingb0;
+       p_fsp_param->noc_timings.ddrtimingc0 = sdram_params->ch.noc_timings.ddrtimingc0;
+       p_fsp_param->noc_timings.devtodev0   = sdram_params->ch.noc_timings.devtodev0;
+       p_fsp_param->noc_timings.ddrmode     = sdram_params->ch.noc_timings.ddrmode;
+       p_fsp_param->noc_timings.ddr4timing  = sdram_params->ch.noc_timings.ddr4timing;
+       p_fsp_param->noc_timings.agingx0     = sdram_params->ch.noc_timings.agingx0;
+       p_fsp_param->noc_timings.aging0      = sdram_params->ch.noc_timings.aging0;
+       p_fsp_param->noc_timings.aging1      = sdram_params->ch.noc_timings.aging1;
+       p_fsp_param->noc_timings.aging2      = sdram_params->ch.noc_timings.aging2;
+       p_fsp_param->noc_timings.aging3      = sdram_params->ch.noc_timings.aging3;
+
+       p_fsp_param->flag = FSP_FLAG;
+}
+
+static void copy_fsp_param_to_ddr(void)
+{
+       memcpy((void *)FSP_PARAM_STORE_ADDR, (void *)&fsp_param, sizeof(fsp_param));
+}
+
+static void phy_calc_skew(struct dram_info *dram, struct rk3568_sdram_params *sdram_params,
+                         u32 freq, u32 dst_fsp)
+{
+       u32 skew, min_skew, clk_skew_a, clk_skew_b, max_clk_skew, dqss2dq_skew;
+       struct skew_info *skew_info = &dram->skew_info;
+       u32 dramtype = sdram_params->base.dramtype;
+       struct rk3568_ddrphy *phy;
+       u32 i;
+
+       if ((dramtype == LPDDR4 || dramtype == LPDDR4X) && !dram->cmd_perbit_skew_bp) {
+               phy = dram->phy;
+               min_skew = 0xff;
+
+               for (i = CMD_INVDELAYSEL_SEL_A_CA0_CS0; i <= CMD_INVDELAYSEL_SEL_B_ODT1; ++i) {
+                       if (sdram_params->ch.cap_info.rank == 1) {
+                               if (i >= CMD_INVDELAYSEL_SEL_A_CA0_CS1 &&
+                                   i <= CMD_INVDELAYSEL_SEL_B_CA5_CS1)
+                                       /* Only interested in skew for cs0 (rank 1) */
+                                       continue;
+
+                               if (i == CMD_INVDELAYSEL_SEL_A_CSB1 ||
+                                   i == CMD_INVDELAYSEL_SEL_B_CSB1)
+                                       /* Not interested in CS1 skew for both channels either */
+                                       continue;
+                       }
+
+                       clrsetbits_le32(&phy->reg6c, CMD_INVDELAYSEL_SEL_MASK,
+                                       i << CMD_INVDELAYSEL_SEL_SHIFT);
+                       skew = (readl(&phy->undocumented1) & CMD_INVDELAYSEL_MASK) >>
+                              CMD_INVDELAYSEL_SHIFT;
+                       if (skew < min_skew)
+                               min_skew = skew;
+               }
+
+               clrsetbits_le32(&phy->reg6c, CMD_INVDELAYSEL_SEL_MASK,
+                               CMD_INVDELAYSEL_SEL_A_CK << CMD_INVDELAYSEL_SEL_SHIFT);
+               clk_skew_a = (readl(&phy->undocumented1) & CMD_INVDELAYSEL_MASK) >>
+                            CMD_INVDELAYSEL_SHIFT;
+
+               clrsetbits_le32(&phy->reg6c, CMD_INVDELAYSEL_SEL_MASK,
+                               CMD_INVDELAYSEL_SEL_B_CK << CMD_INVDELAYSEL_SEL_SHIFT);
+               clk_skew_b = (readl(&phy->undocumented1) & CMD_INVDELAYSEL_MASK) >>
+                            CMD_INVDELAYSEL_SHIFT;
+
+               max_clk_skew = clk_skew_a < clk_skew_b ? clk_skew_b : clk_skew_a;
```

```
+
+               if (skew_info->t_dqss2dq == 0) {
+                       skew_info->clk_skew = 96 - (skew_info->t_dqss_max + skew_info-
>t_dqss_min) /
+                                                       2 * ((s32)freq) * 128 / 1000000;
+               } else {
+                       dqss2dq_skew = skew_info->t_dqss2dq * freq * 128 / 1000000;
+                       if (skew_info->t_dqss_min < 0) {
+                               skew_info->clk_skew =
+                                       dqss2dq_skew < 96 ?
+                                               96 - (skew_info->t_dqss_min * ((s32)freq) * 128 /
+                                                             1000000 +
+                                                     ((s32)dqss2dq_skew)) :
+                                               -(skew_info->t_dqss_min * ((s32)freq) * 128 /
+                                                 1000000);
+                       } else {
+                               skew_info->clk_skew =
+                                       skew_info->t_dqss_max * ((s32)freq) * 128 / 1000000 +
+                                       ((s32)dqss2dq_skew);
+                               skew_info->clk_skew =
+                                       skew_info->clk_skew < 176 ? 176 - skew_info->clk_skew :
0;
+                       }
+
+                       if (max_clk_skew - min_skew > skew_info->clk_skew)
+                               skew_info->clk_skew = max_clk_skew - min_skew;
+               }
+
+               if (dst_fsp != 0 && skew_info->clk_skew < 88)
+                       skew_info->clk_skew = 88;
+
+               skew_info->clk_delta = skew_info->clk_skew - max_clk_skew;
+
+               for (i = 0; i < 4; ++i) {
+                       if (i <= 1) {
+                               skew_info->dqs[0][i] =
+                                       wrlvl_result[0][i] * ((s32)freq) * 128 / 1000000 +
+                                       ((s32)skew_info->clk_delta) + ((s32)clk_skew_a);
+                               skew_info->dqs[1][i] =
+                                       wrlvl_result[1][i] * ((s32)freq) * 128 / 1000000 +
+                                       ((s32)skew_info->clk_delta) + ((s32)clk_skew_a);
+                       } else {
+                               skew_info->dqs[0][i] =
+                                       wrlvl_result[0][i] * ((s32)freq) * 128 / 1000000 +
+                                       ((s32)skew_info->clk_delta) + ((s32)clk_skew_b);
+                               skew_info->dqs[1][i] =
+                                       wrlvl_result[1][i] * ((s32)freq) * 128 / 1000000 +
+                                       ((s32)skew_info->clk_delta) + ((s32)clk_skew_b);
+                       }
+               }
+       } else {
+               if (!skew_info->clk_skew) {
+                       skew_info->clk_skew = (skew_info->t_dqss_max + skew_info->t_dqss_min) / 2
*
+                                               ((s32)skew_info->max_freq) * 128 / 1000000;
+                       if (dramtype == LPDDR4 || dramtype == LPDDR4X) {
+                               if (skew_info->clk_skew < 128)
+                                       skew_info->clk_skew = -128 - ((s32)skew_info->clk_skew);
+                       } else if (skew_info->t_dqss_min < 0) {
+                               if (skew_info->t_dqss_max > 0) {
+                                       if (skew_info->clk_skew < 128)
+                                               skew_info->clk_skew =
+                                                       -128 - ((s32)skew_info->clk_skew);
+                               } else {
+                                       if (skew_info->clk_skew < 96)
```

```
+                                                skew_info->clk_skew =
+                                                        96 - ((s32)skew_info->clk_skew);
+                                }
+                        } else if (skew_info->clk_skew < 160) {
+                                skew_info->clk_skew = -96 - ((s32)skew_info->clk_skew);
+                        }
+                }
+
+                for (i = 0; i < 4; ++i) {
+                        skew_info->dqs[0][i] =
+                                wrlvl_result[0][i] * freq * 128 / 1000000 + skew_info->clk_skew;
+                        skew_info->dqs[1][i] =
+                                wrlvl_result[1][i] * freq * 128 / 1000000 + skew_info->clk_skew;
+                }
+        }
+
+        if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG))
+                printf("clk skew:0x%x\n", skew_info->clk_skew);
+}
+
+static void pctl_modify_trfc(struct ddr_pctl_regs *pctl_regs, struct sdram_cap_info *cap_info,
+                             u32 dram_type, u32 freq, u8 ext_temp_ref, u8 per_bank_ref_en,
+                             u8 derate_en)
+{
+        u32 trfcab_ns, trfcpb_ns, trfc4_ns, tbpr2bpr_ns;
+        u32 trfc, txsnr, trefi;
+        u32 txs_abort_fast = 0;
+        u32 tbpr2bpr = 0;
+        u64 cs0_cap;
+        u32 die_cap;
+        u32 tmp;
+        int i;
+
+        if (dram_type == DDR4 || dram_type == DDR3) {
+                if (per_bank_ref_en)
+                        per_bank_ref_en = 0;
+        }
+
+        cs0_cap = sdram_get_cs_cap(cap_info, 0, dram_type);
+        die_cap = (u32)(cs0_cap >> (20 + (cap_info->bw - cap_info->dbw)));
+
+        switch (dram_type) {
+        case DDR3:
+                if (die_cap <= DIE_CAP_512MBIT)
+                        trfcab_ns = 90;
+                else if (die_cap <= DIE_CAP_1GBIT)
+                        trfcab_ns = 110;
+                else if (die_cap <= DIE_CAP_2GBIT)
+                        trfcab_ns = 160;
+                else if (die_cap <= DIE_CAP_4GBIT)
+                        trfcab_ns = 260;
+                else
+                        trfcab_ns = 350;
+                trfcpb_ns = trfcab_ns;
+                txsnr = MAX(5, ((trfcab_ns + 10) * freq + 999) / 1000);
+                break;
+
+        case DDR4:
+                if (die_cap <= DIE_CAP_2GBIT) {
+                        trfcab_ns = 160;
+                        trfc4_ns = 90;
+                } else if (die_cap <= DIE_CAP_4GBIT) {
+                        trfcab_ns = 260;
+                        trfc4_ns = 110;
+                } else if (die_cap <= DIE_CAP_8GBIT) {
```

```
+                       trfcab_ns = 350;
+                       trfc4_ns = 160;
+               } else {
+                       trfcab_ns = 550;
+                       trfc4_ns = 260;
+               }
+               trfcpb_ns = trfcab_ns;
+               txsnr = ((trfcab_ns + 10) * freq + 999) / 1000;
+               txs_abort_fast = ((trfc4_ns + 10) * freq + 999) / 1000;
+               break;
+
+       case LPDDR3:
+               if (die_cap <= DIE_CAP_4GBIT) {
+                       trfcpb_ns = 60;
+                       trfcab_ns = 130;
+               } else {
+                       trfcpb_ns = 90;
+                       trfcab_ns = 210;
+               }
+               txsnr = MAX(2, ((trfcab_ns + 10) * freq + 999) / 1000);
+               break;
+
+       case LPDDR4:
+               if (die_cap <= DIE_CAP_2GBIT) {
+                       tbpr2bpr_ns = 60;
+                       trfcpb_ns   = 60;
+                       trfcab_ns   = 130;
+               } else if (die_cap <= DIE_CAP_4GBIT) {
+                       tbpr2bpr_ns = 90;
+                       trfcpb_ns   = 90;
+                       trfcab_ns   = 180;
+               } else if (die_cap <= DIE_CAP_8GBIT) {
+                       tbpr2bpr_ns = 90;
+                       trfcpb_ns   = 140;
+                       trfcab_ns   = 280;
+               } else {
+                       tbpr2bpr_ns = 90;
+                       trfcpb_ns   = 190;
+                       trfcab_ns   = 380;
+               }
+               tbpr2bpr = (tbpr2bpr_ns * freq + 999) / 1000;
+               txsnr = MAX(2, ((trfcab_ns + 10) * freq + 999) / 1000);
+               break;
+
+       default:
+               return;
+       }
+
+       if (per_bank_ref_en)
+               trfc = (trfcpb_ns * freq + 999) / 1000;
+       else
+               trfc = (trfcab_ns * freq + 999) / 1000;
+
+       for (i = 0; pctl_regs->pctl[i][0] != 0xffffffff; ++i) {
+               switch (pctl_regs->pctl[i][0]) {
+               case DDR_PCTL2_RFSHTMG:
+                       trefi = (pctl_regs->pctl[i][1] >> 16) & 0xfff;
+                       if (per_bank_ref_en)
+                               trefi *= 2;
+                       else if (!derate_en && (dram_type == DDR4 || dram_type == DDR3))
+                               trefi = ((ext_temp_ref - 1) + trfc) / ext_temp_ref;
+
+                       tmp = pctl_regs->pctl[i][1];
+                       /* t_rfc_min */
+                       tmp &= ~((u32)0x3ff);
```

```
+                       /* t_refi */
+                       tmp &= ~((u32)0xfff << 16);
+                       /* t_rfc_nom_x1_sel */
+                       tmp &= ~((u32)0x1 << 31);
+
+                       tmp |= (per_bank_ref_en << 31) | ((trefi & 0xfff) << 16) |
+                              (((trfc + 1) / 2) & 0x3ff);
+                       pctl_regs->pctl[i][1] = tmp;
+                       break;
+
+               case DDR_PCTL2_RFSHTMG1:
+                       if (dram_type == LPDDR4 || dram_type == LPDDR4X) {
+                               tmp = pctl_regs->pctl[i][1];
+                               /* t_pbr2pbr */
+                               tmp &= ~((u32)0xff << 16);
+                               tmp |= (((tbpr2bpr + 1) / 2) & 0xff) << 16;
+                               pctl_regs->pctl[i][1] = tmp;
+                       }
+                       break;
+
+               case DDR_PCTL2_DRAMTMG8:
+                       if (dram_type == DDR3 || dram_type == DDR4) {
+                               tmp = pctl_regs->pctl[i][1];
+                               /* t_xs_x32 */
+                               tmp &= ~((u32)0x7f);
+                               tmp |= ((txsnr + 63) / 64) & 0x7f;
+
+                               if (dram_type == DDR4) {
+                                       /* t_xs_abort_x32 */
+                                       tmp &= ~((u32)(0x7f << 16));
+                                       tmp |= (((txs_abort_fast + 63) / 64) & 0x7f) << 16;
+                                       /* t_xs_fast_x32 */
+                                       tmp &= ~((u32)(0x7f << 24));
+                                       tmp |= (((txs_abort_fast + 63) / 64) & 0x7f) << 24;
+                               }
+
+                               pctl_regs->pctl[i][1] = tmp;
+                       }
+                       break;
+
+               case DDR_PCTL2_DRAMTMG14:
+                       if (dram_type == LPDDR3 || dram_type == LPDDR4 || dram_type == LPDDR4X) {
+                               tmp = pctl_regs->pctl[i][1];
+                               /* t_xsr */
+                               tmp &= ~((u32)0xfff);
+                               tmp |= ((txsnr + 1) / 2) & 0xfff;
+                               pctl_regs->pctl[i][1] = tmp;
+                       }
+                       break;
+
+               default:
+                       break;
+               }
+       }
+}
+
+static int ddr_set_rate(struct dram_info *dram, struct rk3568_sdram_params *sdram_params, u32
freq,
+                       u32 cur_freq, u32 dst_fsp, u32 dst_fsp_lp4)
+{
+       u32 dest_dll_off, cur_init3, dst_init3, cur_fsp, cur_dll_off;
+       struct rk3568_sdram_params *sdram_params_new;
+       u32 dramtype = sdram_params->base.dramtype;
+       struct rk3568_ddrphy *phy = dram->phy;
+       void __iomem *pctl_base = dram->pctl;
```

```
+       u32 do_ca_training;
+       s32 sig_skew_delta;
+       u32 lp_stat;
+       u32 mr_tmp;
+       int ret;
+
+       sdram_params_new = get_default_sdram_config(freq);
+       sdram_params_new->ch.cap_info.rank = sdram_params->ch.cap_info.rank;
+       sdram_params_new->ch.cap_info.bw = sdram_params->ch.cap_info.bw;
+
+       if ((dramtype == LPDDR4 || dramtype == LPDDR4X) && !dram->cmd_perbit_skew_bp) {
+               do_ca_training = 1;
+       } else {
+               do_ca_training = 0;
+               phy_calc_skew(dram, sdram_params_new, freq, dst_fsp);
+       }
+
+       lp_stat = low_power_update(dram, 0);
+
+       pctl_modify_trfc(&sdram_params_new->pctl_regs, &sdram_params_new->ch.cap_info, dramtype,
+                        freq, dram->ext_temp_ref, dram->per_bank_ref_en, dram->derate_en);
+       pre_set_rate(dram, sdram_params_new, dst_fsp, dst_fsp_lp4);
+
+       rk_setreg(&dram->ddrgrf->ddr_grf_con[0], DFI_INIT_START_BY_GRF_EN);
+       phy_pll_set(dram, sdram_params_new->base.ddr_freq * MHZ, dst_fsp);
+
+       while ((readl(pctl_base + DDR_PCTL2_STAT) & PCTL2_OPERATING_MODE_MASK) ==
+              PCTL2_OPERATING_MODE_SR)
+               continue;
+
+       dest_dll_off = 0;
+       dst_init3 = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT3);
+       if ((dramtype == DDR3 && (dst_init3 & 1)) || (dramtype == DDR4 && !(dst_init3 & 1)))
+               dest_dll_off = 1;
+
+       cur_fsp = readl(pctl_base + DDR_PCTL2_MSTR2) & 0x3;
+       cur_init3 = readl(pctl_base + UMCTL2_REGS_FREQ(cur_fsp) + DDR_PCTL2_INIT3);
+       cur_init3 &= PCTL2_MR_MASK;
+       cur_dll_off = 1;
+       if ((dramtype == DDR3 && !(cur_init3 & 1)) || (dramtype == DDR4 && (cur_init3 & 1)))
+               cur_dll_off = 0;
+
+       if (!cur_dll_off) {
+               if (dramtype == DDR3)
+                       cur_init3 |= 1;
+               else
+                       cur_init3 &= ~1;
+               pctl_write_mr(dram->pctl, 3, 1, cur_init3, dramtype);
+       }
+
+       setbits_le32(pctl_base + DDR_PCTL2_RFSHCTL3, PCTL2_DIS_AUTO_REFRESH);
+       update_refresh_reg(dram);
+
+       enter_sr(dram, 1);
+
+       rk_clrsetreg(&dram->pmugrf->pmu_soc_con0, PMUGRF_CON_DDRPHY_BUFFEREN_MASK,
+                    PMUGRF_CON_DDRPHY_BUFFEREN_EN << PMUGRF_CON_DDRPHY_BUFFEREN_SHIFT);
+       sw_set_req(dram);
+       clrbits_le32(pctl_base + DDR_PCTL2_DFIMISC, PCTL2_DFI_INIT_COMPLETE_EN);
+       sw_set_ack(dram);
+
+       sw_set_req(dram);
+       if ((dramtype == DDR3 || dramtype == DDR4) && dest_dll_off)
+               setbits_le32(pctl_base + DDR_PCTL2_MSTR, PCTL2_DLL_OFF_MODE);
+       else
```

```
+               clrbits_le32(pctl_base + DDR_PCTL2_MSTR, PCTL2_DLL_OFF_MODE);
+
+       setbits_le32(pctl_base + UMCTL2_REGS_FREQ(cur_fsp) + DDR_PCTL2_ZQCTL0,
PCTL2_DIS_SRX_ZQCL);
+       setbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_ZQCTL0,
PCTL2_DIS_SRX_ZQCL);
+       if ((dramtype == LPDDR4 || dramtype == LPDDR4X) && sdram_params_new->ch.cap_info.dbw ==
0)
+               setbits_le32(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_ZQCTL0,
+                            PCTL2_ZQ_RESISTOR_SHARED);
+
+       setbits_le32(pctl_base + DDR_PCTL2_MSTR, PCTL2_FREQUENCY_MODE_FREQ1);
+       clrsetbits_le32(pctl_base + DDR_PCTL2_MSTR2, 0x3, dst_fsp);
+       sw_set_ack(dram);
+
+       if (!do_ca_training) {
+               sig_skew_delta = dramtype == LPDDR3
+                                ? ((s32)freq) * -44800 / 1000000
+                                : freq        *  10240 / 1000000;
+
+               modify_ca_deskew(dram, DESKEW_MDF_ABS_VAL, dram->skew_info.clk_skew,
+                                ((s32)dram->skew_info.clk_skew) - sig_skew_delta, 0, dramtype);
+       }
+
+       rk_setreg(&dram->ddrgrf->ddr_grf_con[0], DFI_INIT_START);
+       while ((readl(pctl_base + DDR_PCTL2_DFISTAT) & PCTL2_DFI_INIT_COMPLETE) ==
+              PCTL2_DFI_INIT_COMPLETE)
+               continue;
+
+       rkclk_set_dpll(dram, freq * MHz / 2);
+       phy_set_fb1xclk_invdela(dram, sdram_params_new->base.ddr_freq);
+       clrsetbits_le32(&phy->reg10_1, FREQ_CHOOSE_T_MASK, dst_fsp << FREQ_CHOOSE_T_SHIFT);
+
+       rk_clrreg(&dram->ddrgrf->ddr_grf_con[0], DFI_INIT_START);
+       while ((readl(pctl_base + DDR_PCTL2_DFISTAT) & PCTL2_DFI_INIT_COMPLETE) !=
+              PCTL2_DFI_INIT_COMPLETE)
+               continue;
+
+       rk_clrsetreg(&dram->pmugrf->pmu_soc_con0, PMUGRF_CON_DDRPHY_BUFFEREN_MASK,
+                    PMUGRF_CON_DDRPHY_BUFFEREN_DIS << PMUGRF_CON_DDRPHY_BUFFEREN_SHIFT);
+
+       update_refresh_reg(dram);
+
+       if (!do_ca_training || dramtype == LPDDR4X) {
+               enter_sr(dram, 0);
+       } else {
+               setbits_le32(pctl_base + DDR_PCTL2_PWRCTL, PCTL2_STAY_IN_SELFREF);
+               clrbits_le32(pctl_base + DDR_PCTL2_PWRCTL, PCTL2_SELFREF_SW);
+               while ((readl(pctl_base + DDR_PCTL2_STAT) & PCTL2_SELFREF_STATE_MASK) >>
+                      PCTL2_SELFREF_STATE_SHIFT != PCTL2_SELFREF_STATE_SR2)
+                       continue;
+       }
+
+       setbits_le32(&phy->reg24, RX_VREF_VALUE_UPDATE);
+       clrbits_le32(&phy->reg24, RX_VREF_VALUE_UPDATE);
+
+       if (do_ca_training) {
+               ret = data_training_ca(dram, dramtype, sdram_params_new->ch.cap_info.rank, freq,
+                                      dst_fsp);
+               if (ret)
+                       return ret;
+
+               phy_calc_skew(dram, sdram_params_new, freq, dst_fsp);
+               record_ca_result(dram, dramtype, sdram_params_new->ch.cap_info.rank);
+
```

```
+               clrbits_le32(pctl_base + DDR_PCTL2_PWRCTL, PCTL2_STAY_IN_SELFREF);
+               while ((readl(pctl_base + DDR_PCTL2_STAT) & PCTL2_OPERATING_MODE_MASK) ==
+                       PCTL2_OPERATING_MODE_SR)
+                        continue;
+       }
+
+       mr_tmp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT4);
+       if (dramtype == LPDDR3) {
+               pctl_write_mr(dram->pctl, 0xf, 1,
+                             (dst_init3 >> PCTL2_LPDDR234_MR1_SHIFT) & PCTL2_MR_MASK, dramtype);
+               pctl_write_mr(dram->pctl, 0xf, 2, dst_init3 & PCTL2_MR_MASK, dramtype);
+               pctl_write_mr(dram->pctl, 0xf, 3,
+                             (mr_tmp >> PCTL2_LPDDR234_MR3_SHIFT) & PCTL2_MR_MASK, dramtype);
+               pctl_write_mr(dram->pctl, 0xf, 11, lp3_odt_value, dramtype);
+       } else if ((dramtype == DDR3) || (dramtype == DDR4)) {
+               if (dramtype == DDR4) {
+                       pctl_write_mr(dram->pctl, 3, 3, mr_tmp & PCTL2_MR_MASK, dramtype);
+                       mr_tmp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT7);
+
+                       pctl_write_mr(dram->pctl, 3, 6,
+                                     (mr_tmp >> PCTL2_DDR4_MR6_SHIFT & PCTL2_MR_MASK) | 0x80,
+                                     dramtype);
+                       pctl_write_mr(dram->pctl, 3, 6,
+                                     (mr_tmp >> PCTL2_DDR4_MR6_SHIFT & PCTL2_MR_MASK) | 0x80,
+                                     dramtype);
+                       pctl_write_mr(dram->pctl, 3, 6,
+                                     mr_tmp >> PCTL2_DDR4_MR6_SHIFT & PCTL2_MR_MASK, dramtype);
+
+                       mr_tmp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT6);
+                       pctl_write_mr(dram->pctl, 3, 4,
+                                     (mr_tmp >> PCTL2_DDR4_MR4_SHIFT) & PCTL2_MR_MASK,
dramtype);
+                       pctl_write_mr(dram->pctl, 3, 5,
+                                     mr_tmp >> PCTL2_DDR4_MR5_SHIFT & PCTL2_MR_MASK, dramtype);
+                       mr_tmp = readl(pctl_base + UMCTL2_REGS_FREQ(dst_fsp) + DDR_PCTL2_INIT4);
+               }
+
+               pctl_write_mr(dram->pctl, 3, 2, ((mr_tmp >> PCTL2_DDR34_MR2_SHIFT) &
PCTL2_MR_MASK),
+                             dramtype);
+
+               if (dramtype == DDR3)
+                       pctl_write_mr(dram->pctl, 3, 3, mr_tmp & PCTL2_MR_MASK, dramtype);
+
+               pctl_write_mr(dram->pctl, 3, 1, dst_init3 & PCTL2_MR_MASK, dramtype);
+
+               if (!dest_dll_off) {
+                       pctl_write_mr(dram->pctl, 3, 0,
+                                     ((dst_init3 >> PCTL2_DDR34_MR0_SHIFT) & PCTL2_MR_MASK) |
+                                             DDR3_DLL_RESET,
+                                     dramtype);
+                       udelay(2);
+               }
+               pctl_write_mr(dram->pctl, 3, 0,
+                             (dst_init3 >> PCTL2_DDR34_MR0_SHIFT & PCTL2_MR_MASK) &
+                                     (~DDR3_DLL_RESET),
+                             dramtype);
+       } else if (dramtype == LPDDR4 || dramtype == LPDDR4X) {
+               pctl_write_mr(dram->pctl, 0xf, 13,
+                             ((mr_tmp >> PCTL2_LPDDR4_MR13_SHIFT & PCTL2_MR_MASK) & (~(BIT(7))))
|
+                                     dst_fsp_lp4 << 7,
+                             dramtype);
+       }
+
```

```
+       clrbits_le32(pctl_base + DDR_PCTL2_RFSHCTL3, PCTL2_DIS_AUTO_REFRESH);
+       update_refresh_reg(dram);
+
+       /* training */
+       ret = high_freq_training(dram, sdram_params_new, dst_fsp);
+       if (ret)
+               return ret;
+
+       low_power_update(dram, lp_stat);
+
+       save_fsp_param(dram, dst_fsp, sdram_params_new);
+
+       return 0;
+}
+
+static void phy_compute_tdqss2dq(struct dram_info *dram, struct rk3568_sdram_params
*sdram_params,
+                                u32 freq)
+{
+       struct skew_info *skew_info = &dram->skew_info;
+       struct rk3568_ddrphy *phy = dram->phy;
+       u32 dq_skew_sum = 0;
+       u32 dqs_skew;
+       int dq;
+
+       if (sdram_params->base.dramtype != LPDDR4 && sdram_params->base.dramtype != LPDDR4X)
+               return;
+
+       for (dq = 0; dq < 8; ++dq) {
+               clrsetbits_le32(&phy->pad_group[0].regc, ABC_LH_CSN_LOOP_INVDELAYSEL_MASK,
+                               dq << ABC_LH_CSN_LOOP_INVDELAYSEL_SHIFT);
+               dq_skew_sum +=
+                       (readl(&phy->pad_group[0].reg1f) & ABC_LH_CSN_VALUE_DQX_INVDELAYSEL_MASK)
>>
+                       ABC_LH_CSN_VALUE_DQX_INVDELAYSEL_SHIFT;
+       }
+
+       clrsetbits_le32(&phy->pad_group[0].regc, ABC_LH_CSN_LOOP_INVDELAYSEL_MASK,
+                       9 << ABC_LH_CSN_LOOP_INVDELAYSEL_SHIFT);
+
+       dqs_skew = (readl(&phy->pad_group[0].reg1f) & ABC_LH_CSN_VALUE_DQX_INVDELAYSEL_MASK) >>
+                  ABC_LH_CSN_VALUE_DQX_INVDELAYSEL_SHIFT;
+
+       skew_info->t_dqss2dq = ((((dq_skew_sum / 8) - dqs_skew) * 1000000) / freq) / 128;
+
+       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG))
+               printf("get tdqqs2dq:%d ps\n", skew_info->t_dqss2dq);
+}
+
+static int ddr_set_rate_for_fsp(struct dram_info *dram, struct rk3568_sdram_params
*sdram_params)
+{
+       struct skew_info *skew_info = &dram->skew_info;
+       u32 dramtype = sdram_params->base.dramtype;
+       struct ddr2_3_4_lp2_3_info *ddr_info;
+       u32 f0, f1, f2, f3, max_freq;
+       int ret;
+
+       ddr_info = get_ddr_drv_odt_info(dramtype);
+       if (!ddr_info)
+               return -1;
+
+       memset((void *)FSP_PARAM_STORE_ADDR, 0, sizeof(fsp_param));
+       memset((void *)&fsp_param, 0, sizeof(fsp_param));
+
```

```
+       f0 = (ddr_info->ddr_freq0_1 >> DDR_FREQ_F0_SHIFT) & DDR_FREQ_MASK;
+       f1 = (ddr_info->ddr_freq0_1 >> DDR_FREQ_F1_SHIFT) & DDR_FREQ_MASK;
+       f2 = (ddr_info->ddr_freq2_3 >> DDR_FREQ_F2_SHIFT) & DDR_FREQ_MASK;
+       f3 = (ddr_info->ddr_freq2_3 >> DDR_FREQ_F3_SHIFT) & DDR_FREQ_MASK;
+
+       max_freq = f0;
+       if (f1 > max_freq)
+               max_freq = f1;
+       if (f2 > max_freq)
+               max_freq = f2;
+       if (f3 > max_freq)
+               max_freq = f3;
+
+       skew_info->max_freq = max_freq;
+       skew_info->t_dqss2dq = 0;
+       skew_info->clk_skew = 0;
+
+       ret = get_wrlvl_val(dram, sdram_params);
+       if (ret) {
+               printascii("get wrlvl value fail\n");
+               return ret;
+       }
+
+       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG)) {
+               printascii("change to: ");
+               printdec(f1);
+               printascii("MHz\n");
+       }
+       setup_data_training(dram, dramtype, 0);
+       ret = ddr_set_rate(dram, sdram_params, f1, sdram_params->base.ddr_freq, 1, 1);
+       if (ret)
+               return ret;
+
+       phy_compute_tdqss2dq(dram, sdram_params, f1);
+
+       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG)) {
+               printascii("change to: ");
+               printdec(f2);
+               printascii("MHz\n");
+       }
+       setup_data_training(dram, dramtype, 0);
+       ret = ddr_set_rate(dram, sdram_params, f2, f1, 2, 0);
+       if (ret)
+               return ret;
+
+       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG)) {
+               printascii("change to: ");
+               printdec(f3);
+               printascii("MHz\n");
+       }
+       setup_data_training(dram, dramtype, 0);
+       ret = ddr_set_rate(dram, sdram_params, f3, f2, 3, 1);
+       if (ret)
+               return ret;
+
+       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG)) {
+               printascii("change to: ");
+               printdec(f0);
+               printascii("MHz(final freq)\n");
+       }
+       setup_data_training(dram, dramtype, 0);
+       return ddr_set_rate(dram, sdram_params, f0, f3, 0, 0);
+}
+
+static void train_logic_clk_gate(struct dram_info *dram, struct rk3568_sdram_params
```

```
*sdram_params)
+{
+       struct rk3568_ddrphy *phy = dram->phy;
+       u32 lp_stat;
+
+       lp_stat = low_power_update(dram, 0);
+
+       if (sdram_params->base.dramtype == LPDDR4 || sdram_params->base.dramtype == LPDDR4X)
+               setbits_le32(&phy->reg8, CMD_RANK_SWITCH_DIS);
+
+       clrbits_le32(&phy->reg20, TRAIN_REG_UPDATE_EN);
+
+       low_power_update(dram, lp_stat);
+}
+
+static void rk3568_dmc_init(struct udevice *dev)
+{
+       struct sdram_head_info_index_v2 *index = (struct sdram_head_info_index_v2 *)common_info;
+       struct rk3568_dmc_plat *plat = dev_get_plat(dev);
+       struct rk3568_sdram_params *sdram_params;
+       struct global_info *gbl_info;
+       int ret = 0;
+
+       dram_info.pctl = (void *)plat->of_plat.reg[0];
+       dram_info.phy  = (void *)plat->of_plat.reg[2];
+
+       dram_info.sgrf = syscon_get_first_range(ROCKCHIP_SYSCON_SGRF);
+       if (IS_ERR(dram_info.sgrf)) {
+               ret = PTR_ERR(dram_info.sgrf);
+               goto error;
+       }
+
+       dram_info.cru = rockchip_get_cru();
+       if (IS_ERR(dram_info.cru)) {
+               ret = PTR_ERR(dram_info.cru);
+               goto error;
+       }
+
+       dram_info.msch = syscon_get_first_range(ROCKCHIP_SYSCON_MSCH);
+       if (IS_ERR(dram_info.msch)) {
+               ret = PTR_ERR(dram_info.msch);
+               goto error;
+       }
+
+       dram_info.ddrgrf = syscon_get_first_range(ROCKCHIP_SYSCON_DDRGRF);
+       if (IS_ERR(dram_info.ddrgrf)) {
+               ret = PTR_ERR(dram_info.ddrgrf);
+               goto error;
+       }
+
+       dram_info.pmugrf = syscon_get_first_range(ROCKCHIP_SYSCON_PMUGRF);
+       if (IS_ERR(dram_info.pmugrf)) {
+               ret = PTR_ERR(dram_info.pmugrf);
+               goto error;
+       }
+
+       /*
+        * TODO: Needed?
+        *
+        * writel(dram_info.sgrf + 0x200, 0x00800080);
+        *  or
+        * writel(dram_info.sgrf + 0x200, 0xffff8280);
+        * writel(dram_info.sgrf + 0x204, 0xffff1240);
+        */
+
```

```
+       if (index->version_info != 2 ||
+           (index->global_index.size != sizeof(struct global_info) / 4) ||
+           /* (index->ddr3_index.size != sizeof(struct ddr2_3_4_lp2_3_info) / 4) || */
+           (index->ddr4_index.size != sizeof(struct ddr2_3_4_lp2_3_info) / 4) ||
+           /* (index->lp3_index.size != sizeof(struct ddr2_3_4_lp2_3_info) / 4) || */
+           (index->lp4_index.size != (sizeof(struct lp4_info) / 4)) ||
+           /* (index->lp4x_index.size != (sizeof(struct lp4_info) / 4)) || */
+           index->global_index.offset == 0 ||
+           /*index->ddr3_index.offset == 0 ||*/
+           index->ddr4_index.offset == 0 ||
+           /*index->lp3_index.offset == 0 ||
+           index->lp4_index.offset == 0 ||
+           index->lp4x_index.offset == 0) {*/
+           index->lp4_index.offset == 0) {
+               printascii("common info error\n");
+               goto error;
+       }
+
+       gbl_info = (struct global_info *)((void *)common_info + index->global_index.offset * 4);
+
+       dram_info.sr_idle = SR_INFO(gbl_info->sr_pd_info);
+       dram_info.pd_idle = PD_INFO(gbl_info->sr_pd_info);
+
+       dram_info.ext_temp_ref    = ext_temp_ref_from_index(gbl_info);
+       dram_info.derate_en       = DDR_DERATE_EN(gbl_info->info_2t);
+       dram_info.per_bank_ref_en = DDR_PER_BANK_REF_EN(gbl_info->info_2t);
+
+       sdram_params = &sdram_configs[0];
+       if (sdram_params->base.dramtype == DDR3 || sdram_params->base.dramtype == DDR4) {
+               if (DDR_2T_INFO(gbl_info->info_2t))
+                       sdram_params->pctl_regs.pctl[0][1] |= 0x1 << 10;
+               else
+                       sdram_params->pctl_regs.pctl[0][1] &= ~(0x1 << 10);
+       }
+
+       ret = sdram_init_detect(&dram_info, sdram_params);
+       if (ret) {
+               if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG)) {
+                       sdram_print_dram_type(sdram_params->base.dramtype);
+                       printascii(", ");
+                       printdec(sdram_params->base.ddr_freq);
+                       printascii("MHz\n");
+               }
+               goto error;
+       }
+
+       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG))
+               print_ddr_info(&dram_info, sdram_params);
+
+       ret = ddr_set_rate_for_fsp(&dram_info, sdram_params);
+       if (ret)
+               goto error;
+
+       train_logic_clk_gate(&dram_info, sdram_params);
+       copy_fsp_param_to_ddr();
+
+       if (IS_ENABLED(CONFIG_RAM_ROCKCHIP_DEBUG))
+               printascii("out\n");
+
+       return;
+error:
+       printf("DRAM init failed! %d\n", ret);
+       hang();
+}
+
```

```
+#endif
+
 static int rk3568_dmc_probe(struct udevice *dev)
 {
+#if defined(CONFIG_TPL_BUILD) || (!defined(CONFIG_TPL) && defined(CONFIG_XPL_BUILD))
+       rk3568_dmc_init(dev);
+#else
        struct dram_info *priv = dev_get_priv(dev);

        priv->pmugrf = syscon_get_first_range(ROCKCHIP_SYSCON_PMUGRF);
+       debug("%s: grf=%p\n", __func__, priv->pmugrf);
        priv->info.base = CFG_SYS_SDRAM_BASE;
-       priv->info.size =
-               rockchip_sdram_size((phys_addr_t)&priv->pmugrf->pmu_os_reg2);
-
+       priv->info.size = rockchip_sdram_size((phys_addr_t)&priv->pmugrf->pmu_os_reg2);
+#endif
        return 0;
 }

@@ -53,4 +4419,14 @@ U_BOOT_DRIVER(dmc_rk3568) = {
        .ops = &rk3568_dmc_ops,
        .probe = rk3568_dmc_probe,
        .priv_auto = sizeof(struct dram_info),
+#if CONFIG_IS_ENABLED(OF_PLATDATA)
+       .plat_auto = sizeof(struct rk3568_dmc_plat),
+#endif
 };
+
+/*
+ * This is there to suppress
+ *
+ * WARNING: the driver rockchip_rk3568_dmc was not found in the driver list
+ */
+DM_DRIVER_ALIAS(rockchip_rk3568_dmc, rockchip_rk3568_dmc)

--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** [PATCH RFC 12/18] clk: rockchip: rk3568: Reduce code size for TPL build**
2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
` (10 preceding siblings ...)
2026-05-17 19:24 ` [PATCH RFC 11/18] ram: rockchip: Add SDRAM initialization support for rk3568 Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 13/18] rockchip: rk356x: Add register spaces to DMC in u-boot.dtsi Pavel Golikov
` (5 subsequent siblings)
17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
To: u-boot
Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
Pavel Golikov

RK3568 has only 59.5Kb usable SRAM for TPL stage. Disable all clock
related functionality for TPL build. Clock driver can still be used to

```
get GRF syscon reference from it.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 drivers/clk/rockchip/clk_rk3568.c | 24 +++++++++++++++++++------
 1 file changed, 18 insertions(+), 6 deletions(-)

diff --git a/drivers/clk/rockchip/clk_rk3568.c b/drivers/clk/rockchip/clk_rk3568.c
index 199bb51fa6f..384d0c750d5 100644
--- a/drivers/clk/rockchip/clk_rk3568.c
+++ b/drivers/clk/rockchip/clk_rk3568.c
@@ -28,6 +28,7 @@ struct rk3568_pmuclk_plat {
 };
 #endif

+#if !IS_ENABLED(CONFIG_TPL_BUILD)
 #define RK3568_CPUCLK_RATE(_rate, _aclk_div, _pclk_div)                 \
 {                                                                       \
        .rate   = _rate##U,                                             \
@@ -2315,6 +2316,7 @@ static ulong rk3568_uart_set_rate(struct rk3568_clk_priv *priv,

        return rk3568_uart_get_rate(priv, clk_id);
 }
+#endif

 static ulong rk3568_clk_get_rate(struct clk *clk)
 {
@@ -2327,6 +2329,7 @@ static ulong rk3568_clk_get_rate(struct clk *clk)
        }

        switch (clk->id) {
+#if !IS_ENABLED(CONFIG_TPL_BUILD)
        case PLL_APLL:
        case ARMCLK:
                rate = rockchip_pll_get_rate(&rk3568_pll_clks[APLL], priv->cru,
@@ -2486,6 +2489,7 @@ static ulong rk3568_clk_get_rate(struct clk *clk)
        case CPLL_25M:
                rate = rk3568_cpll_div_get_rate(priv, clk->id);
                break;
+#endif
        default:
                return -ENOENT;
        }
@@ -2504,6 +2508,7 @@ static ulong rk3568_clk_set_rate(struct clk *clk, ulong rate)
        }

        switch (clk->id) {
+#if !IS_ENABLED(CONFIG_TPL_BUILD)
        case PLL_APLL:
        case ARMCLK:
                if (priv->armclk_hz)
@@ -2673,6 +2678,7 @@ static ulong rk3568_clk_set_rate(struct clk *clk, ulong rate)
        case CPLL_25M:
                ret = rk3568_cpll_div_set_rate(priv, clk->id, rate);
                break;
+#endif
        default:
                return -ENOENT;
        }
@@ -2681,6 +2687,7 @@ static ulong rk3568_clk_set_rate(struct clk *clk, ulong rate)
 };

 #if (CONFIG_IS_ENABLED(OF_CONTROL)) || (!CONFIG_IS_ENABLED(OF_PLATDATA))
+#if !IS_ENABLED(CONFIG_TPL_BUILD)
 static int rk3568_gmac0_src_set_parent(struct clk *clk, struct clk *parent)
```

```
 {
        struct rk3568_clk_priv *priv = dev_get_priv(clk->dev);
@@ -2823,10 +2830,12 @@ static int rk3568_rkvdec_set_parent(struct clk *clk, struct clk *parent)

        return 0;
 }
+#endif

 static int rk3568_clk_set_parent(struct clk *clk, struct clk *parent)
 {
        switch (clk->id) {
+#if !IS_ENABLED(CONFIG_TPL_BUILD)
        case SCLK_GMAC0:
                return rk3568_gmac0_src_set_parent(clk, parent);
        case SCLK_GMAC1:
@@ -2848,6 +2857,7 @@ static int rk3568_clk_set_parent(struct clk *clk, struct clk *parent)
        case SCLK_GMAC1_RGMII_SPEED:
        case SCLK_GMAC1_RMII_SPEED:
                break;
+#endif
        default:
                return -ENOENT;
        }
@@ -2864,6 +2874,7 @@ static struct clk_ops rk3568_clk_ops = {
 #endif
 };

+#if !IS_ENABLED(CONFIG_TPL_BUILD)
 static void rk3568_clk_init(struct rk3568_clk_priv *priv)
 {
        int ret;
@@ -2903,18 +2914,14 @@ static void rk3568_clk_init(struct rk3568_clk_priv *priv)
        priv->ppll_hz = rk3568_pmu_pll_get_rate(priv, PPLL);
        priv->hpll_hz = rk3568_pmu_pll_get_rate(priv, HPLL);
 }
+#endif

 static int rk3568_clk_probe(struct udevice *dev)
 {
        struct rk3568_clk_priv *priv = dev_get_priv(dev);
-#if CONFIG_IS_ENABLED(OF_PLATDATA)
-       struct rk3568_clk_plat *plat = dev_get_plat(dev);
-#endif
+#if !IS_ENABLED(CONFIG_TPL_BUILD)
        int ret;

-#if CONFIG_IS_ENABLED(OF_PLATDATA)
-       priv->cru = map_sysmem(plat->dtd.reg[0], plat->dtd.reg[1]);
-#endif
        priv->grf = syscon_get_first_range(ROCKCHIP_SYSCON_GRF);
        if (IS_ERR(priv->grf))
                return PTR_ERR(priv->grf);
@@ -2927,7 +2934,12 @@ static int rk3568_clk_probe(struct udevice *dev)
                debug("%s clk_set_defaults failed %d\n", __func__, ret);
        else
                priv->sync_kernel = true;
+#endif
+#if CONFIG_IS_ENABLED(OF_PLATDATA)
+       struct rk3568_clk_plat *plat = dev_get_plat(dev);

+       priv->cru = map_sysmem(plat->dtd.reg[0], plat->dtd.reg[1]);
+#endif
        return 0;
 }
```

```
--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** [PATCH RFC 13/18] rockchip: rk356x: Add register spaces to DMC in u-boot.dtsi**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
  ` (11 preceding siblings ...)
  2026-05-17 19:24 ` [PATCH RFC 12/18] clk: rockchip: rk3568: Reduce code size for TPL build Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 20:50 ` Jonas Karlman
  2026-05-17 19:24 ` [PATCH RFC 14/18] rockchip: rk356x: Add DMC related syscons to u-boot.dtsi Pavel Golikov
  ` (4 subsequent siblings)
  17 siblings, 1 reply; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
  Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
  Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
  Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
  Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
  Pavel Golikov

```
Add UTPCTL2 and DDRPHY register spaces, both needed for RK3568's SDRAM
controller initialization.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 arch/arm/dts/rk356x-u-boot.dtsi | 5 ++++-
 1 file changed, 4 insertions(+), 1 deletion(-)

diff --git a/arch/arm/dts/rk356x-u-boot.dtsi b/arch/arm/dts/rk356x-u-boot.dtsi
index 738b9673d35..b9f300a3274 100644
--- a/arch/arm/dts/rk356x-u-boot.dtsi
+++ b/arch/arm/dts/rk356x-u-boot.dtsi
@@ -16,8 +16,11 @@
                u-boot,spl-boot-order = "same-as-spl", &sdmmc0, &sdhci;
        };

-       dmc: dmc {
+       dmc: dmc@fe250000 {
                compatible = "rockchip,rk3568-dmc";
+               reg = <0 0xfe250000 0 0x10000
+                      0 0xfe800000 0 0x10000>;
+
                bootph-all;
        };


--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** [PATCH RFC 14/18] rockchip: rk356x: Add DMC related syscons to u-boot.dtsi**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
  ` (12 preceding siblings ...)

```
  2026-05-17 19:24 ` [PATCH RFC 13/18] rockchip: rk356x: Add register spaces to DMC in u-
boot.dtsi Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 19:24 ` [PATCH RFC 15/18] sysreset: psci: Stop building for XPL Pavel Golikov
                   ` (3 subsequent siblings)
  17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

Add ddrgrf, sgrf and msch syscons in u-boot.dtsi. Those are needed for
RK3568's DRAM controller initialization. Reference them in DMC node.
DRAM controller also needs CRU, so, add reference to it too.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 arch/arm/dts/rk356x-u-boot.dtsi | 25 +++++++++++++++++++++++++
 1 file changed, 25 insertions(+)

diff --git a/arch/arm/dts/rk356x-u-boot.dtsi b/arch/arm/dts/rk356x-u-boot.dtsi
index b9f300a3274..3202c1bf534 100644
--- a/arch/arm/dts/rk356x-u-boot.dtsi
+++ b/arch/arm/dts/rk356x-u-boot.dtsi
@@ -16,8 +16,33 @@
                u-boot,spl-boot-order = "same-as-spl", &sdmmc0, &sdhci;
        };

+       ddrgrf: syscon@fdc40000 {
+               compatible = "rockchip,rk3568-ddrgrf", "syscon";
+               reg = <0x0 0xfdc40000 0x0 0x10000>;
+
+               bootph-all;
+       };
+
+       sgrf: syscon@fdd18000 {
+               compatible = "rockchip,rk3568-sgrf", "syscon";
+               reg = <0x0 0xfdd18000 0x0 0x4000>;
+
+               bootph-all;
+       };
+
+       msch: syscon@fe100000 {
+               compatible = "rockchip,rk3568-msch", "syscon";
+               reg = <0x0 0xfe100000 0x0 0x2000>;
+
+               bootph-all;
+       };
+
        dmc: dmc@fe250000 {
                compatible = "rockchip,rk3568-dmc";
+               rockchip,ddrgrf = <&ddrgrf>;
+               rockchip,cru = <&cru>;
+               rockchip,msch = <&msch>;
+               rockchip,pmugrf = <&pmugrf>;
                reg = <0 0xfe250000 0 0x10000
                       0 0xfe800000 0 0x10000>;


--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** [PATCH RFC 15/18] sysreset: psci: Stop building for XPL**
2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
` (13 preceding siblings ...)
2026-05-17 19:24 ` [PATCH RFC 14/18] rockchip: rk356x: Add DMC related syscons to u-boot.dtsi Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 16/18] rockchip: serial: Add driver alias for rk3568 Pavel Golikov
` (2 subsequent siblings)
17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
To: u-boot
Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
Pavel Golikov

It's highly unlikely that ARM Trusted Firmware BL31 is up and running at
TPL or SPL stages. So, stop building this driver for those stages.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
drivers/sysreset/Makefile | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)

```
diff --git a/drivers/sysreset/Makefile b/drivers/sysreset/Makefile
index b5b99235b6e..070e3634942 100644
--- a/drivers/sysreset/Makefile
+++ b/drivers/sysreset/Makefile
@@ -16,7 +16,7 @@ obj-$(CONFIG_SYSRESET_MPC83XX) += sysreset_mpc83xx.o
 obj-$(CONFIG_SYSRESET_MICROBLAZE) += sysreset_microblaze.o
 obj-$(CONFIG_SYSRESET_OCTEON) += sysreset_octeon.o
 obj-$(CONFIG_$(PHASE_)SYSRESET_PALMAS) += sysreset_palmas.o
-obj-$(CONFIG_SYSRESET_PSCI) += sysreset_psci.o
+obj-$(CONFIG_$(PHASE_)SYSRESET_PSCI) += sysreset_psci.o
 obj-$(CONFIG_SYSRESET_SBI) += sysreset_sbi.o
 obj-$(CONFIG_SYSRESET_SOCFPGA) += sysreset_socfpga.o
 obj-$(CONFIG_SYSRESET_SOCFPGA_SOC64) += sysreset_socfpga_soc64.o
```

--
2.25.1

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** [PATCH RFC 16/18] rockchip: serial: Add driver alias for rk3568**
2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
` (14 preceding siblings ...)
2026-05-17 19:24 ` [PATCH RFC 15/18] sysreset: psci: Stop building for XPL Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 17/18] rockchip: rk356x-generic: Initialize SDRAM at TPL stage Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 18/18] ram: rockchip: rk3568: Limit maximum DDR frequency to 1056 MHz Pavel Golikov
17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)

```
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

To fix warning when building with CONFIG_OF_PLATDATA enabled.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 drivers/serial/serial_rockchip.c | 1 +
 1 file changed, 1 insertion(+)

diff --git a/drivers/serial/serial_rockchip.c b/drivers/serial/serial_rockchip.c
index 8a15173f238..98d4deac3c1 100644
--- a/drivers/serial/serial_rockchip.c
+++ b/drivers/serial/serial_rockchip.c
@@ -54,3 +54,4 @@ DM_DRIVER_ALIAS(rockchip_uart, rockchip_rk3188_uart)
 DM_DRIVER_ALIAS(rockchip_uart, rockchip_rk3288_uart)
 DM_DRIVER_ALIAS(rockchip_uart, rockchip_rk3328_uart)
 DM_DRIVER_ALIAS(rockchip_uart, rockchip_rk3368_uart)
+DM_DRIVER_ALIAS(rockchip_uart, rockchip_rk3568_uart)

--
2.25.1
```

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** [PATCH RFC 17/18] rockchip: rk356x-generic: Initialize SDRAM at TPL stage**
  2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
  ` (15 preceding siblings ...)
  2026-05-17 19:24 ` [PATCH RFC 16/18] rockchip: serial: Add driver alias for rk3568 Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
  2026-05-17 21:06 ` Jonas Karlman
  2026-05-17 19:24 ` [PATCH RFC 18/18] ram: rockchip: rk3568: Limit maximum DDR frequency to 1056 MHz Pavel Golikov
  17 siblings, 1 reply; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)

```
  To: u-boot
  Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
        Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
        Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
        Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
        Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
        Pavel Golikov

Only LPDDR4 initialization is confirmed to be working for now.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 arch/arm/mach-rockchip/Kconfig        | 28 ++++++++++++++++++++++++++-
 arch/arm/mach-rockchip/rk3568/Kconfig | 16 ++++++++++++++++
 common/spl/Kconfig.tpl                |  1 +
 configs/generic-rk3568_defconfig      |  1 +
 4 files changed, 45 insertions(+), 1 deletion(-)

diff --git a/arch/arm/mach-rockchip/Kconfig b/arch/arm/mach-rockchip/Kconfig
index 108713488af..9a6a580025e 100644
--- a/arch/arm/mach-rockchip/Kconfig
```

```
+++ b/arch/arm/mach-rockchip/Kconfig
@@ -413,11 +413,15 @@ config ROCKCHIP_RK3528
 config ROCKCHIP_RK3568
        bool "Support Rockchip RK3568"
        select ARM64
+       select SUPPORT_TPL
+       select TPL_HAVE_INIT_STACK if TPL
+       select TPL_ROCKCHIP_BACK_TO_BROM if TPL
        select SUPPORT_SPL
        select SPL
        select CLK
        select PINCTRL
        select RAM
+       select ROCKCHIP_SDRAM_COMMON
        select REGMAP
        select SYSCON
        select BOARD_LATE_INIT
@@ -433,10 +437,32 @@ config ROCKCHIP_RK3568
        imply RNG_ROCKCHIP
        imply ROCKCHIP_COMMON_BOARD
        imply ROCKCHIP_COMMON_STACK_ADDR
-       imply ROCKCHIP_EXTERNAL_TPL
        imply ROCKCHIP_OTP
+       imply ROCKCHIP_SERIAL if !ROCKCHIP_EXTERNAL_TPL
        imply SPL_ATF_NO_PLATFORM_PARAM if SPL_ATF
        imply SPL_MMC_HS200_SUPPORT if SPL_MMC && MMC_HS200_SUPPORT
+       imply TPL if !ROCKCHIP_EXTERNAL_TPL
+       imply TPL_CLK
+       imply TPL_DM
+       imply TPL_LIBCOMMON_SUPPORT
+       imply TPL_LIBGENERIC_SUPPORT
+       imply TPL_OF_CONTROL
+       imply TPL_OF_PLATDATA
+       imply TPL_RAM
+       imply TPL_ROCKCHIP_COMMON_BOARD
+       imply TPL_SERIAL
+       imply TPL_SYSCON
+       imply SPL_CLK
+       imply SPL_DM
+       imply SPL_DRIVERS_MISC
+       imply SPL_LIBCOMMON_SUPPORT
+       imply SPL_LIBGENERIC_SUPPORT
+       imply SPL_OF_CONTROL
+       imply SPL_RAM
+       imply SPL_REGMAP
+       imply SPL_ROCKCHIP_COMMON_BOARD
+       imply SPL_SERIAL
+       imply SPL_SYSCON
        help
          The Rockchip RK3568 is a ARM-based SoC with quad-core Cortex-A55,
          including NEON and GPU, 512K L3 cache, Mali-G52 based graphics,
diff --git a/arch/arm/mach-rockchip/rk3568/Kconfig b/arch/arm/mach-rockchip/rk3568/Kconfig
index 2730220a18e..893185291d8 100644
--- a/arch/arm/mach-rockchip/rk3568/Kconfig
+++ b/arch/arm/mach-rockchip/rk3568/Kconfig
@@ -61,6 +61,10 @@ config TARGET_ORANGEPI_3B_RK3566

 endchoice

+config SOC_SPECIFIC_OPTIONS # dummy
+       def_bool y
+       select HAS_CUSTOM_SYS_INIT_SP_ADDR
+
 config ROCKCHIP_BOOT_MODE_REG
        default 0xfdc20200
```

```
@@ -70,6 +74,18 @@ config ROCKCHIP_STIMER_BASE
 config SYS_SOC
        default "rk3568"

+config TPL_LDSCRIPT
+       default "arch/arm/mach-rockchip/u-boot-tpl-v8.lds"
+
+config TPL_STACK
+       default 0xfdcd8000
+
+config TPL_SYS_MALLOC_F_LEN
+       default 0x2000
+
+config TPL_TEXT_BASE
+       default 0xfdcc1000
+
 source "board/rockchip/evb_rk3568/Kconfig"
 source "board/anbernic/rgxx3_rk3566/Kconfig"
 source "board/hardkernel/odroid_m1/Kconfig"
diff --git a/common/spl/Kconfig.tpl b/common/spl/Kconfig.tpl
index a535b61ecd3..e64316b8eb3 100644
--- a/common/spl/Kconfig.tpl
+++ b/common/spl/Kconfig.tpl
@@ -127,6 +127,7 @@ config TPL_TEXT_BASE
 config TPL_MAX_SIZE
        hex "Maximum size (in bytes) for the TPL stage"
        default 0x2e000 if ROCKCHIP_RK3399
+       default 0xe800 if ROCKCHIP_RK3568
        default 0x8000 if ROCKCHIP_RK3288 || ROCKCHIP_RV1126
        default 0x7000 if ROCKCHIP_RK322X || ROCKCHIP_RK3328 || ROCKCHIP_RK3368
        default 0x2800 if ROCKCHIP_PX30
diff --git a/configs/generic-rk3568_defconfig b/configs/generic-rk3568_defconfig
index a33c3af9255..4510fa94775 100644
--- a/configs/generic-rk3568_defconfig
+++ b/configs/generic-rk3568_defconfig
@@ -81,3 +81,4 @@ CONFIG_USB_GADGET=y
 CONFIG_USB_GADGET_DOWNLOAD=y
 CONFIG_USB_FUNCTION_ROCKUSB=y
 CONFIG_ERRNO_STR=y
+CONFIG_RAM_ROCKCHIP_LPDDR4=y

--
2.25.1
```

^ permalink raw reply related [**flat**|nested] 22+ messages in thread

---

*** [PATCH RFC 18/18] ram: rockchip: rk3568: Limit maximum DDR frequency to 1056 MHz**
2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
` (16 preceding siblings ...)
2026-05-17 19:24 ` [PATCH RFC 17/18] rockchip: rk356x-generic: Initialize SDRAM at TPL stage Pavel Golikov
@ 2026-05-17 19:24 ` Pavel Golikov
17 siblings, 0 replies; 22+ messages in thread
From: Pavel Golikov @ 2026-05-17 19:24 UTC (permalink / raw)
To: u-boot
Cc: Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
Jonas Karlman, Casey Connolly, Kuan-Wei Chiu,
Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut,
Pavel Golikov

RK3566 has the same DDR memory controller as RK3568 has. The only
difference is a maximum supported DDR frequency.

Limit maximum DDR frequency in DDR loader params, so DDR controller can
be initialized on both RK3566 and RK3568 SoCs.

Signed-off-by: Pavel Golikov <paullo612@ya.ru>
---
 drivers/ram/rockchip/sdram-rk3568-loader_params.inc | 4 ++--
 1 file changed, 2 insertions(+), 2 deletions(-)

```
diff --git a/drivers/ram/rockchip/sdram-rk3568-loader_params.inc b/drivers/ram/rockchip/sdram-
rk3568-loader_params.inc
index d9beea3519a..3fcc70d8495 100644
--- a/drivers/ram/rockchip/sdram-rk3568-loader_params.inc
+++ b/drivers/ram/rockchip/sdram-rk3568-loader_params.inc
@@ -17,7 +17,7 @@
 0, 0, /* reserved */

 /* ddr4 */
-(1560 << DDR_FREQ_F0_SHIFT) | (324 << DDR_FREQ_F1_SHIFT),
+(1056 << DDR_FREQ_F0_SHIFT) | (324 << DDR_FREQ_F1_SHIFT),
 (528 << DDR_FREQ_F2_SHIFT) | (780 << DDR_FREQ_F3_SHIFT),
 (0 << DDR_FREQ_F4_SHIFT) | (0 << DDR_FREQ_F5_SHIFT),
 /* drv when odt on */
@@ -39,7 +39,7 @@
        (0x1 << PHY_CLK_SR_SHIFT),

 /* lpddr4 */
-(1560 << DDR_FREQ_F0_SHIFT) | (324 << DDR_FREQ_F1_SHIFT),
+(1056 << DDR_FREQ_F0_SHIFT) | (324 << DDR_FREQ_F1_SHIFT),
 (528 << DDR_FREQ_F2_SHIFT) | (780 << DDR_FREQ_F3_SHIFT),
 (0 << DDR_FREQ_F4_SHIFT) | (0 << DDR_FREQ_F5_SHIFT),
```

--
2.25.1

^ permalink raw reply related [flat|nested] 22+ messages in thread

---

*** Re: [PATCH RFC 04/18] rockchip: rk3568: syscon: Add MSCH, SGRF and DDRGRF support**
2026-05-17 19:24 ` [PATCH RFC 04/18] rockchip: rk3568: syscon: Add MSCH, SGRF and DDRGRF support Pavel Golikov
@ 2026-05-17 20:42 ` Jonas Karlman
0 siblings, 0 replies; 22+ messages in thread
From: Jonas Karlman @ 2026-05-17 20:42 UTC (permalink / raw)
To: Pavel Golikov
Cc: u-boot@lists.denx.de, Lukasz Majewski, Simon Glass,
Philipp Tomsich, Kever Yang, Tom Rini, Peng Fan, Patrice Chotard,
Yao Zi, Finley Xiao, Casey Connolly, Kuan-Wei Chiu,
Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski,
Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut

Hi Pavel,

On 5/17/2026 9:24 PM, Pavel Golikov wrote:
> Add support for the MSCH, SGRF and DDRGRF register regions in syscon,
> which are necessary for RK3568's DRAM controller initialization.
>
> Signed-off-by: Pavel Golikov <paullo612@ya.ru>
> ---
> arch/arm/include/asm/arch-rockchip/clock.h | 1 +

```
>  arch/arm/mach-rockchip/rk3568/syscon_rk3568.c | 26 +++++++++++++++++++++++++-
>  2 files changed, 26 insertions(+), 1 deletion(-)
>
> diff --git a/arch/arm/include/asm/arch-rockchip/clock.h b/arch/arm/include/asm/arch-rockchip/
clock.h
> index 95b08bfd046..a6b399da4ff 100644
> --- a/arch/arm/include/asm/arch-rockchip/clock.h
> +++ b/arch/arm/include/asm/arch-rockchip/clock.h
> @@ -39,6 +39,7 @@ enum {
>       ROCKCHIP_SYSCON_PIPE_PHY2_GRF,
>       ROCKCHIP_SYSCON_VOP_GRF,
>       ROCKCHIP_SYSCON_VO_GRF,
> +     ROCKCHIP_SYSCON_DDRGRF,
>  };
>
>  /* Standard Rockchip clock numbers */
> diff --git a/arch/arm/mach-rockchip/rk3568/syscon_rk3568.c b/arch/arm/mach-rockchip/rk3568/
syscon_rk3568.c
> index b9d59a4721e..7f6caf9ca67 100644
> --- a/arch/arm/mach-rockchip/rk3568/syscon_rk3568.c
> +++ b/arch/arm/mach-rockchip/rk3568/syscon_rk3568.c
> @@ -8,8 +8,11 @@
>  #include <asm/arch-rockchip/clock.h>
>
>  static const struct udevice_id rk3568_syscon_ids[] = {
> -     { .compatible = "rockchip,rk3568-grf", .data = ROCKCHIP_SYSCON_GRF },
> +     { .compatible = "rockchip,rk3568-grf",    .data = ROCKCHIP_SYSCON_GRF    },
>       { .compatible = "rockchip,rk3568-pmugrf", .data = ROCKCHIP_SYSCON_PMUGRF },
> +     { .compatible = "rockchip,rk3568-msch",   .data = ROCKCHIP_SYSCON_MSCH   },
> +     { .compatible = "rockchip,rk3568-sgrf",   .data = ROCKCHIP_SYSCON_SGRF   },
> +     { .compatible = "rockchip,rk3568-ddrgrf", .data = ROCKCHIP_SYSCON_DDRGRF },
```

Please do not add more mappings, all ROCKCHIP_SYSCON usage should
instead be removed. See [1] and [2] for some uncompleted work.

Please define constants instead of trying to do dynamic lookup. Drivers
that typically need to know these reg addresses are already specific for
a single SoC or needs to be known at compile-time to save space.

No need to waste cpu cycles at runtime trying to resolve them :-)

[1] https://source.denx.de/u-boot/contributors/kwiboo/u-boot/-/commits/optimize
[2] https://source.denx.de/u-boot/contributors/kwiboo/u-boot/-/commit/
d1601adc6b0bb092b66863e0c515dda5746c6f6f

Regards,
Jonas

```
>       { }
>  };
>
> @@ -37,4 +40,25 @@ U_BOOT_DRIVER(rockchip_rk3568_pmugrf) = {
>       .of_match = rk3568_syscon_ids + 1,
>       .bind = rk3568_syscon_bind_of_plat,
>  };
> +
> +U_BOOT_DRIVER(rockchip_rk3568_msch) = {
> +     .name = "rockchip_rk3568_msch",
> +     .id = UCLASS_SYSCON,
> +     .of_match = rk3568_syscon_ids + 2,
> +     .bind = rk3568_syscon_bind_of_plat,
> +};
> +
> +U_BOOT_DRIVER(rockchip_rk3568_sgrf) = {
> +     .name = "rockchip_rk3568_sgrf",
```

```
> +     .id = UCLASS_SYSCON,
> +     .of_match = rk3568_syscon_ids + 3,
> +     .bind = rk3568_syscon_bind_of_plat,
> +};
> +
> +U_BOOT_DRIVER(rockchip_rk3568_ddrgrf) = {
> +     .name = "rockchip_rk3568_ddrgrf",
> +     .id = UCLASS_SYSCON,
> +     .of_match = rk3568_syscon_ids + 4,
> +     .bind = rk3568_syscon_bind_of_plat,
> +};
>  #endif
>
```

^ permalink raw reply [**flat**|nested] 22+ messages in thread

---

*** Re: [PATCH RFC 13/18] rockchip: rk356x: Add register spaces to DMC in u-boot.dtsi**
2026-05-17 19:24 ` [PATCH RFC 13/18] rockchip: rk356x: Add register spaces to DMC in u-boot.dtsi Pavel Golikov
@ 2026-05-17 20:50 ` Jonas Karlman
0 siblings, 0 replies; 22+ messages in thread
From: Jonas Karlman @ 2026-05-17 20:50 UTC (permalink / raw)
To: Pavel Golikov
Cc: u-boot, Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang, Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao, Casey Connolly, Kuan-Wei Chiu, Varadarajan Narayanan, Quentin Schulz, Lukasz Czechowski, Mattijs Korpershoek, Heinrich Schuchardt, Marek Vasut

Hi Pavel,

On 5/17/2026 9:24 PM, Pavel Golikov wrote:
> Add UTPCTL2 and DDRPHY register spaces, both needed for RK3568's SDRAM
> controller initialization.
>
> Signed-off-by: Pavel Golikov <paullo612@ya.ru>
> ---
> arch/arm/dts/rk356x-u-boot.dtsi | 5 ++++-
> 1 file changed, 4 insertions(+), 1 deletion(-)
>
> diff --git a/arch/arm/dts/rk356x-u-boot.dtsi b/arch/arm/dts/rk356x-u-boot.dtsi
> index 738b9673d35..b9f300a3274 100644
> --- a/arch/arm/dts/rk356x-u-boot.dtsi
> +++ b/arch/arm/dts/rk356x-u-boot.dtsi
> @@ -16,8 +16,11 @@
>   u-boot,spl-boot-order = "same-as-spl", &sdmmc0, &sdhci;
>   };
>
> -   dmc: dmc {
> +   dmc: dmc@fe250000 {
>   compatible = "rockchip,rk3568-dmc";
> +   reg = <0 0xfe250000 0 0x10000
> +   0 0xfe800000 0 0x10000>;
> +

Please define these as constants in the driver, no need to waste space
or time adding these TPL only information unless we also have a proper
dt-bindings for the dram controller.

Regards,
Jonas

>   bootph-all;
```
```

```
>       };
>
>
```

^ permalink raw reply [**flat**|nested] 22+ messages in thread

---

*** Re: [PATCH RFC 17/18] rockchip: rk356x-generic: Initialize SDRAM at TPL stage**
2026-05-17 19:24 ` [PATCH RFC 17/18] rockchip: rk356x-generic: Initialize SDRAM at TPL stage
Pavel Golikov
@ 2026-05-17 21:06 ` Jonas Karlman
0 siblings, 0 replies; 22+ messages in thread
From: Jonas Karlman @ 2026-05-17 21:06 UTC (permalink / raw)
To: Pavel Golikov
Cc: u-boot, Lukasz Majewski, Simon Glass, Philipp Tomsich, Kever Yang,
Tom Rini, Peng Fan, Patrice Chotard, Yao Zi, Finley Xiao,
Casey Connolly, Kuan-Wei Chiu, Varadarajan Narayanan,
Quentin Schulz, Lukasz Czechowski, Mattijs Korpershoek,
Heinrich Schuchardt, Marek Vasut

Hi Pavel,

On 5/17/2026 9:24 PM, Pavel Golikov wrote:
> Only LPDDR4 initialization is confirmed to be working for now.
>
> Signed-off-by: Pavel Golikov <paullo612@ya.ru>
> ---
> arch/arm/mach-rockchip/Kconfig | 28 +++++++++++++++++++++++++++-
> arch/arm/mach-rockchip/rk3568/Kconfig | 16 +++++++++++++++++
> common/spl/Kconfig.tpl | 1 +
> configs/generic-rk3568_defconfig | 1 +
> 4 files changed, 45 insertions(+), 1 deletion(-)
>
> diff --git a/arch/arm/mach-rockchip/Kconfig b/arch/arm/mach-rockchip/Kconfig
> index 108713488af..9a6a580025e 100644
> --- a/arch/arm/mach-rockchip/Kconfig
> +++ b/arch/arm/mach-rockchip/Kconfig
> @@ -413,11 +413,15 @@ config ROCKCHIP_RK3528
> config ROCKCHIP_RK3568
> bool "Support Rockchip RK3568"
> select ARM64
> + select SUPPORT_TPL
> + select TPL_HAVE_INIT_STACK if TPL
> + select TPL_ROCKCHIP_BACK_TO_BROM if TPL
> select SUPPORT_SPL
> select SPL
> select CLK
> select PINCTRL
> select RAM
> + select ROCKCHIP_SDRAM_COMMON
> select REGMAP
> select SYSCON
> select BOARD_LATE_INIT
> @@ -433,10 +437,32 @@ config ROCKCHIP_RK3568
> imply RNG_ROCKCHIP
> imply ROCKCHIP_COMMON_BOARD
> imply ROCKCHIP_COMMON_STACK_ADDR
> - imply ROCKCHIP_EXTERNAL_TPL

Not until this has been proven to work for all current supported dram
types and configurations, please select TPL in board specific defconfigs.

> imply ROCKCHIP_OTP
> + imply ROCKCHIP_SERIAL if !ROCKCHIP_EXTERNAL_TPL

```
>       imply SPL_ATF_NO_PLATFORM_PARAM if SPL_ATF
>       imply SPL_MMC_HS200_SUPPORT if SPL_MMC && MMC_HS200_SUPPORT
> +     imply TPL if !ROCKCHIP_EXTERNAL_TPL
> +     imply TPL_CLK
> +     imply TPL_DM
> +     imply TPL_LIBCOMMON_SUPPORT
> +     imply TPL_LIBGENERIC_SUPPORT
> +     imply TPL_OF_CONTROL
> +     imply TPL_OF_PLATDATA
> +     imply TPL_RAM
> +     imply TPL_ROCKCHIP_COMMON_BOARD
> +     imply TPL_SERIAL
> +     imply TPL_SYSCON
> +     imply SPL_CLK
> +     imply SPL_DM
> +     imply SPL_DRIVERS_MISC
> +     imply SPL_LIBCOMMON_SUPPORT
> +     imply SPL_LIBGENERIC_SUPPORT
> +     imply SPL_OF_CONTROL
> +     imply SPL_RAM
> +     imply SPL_REGMAP
> +     imply SPL_ROCKCHIP_COMMON_BOARD
> +     imply SPL_SERIAL
> +     imply SPL_SYSCON
```

These should all be sorted alphanumerically, also this is adding SPL related implys, probably something that should be in a separate patch?

```
>       help
>         The Rockchip RK3568 is a ARM-based SoC with quad-core Cortex-A55,
>         including NEON and GPU, 512K L3 cache, Mali-G52 based graphics,
> diff --git a/arch/arm/mach-rockchip/rk3568/Kconfig b/arch/arm/mach-rockchip/rk3568/Kconfig
> index 2730220a18e..893185291d8 100644
> --- a/arch/arm/mach-rockchip/rk3568/Kconfig
> +++ b/arch/arm/mach-rockchip/rk3568/Kconfig
> @@ -61,6 +61,10 @@ config TARGET_ORANGEPI_3B_RK3566
>
>  endchoice
>
> +config SOC_SPECIFIC_OPTIONS # dummy
> +     def_bool y
> +     select HAS_CUSTOM_SYS_INIT_SP_ADDR
> +
>  config ROCKCHIP_BOOT_MODE_REG
>       default 0xfdc20200
>
> @@ -70,6 +74,18 @@ config ROCKCHIP_STIMER_BASE
>  config SYS_SOC
>       default "rk3568"
>
> +config TPL_LDSCRIPT
> +     default "arch/arm/mach-rockchip/u-boot-tpl-v8.lds"
> +
> +config TPL_STACK
> +     default 0xfdcd8000
> +
> +config TPL_SYS_MALLOC_F_LEN
> +     default 0x2000
> +
> +config TPL_TEXT_BASE
> +     default 0xfdcc1000
> +
>  source "board/rockchip/evb_rk3568/Kconfig"
>  source "board/anbernic/rgxx3_rk3566/Kconfig"
>  source "board/hardkernel/odroid_m1/Kconfig"
```

```
> diff --git a/common/spl/Kconfig.tpl b/common/spl/Kconfig.tpl
> index a535b61ecd3..e64316b8eb3 100644
> --- a/common/spl/Kconfig.tpl
> +++ b/common/spl/Kconfig.tpl
> @@ -127,6 +127,7 @@ config TPL_TEXT_BASE
>  config TPL_MAX_SIZE
>       hex "Maximum size (in bytes) for the TPL stage"
>       default 0x2e000 if ROCKCHIP_RK3399
> +     default 0xe800 if ROCKCHIP_RK3568
>       default 0x8000 if ROCKCHIP_RK3288 || ROCKCHIP_RV1126
>       default 0x7000 if ROCKCHIP_RK322X || ROCKCHIP_RK3328 || ROCKCHIP_RK3368
>       default 0x2800 if ROCKCHIP_PX30
> diff --git a/configs/generic-rk3568_defconfig b/configs/generic-rk3568_defconfig
> index a33c3af9255..4510fa94775 100644
> --- a/configs/generic-rk3568_defconfig
> +++ b/configs/generic-rk3568_defconfig
> @@ -81,3 +81,4 @@ CONFIG_USB_GADGET=y
>  CONFIG_USB_GADGET_DOWNLOAD=y
>  CONFIG_USB_FUNCTION_ROCKUSB=y
>  CONFIG_ERRNO_STR=y
> +CONFIG_RAM_ROCKCHIP_LPDDR4=y
```

This generic-rk3568 target is explicitly created to be generic with
dynamic DRAM type lookup. So we cannot add explicit DRAM type support
for this target.

Regards,
Jonas

^ permalink raw reply [**flat**|nested] 22+ messages in thread

---

end of thread, other threads:[~2026-05-17 21:06 UTC | newest]

Thread overview: 22+ messages (download: mbox.gz follow: Atom feed
-- links below jump to the message on this page --

2026-05-17 19:24 [PATCH RFC 00/18] rockchip: rk3568: add DRAM controller driver and initialize DRAM at TPL stage Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 01/18] clk: rockchip: rk3568: Fix building with OF_PLATDATA enabled Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 02/18] clk: rockchip: rk3568: Fix probe " Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 03/18] rockchip: rk3568: syscon: Add PMUGRF support for OF_PLATDATA Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 04/18] rockchip: rk3568: syscon: Add MSCH, SGRF and DDRGRF support Pavel Golikov
2026-05-17 20:42 ` Jonas Karlman
2026-05-17 19:24 ` [PATCH RFC 05/18] ram: rockchip: Update ddr pctl regs for px30 Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 06/18] ram: rockchip: Add rk3568 lpddr4 support Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 07/18] ram: rockchip: Add rk3568 ddr4 support Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 08/18] ram: rockchip: Add rk3568 ddr loader params Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 09/18] ram: rockchip: Add 4 ranks support to controller code for PX30 Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 10/18] ram: rockchip: Add missing bits to common code for sdram driver Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 11/18] ram: rockchip: Add SDRAM initialization support for rk3568 Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 12/18] clk: rockchip: rk3568: Reduce code size for TPL build Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 13/18] rockchip: rk356x: Add register spaces to DMC in u-boot.dtsi Pavel Golikov
2026-05-17 20:50 ` Jonas Karlman
2026-05-17 19:24 ` [PATCH RFC 14/18] rockchip: rk356x: Add DMC related syscons to u-boot.dtsi Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 15/18] sysreset: psci: Stop building for XPL Pavel Golikov
2026-05-17 19:24 ` [PATCH RFC 16/18] rockchip: serial: Add driver alias for rk3568 Pavel Golikov

2026-05-17 19:24 ` [PATCH RFC 17/18] rockchip: rk356x-generic: Initialize SDRAM at TPL stage Pavel Golikov
2026-05-17 21:06 ` Jonas Karlman
2026-05-17 19:24 ` [PATCH RFC 18/18] ram: rockchip: rk3568: Limit maximum DDR frequency to 1056 MHz Pavel Golikov

---

This is a public inbox, see mirroring instructions
for how to clone and mirror all data and code used for this inbox;
as well as URLs for NNTP newsgroup(s).